SIRC-1 CPU Reference Manual

Featuring the SIRCIS Instruction Set

Technical Specifications

Architecture:	16-bit RISC Load/Store
Address Bus:	24-bit word address
Data Bus:	16-bit
Instruction Size:	32-bit (fixed)
Registers:	16 x 16-bit registers
Execution:	6-stage execution phases

Version 1.0

June 26, 2026

About This Manual

This reference manual provides comprehensive documentation for the SIRC-1 CPU architecture and the SIRCIS (SIRC Instruction Set) instruction set. It is intended for:

- Assembly language programmers
- Compiler and toolchain developers
- Hardware designers and implementers
- System software developers

Document Conventions

Throughout this manual, the following conventions are used:

- Register names appear in typewriter font: `r1`, `ph`, `sr`
- Hexadecimal values are prefixed with `0x`: `0x1A`
- Immediate values are prefixed with `#`: `#123`
- Instruction mnemonics appear in bold typewriter font: **`ADDI`**
- Addressing modes use parentheses for indirection: `(#4, a)`

Contents

List of Tables

List of Figures

Part I

Architecture Overview

Chapter 1

Introduction

1.1 Overview

The SIRC-1 is a 16-bit RISC processor ideal for general purpose computing.
It implements the SIRCIS (SIRC Instruction Set) instruction set architecture, which provides a powerful set of operations while still ensuring simplicity of CPU implementation.

1.2 Design Philosophy

1.2.1 RISC Principles

The SIRC-1 adheres to Reduced Instruction Set Computer (RISC) design principles:

- **Load/Store Architecture:** All ALU operations work exclusively on registers. Memory access is performed only through dedicated load and store instructions.
- **Fixed Instruction Length:** All instructions are exactly 32 bits (2 words) in length, simplifying instruction fetch and decode logic.
- **Simple Instruction Formats:** Only three instruction formats are supported: Immediate, Short Immediate with Shift, and Register.
- **Register-Rich:** Sixteen addressable registers provide ample workspace for computations without frequent memory access.
- **Consistent Execution Time:** All instructions use exactly 6 execution phases and complete in 6 clock cycles when no bus wait states are required, eliminating the need for microcode and simplifying hardware implementation.

1.2.2 Design Goals

The SIRC-1 was designed with several key goals in order of importance:

Simplicity The CPU should be simple enough to be implemented in hardware without requiring complex microcode engines or multi-level instruction decoders.

Developer Ergonomics It should have powerful instruction encodings that include conditional execution and bit shifting that can help reduce the number of instructions and branches to keep track of.

Performance While simple, the design should allow for reasonable performance through efficient instruction encoding and the potential for future enhancements like pipelining.

Memory Protection Basic privilege levels and memory segmentation provide rudimentary protection for system software.

1.2.3 Applications

The SIRC-1 is designed for ideal for general purpose computing, such as the core of a microcomputer.
It should not be used in memory constrained environments, as the fixed length instructions and lack of byte addressing can potentially mean larger program sizes.

1.3 Key Features

1.3.1 Architecture Characteristics

Feature	Specification
Data Width	16-bit
Addressing	Word Addressing
Address Bus	24-bit word address
Instruction Size	32-bit fixed
General Purpose Registers	7 (r1–r7)
Address Register Pairs	4 (l, a, s, p)
Total Addressable Registers	16
Instruction Formats	3
Addressing Modes	7
Condition Codes	16

Table 1.1: SIRC-1 CPU Specifications

1.3.2 Execution Model

The SIRC-1 executes every instruction through six sequential execution phases (normally one phase per clock cycle). For a full description of each phase, see Chapter 2.

This fixed six-stage model means that all instructions take 6 clock cycles to complete when bus operations do not require wait states, regardless of their complexity. A bus device may hold the current phase by delaying bus acknowledgement, adding wait cycles without adding execution phases. While this may seem wasteful for simple register operations that don't require memory access, it dramatically simplifies the hardware design by eliminating the need for variable-length execution sequences or microcode.

1.3.3 Coprocessors

The SIRC-1 is made up of multiple coprocessors (also known as modules) that all share the same address/data bus and register file. Only one coprocessor can be active at one time and are usually activated using the COP instructions (except for the "exception unit" which can be activated when physical pins are asserted on the chip). All implementations of the SIRC-1 will have the "processing unit" as coprocessor 0x0, the "exception unit" as coprocessor 0x1, and the DMA unit as coprocessor 0x2. Other coprocessors are optional and will vary depending on which SIRC-1 model you have. The instruction set supports up to 16 coprocessors. The coprocessor IDs are stable between models, so that programs are cross compatible. For example, coprocessor 0x3 will always be the Integer Maths Unit. This means if the CPU model is missing an optional coprocessor, and a program requires it, the CPU will raise an exception so that the behaviour can be emulated in software where practical.

ID	Name	Optional	Purpose
0x0	Processing Unit	No	Executes instructions
0x1	Exception Unit	No	Handles interrupts and exceptions
0x2	DMA Unit	No	Transfers blocks of data via the bus
0x3	Integer Maths Unit	Yes	Multiplies and divides integer values
0x4–0xF	Reserved/model use	Yes	Defined by future or model-specific manuals

Table 1.2: List of coprocessors

Each coprocessor can execute 16 opcodes, the first 8 (0x0-0x7) can be called in any mode. The last 8 (0x8-0xF) can only be called in supervisor mode.

1.3.4 Privilege Levels

The SIRC-1 supports two privilege levels:

Supervisor Mode Full access to all registers and instructions. Can set the high address-register components, access privileged status register bits, and execute privileged coprocessor operations.

Protected Mode Restricted access for user programs. Cannot write to the high address-register components (providing memory segmentation/protection), cannot access privileged status register bits, and cannot execute privileged coprocessor instructions.

The privilege level is controlled by the Protected Mode bit in the status register.

1.4 Memory Organization

1.4.1 Address Space

The SIRC-1 uses word addressing, where each address refers to a 16-bit word. With a 24-bit address space, this allows access to 2^24 = 16,777,216 words, or 32 megabytes of external byte

storage. Addresses are formed by combining the low 8 bits of the high address-register component with the full 16-bit low address-register component:

$$\text{Address} = (\text{RegHigh}[7:0] \ll 16) \mid \text{RegLow}[15:0] \tag{1.1}$$

Note that the upper 8 bits of the high register (bits 15–8) are not used for addressing but are available for storage, enabling techniques like tagged pointers.

For normative byte order, multi-word storage, exception vector layout, immediate interpretation, and wraparound rules, see Chapter 4.

1.4.2 Memory Map

The CPU expects the exception vector table to begin at word address `0x000000`. Each vector-table entry occupies two consecutive 16-bit words: the target address high word followed by the target address low word.

Region	Vector IDs	Word addresses	Purpose
Reset	0x00	`0x000000`–`0x000001`	Initial program counter after reset
Faults	0x01–0x0F	`0x000002`–`0x00001F`	Fault and reserved privileged exception vectors
Hardware	0x10, 0x20, 0x30, 0x40, 0x50	`0x000020`–`0x0000A1`	Hardware exception vectors, spaced by vector ID
User traps	0x60–0xFF	`0x0000C0`–`0x0001FF`	User software exception vectors

Table 1.3: Exception Vector Table Regions

Beyond these reserved regions, the memory layout is flexible and can be defined by system software.

1.4.3 Alignment

- All memory addresses refer to 16-bit words
- There are no alignment restrictions for general memory I/O
- All instructions span 2 words and must begin at an even word address
- Unaligned instruction fetches (odd word address) trigger an alignment fault exception

1.5 Document Organization

This manual is organized into three main parts:

Part I: Architecture Overview Covers the register model, status register, and overall CPU architecture.

Part II: Instruction Set Architecture Details instruction formats, addressing modes, shift operations, and condition codes.

Part III: SIRCIS Instruction Reference Provides detailed documentation for each instruction, including encoding, operation, and examples.

Appendices provide quick reference materials including complete opcode maps, timing diagrams, and notes on undocumented instructions.

Chapter 2

CPU Architecture

2.1 Overview

The SIRC-1 CPU implements a combinatorial logic design with no microcode, utilizing six sequential execution phases to execute instructions in 6 clock cycles when no bus wait states are required. The architecture is composed of several key functional units that work together to fetch, decode, execute, and complete instructions.

2.2 Six-Stage Execution Phases

All SIRCIS instructions follow a six-phase execution model and complete in 6 clock cycles when no bus wait states are required. A phase that performs a bus operation may be held until the bus acknowledges it. This fixed execution sequence greatly simplifies software development and hardware design.

1. **Instruction Fetch (High Word)** – 1 cycle
    - Fetch bits 31–16 from memory address in PC
    - Increment PC by 1
2. **Instruction Fetch (Low Word)** – 1 cycle
    - Fetch bits 15–0 from memory address in PC
    - Increment PC by 1
    - Instruction is now fully available for decode
3. **Decode and Register Fetch** – 1 cycle
    - Decode instruction opcode and operands
    - Read source registers (optionally shifted)
    - Evaluate condition codes
4. **Execute and Address Calculation** – 1 cycle
    - Perform ALU operation, and/or
    - Calculate effective memory address

5. **Memory Access** – 1 cycle
    - Read from memory (LOAD), or
    - Write to memory (STOR), or
    - No operation (register-only instructions)
6. **Write Back** – 1 cycle
    - Write result to destination register
    - Update status register flags if applicable
    - Update address registers for post-increment/pre-decrement

2.3 Architectural Components

Figure 2.1 illustrates the major functional units within the SIRC-1 CPU and their interconnections.

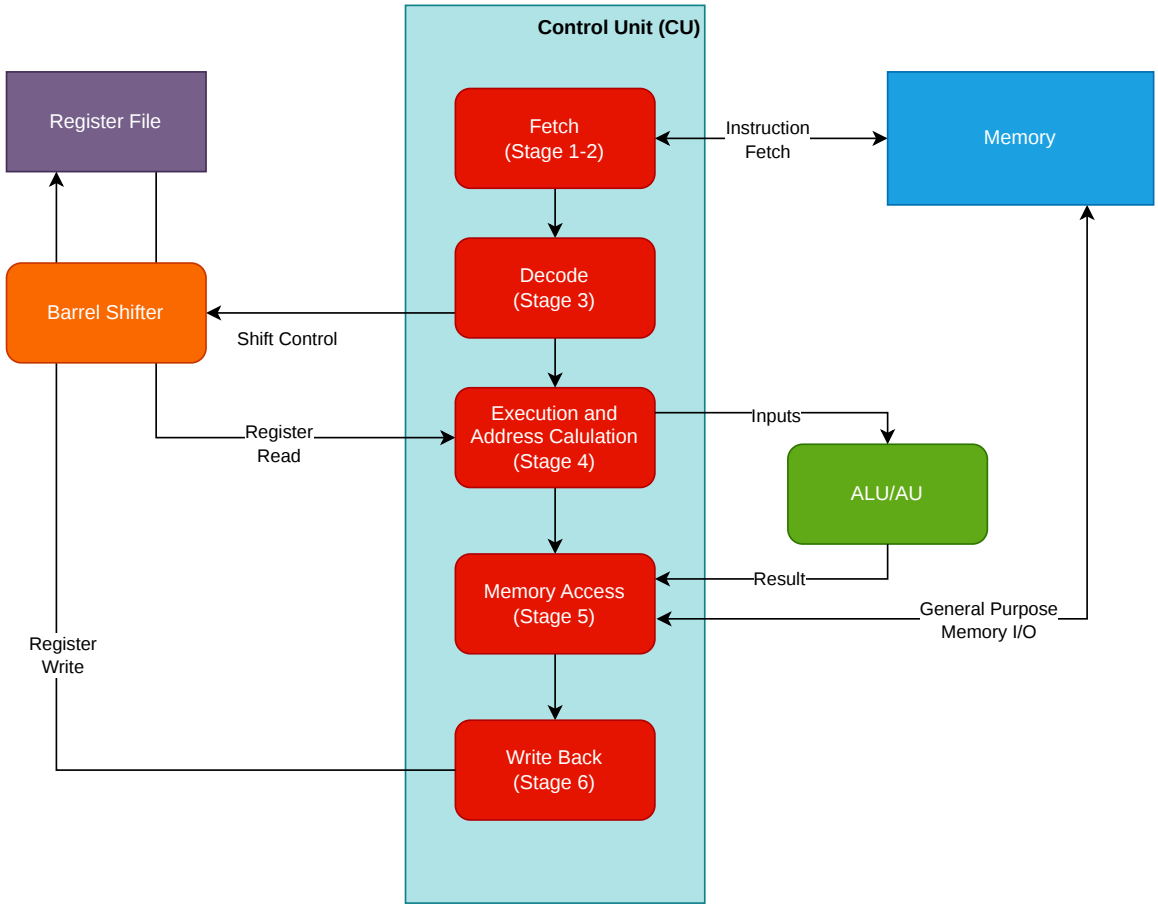

Figure 2.1: SIRC-1 CPU Execution Phase Architecture

2.3.1 Control Unit (CU)

The Control Unit sits at the heart of the SIRC-1 CPU, coordinating all other functional units throughout the instruction execution phases. It decodes instructions and generates control signals that orchestrate data movement and operations across the CPU.

Key responsibilities include:

- Instruction fetch and decode
- Execution phase coordination
- Control signal generation for all functional units
- Condition code evaluation for conditional execution

2.3.2 Arithmetic Logic Unit (ALU)

The ALU performs all arithmetic and logical operations on register values. It supports the following operations:

- **Arithmetic:** Addition, Subtraction (with and without carry)
- **Logical:** AND, OR, XOR
- **Comparison:** Test and compare operations
- **Status Flag Generation:** Carry, Overflow, Negative, Zero

The ALU operates exclusively on data from the Register File (after optional shifting) and outputs results either back to the Register File or to memory via the data bus.

2.3.3 Addressing Unit (AU)

The Addressing Unit calculates memory addresses and manages address-related operations. It is responsible for:

- Computing effective addresses from base + displacement
- Handling pre-decrement and post-increment addressing modes
- Updating address register pairs (l, a, s, p) for LDEA, jump, and branch instructions
- Presenting calculated addresses to the address bus

The AU works in parallel with the ALU during the execution stage, allowing address calculation to occur simultaneously with ALU operations.

2.3.4 Register File

The Register File contains all CPU registers organized into three categories:

- **General Purpose Registers:** `r1`–`r7`
- **Address Register Pairs:** `l` (link), `a` (address), `s` (stack), `p` (program counter)
- **Status Register:** `sr`

The Register File provides multi-port access, allowing the ALU, AU, and Control Unit to read and write registers as needed during each execution phase. All registers are 16 bits wide, though address register pairs can be combined to form 24-bit addresses.

2.3.5 Barrel Shifters

Two barrel shifter units sit between the Register File and the CU, enabling efficient bit manipulation operations.

The barrel shifters support logical shift left (LSL), logical shift right (LSR), arithmetic shift right (ASR), and rotate operations. Shift amounts can be specified as immediate values or dynamically from register contents.

The shift is applied to the first source operand when fetching the source registers for an instruction. Shifts cannot be applied to the result or any other source operand.

2.4 Design Characteristics

2.4.1 Combinatorial Logic Implementation

The SIRC-1 uses pure combinatorial logic with no microcode engine. Each execution phase is implemented as combinatorial circuits that process data in a single clock cycle. This approach:

- Simplifies the hardware design
- Provides a deterministic, fixed execution sequence
- Eliminates the complexity of microcode sequencing
- Makes the CPU behavior fully predictable and easy to reason about

2.4.2 Fixed Execution Sequence

All instructions execute in exactly 6 phases, regardless of instruction type or operands, and complete in 6 clock cycles when no bus wait states are required. Delayed bus acknowledgement can hold a phase and add wait cycles. This design choice simplifies software timing and eliminates the need for branch prediction or complex multi-stage control logic found in variable-time architectures.

2.4.3 Parallel Functional Units

The ALU and AU operate in parallel during Stage 4, allowing address calculation to proceed simultaneously with arithmetic operations. This parallelism enables efficient execution of memory reference instructions without additional clock cycles.

2.5 External Interface

2.5.1 Chip Layout

The SIRC CPU is most commonly provided as a DIP-64 package.

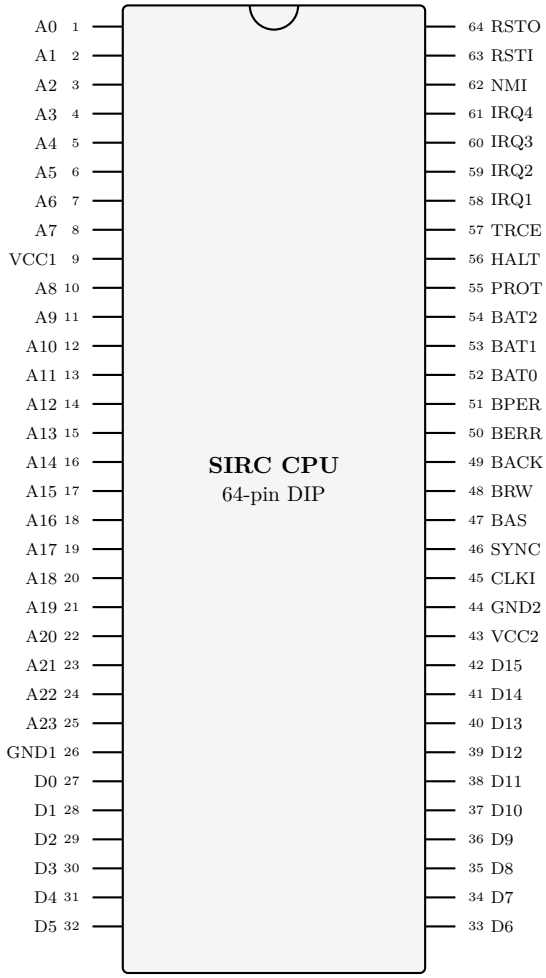

Figure 2.2: SIRC CPU 64-pin DIP pinout, top view

Each signal lists its own assertion convention. Address, data, and bus-type value pins use positive logic, while bus control, handshake, interrupt, halt, trace, and reset pins generally use active-low conventions. The descriptions below use "asserted" and "deasserted" to refer to the logical state of a signal according to its signal type.

2.5.2 Pin Descriptions

Address Bus Bits (A0-23)

- **Direction:** Output
- **Signal Type:** Active High

Connected to the address bus.

Pin	Name	Description
1-8	A0-A7	Address bus bits 0-7
9	VCC1	Power supply
10-25	A8-A23	Address bus bits 8-23
26	GND1	Ground
27-42	D0-D15	Data bus bit 0-15
43	VCC2	Power supply
44	GND2	Ground
45	CLKI	Clock input
46	SYNC	Instruction-sync output
47	BAS	Bus access strobe output
48	BRW	Bus read/write output
49	BACK	Bus acknowledge input
50	BERR	Bus error input
51	BPER	Bus protection error
52-54	BAT0-BAT2	Bus access type bits 0-2
55	PROT	Protected-mode output
56	HALT	External CPU halt input
57	TRCE	Force trace mode input
58-61	IRQ1-IRQ4	Maskable interrupt inputs 1-4
62	NMI	Non-maskable interrupt input
63	RSTI	Reset input
64	RSTO	Reset output

Table 2.1: SIRC CPU 64-pin DIP pinout

These 24 pins declare the address on the bus that the CPU will read or write from when BAS (bus access strobe) is asserted.

Power Pins (VCC1-2, GND1-2)

These pins provide the 5V power that the CPU requires to function.

The VCC pins must be connected to 5V +/- 5% and GND pins must be connected to a 0V or ground rail.

Data Bus Bits (D0-D15)

- **Direction:** Bi-directional
- **Signal Type:** Active High

Connected to the data bus.

These 16 pins will be driven to a 16-bit value when the CPU is outputting to the bus, and will be an input for a 16-bit value when the CPU is reading from the bus.

Clock Input (CLKI)

- **Direction:** Input
- **Signal Type:** Rising Edge
- **Max Rate:** 24 Mhz

Progresses the CPU to the next cycle when the input transitions from low to high (rising edge).

Each transition from low to high will progress the CPU internal six stage sequence, and so every six clock transitions will be a full instruction execution (assuming no wait for bus devices).

Instruction Sync Output (SYNC)

- **Direction:** Output
- **Signal Type:** Active High

Asserted while the CPU is in the first execution phase of an instruction or exception-unit dispatch. It remains asserted if a wait state holds that phase, and is deasserted during every other phase.

It can be useful for debugging purposes.

Bus Access Strobe Output (BAS)

- **Direction:** Output
- **Signal Type:** Active Low

When the CPU is ready to perform a bus operation, BAS is asserted after the address pins, BRW, BAT, PROT, and any CPU-driven data pins are valid.

BAS remains asserted until the bus responds with BACK, BERR, or BPER. If no response is asserted, the CPU holds the current execution phase and repeats the same bus request on the following clock.

Bus Read/Write Output (BRW)

- **Direction:** Output
- **Signal Type:** Read High / Write Low

Only relevant when the BAS (Bus access strobe output) is asserted.

When high, the CPU is requesting a read from the bus. When low, the CPU is requesting a write to the bus.

Bus Acknowledge Input (BACK)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the bus is reporting that the requested read/write completed successfully. On reads, the CPU samples D0-D15 on the clock edge that accepts BACK.

If all bus devices are fast and can complete reads/writes in a single cycle, this pin can be permanently held asserted.

Bus Error Input (BERR)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the bus is reporting that the requested read/write was unsuccessful.

This will cause the CPU to abort the I/O operation and raise a bus fault exception.

An example of a situation where a bus error might be raised would be I/O to an address that is not connected to any device.

Bus Protection Error Input (BPER)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the bus is reporting that the requested read/write was to a valid address/device, but that the I/O is disallowed given the current state of the CPU or bus.

This will cause the CPU to abort the I/O operation and raise a bus protection fault exception.

It acts very similar to the BERR pin but raises a different fault so that programs can handle it differently.

An example of a situation where a bus protection error might be raised would be a write to ROM, or a write to a privileged section of memory when the CPU is not in supervisor mode.

BAT2	BAT1	BAT0	Description
0	0	0	No I/O
0	0	1	Instruction Fetch
0	1	0	Data Read
0	1	1	Data Write
1	0	0	Exception Vector Fetch
1	0	1	DMA Co-processor Read Burst
1	1	0	DMA Co-processor Write Burst
1	1	1	Reserved

Table 2.2: Bus Access Type Bits Meaning

Bus Access Type Bits (BAT0-BAT2)

- **Direction:** Output
- **Signal Type:** Active High

Output by the CPU to describe what I/O operation it is performing.

Can be used to provide useful hints to external devices during I/O.

For example, if a memory controller knows that the DMA co-processor will be reading more than one sequential address, it can prefetch the next address.

Another example might be a memory controller providing memory protection if these pins are used in conjuction with the PROT (Protected-mode output) pin. A memory controller might trigger a bus error if BAT is 010 (data read), PROT is 1 (user mode) and the address is in a protected memory region.

Protected-mode Output (PROT)

- **Direction:** Output
- **Signal Type:** Active High

Indicates whether the CPU is in protected mode (high) or in supervisor mode (low).

Connected directly to the status register Protected Mode (P) bit.

When the CPU is in protected mode, programs have access to less types of instructions (e.g. they cannot write to the upper word of the address registers).

Exposing this as an external pin allows other devices such as memory controllers to block access to protected regions of memory by raising bus protection errors.

External CPU halt input (HALT)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the CPU will stop executing instructions.

HALT is sampled at instruction boundaries. If asserted, the CPU completes the current instruction and halts before the next instruction fetch. It resumes as normal when the input is deasserted.

This pin is useful for hardware driven debugging, as the CPU can be halted and the state of the system examined.

Force trace mode input (TRCE)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted at an instruction boundary, the instruction being started is traced regardless of whether software has cleared the Trace Mode (T) flag in the status register.

When in trace mode the CPU will raise exceptions every instruction to allow software debugging.

Maskable interrupt inputs (IRQ1-IRQ4)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the corresponding hardware interrupt line is sampled by the CPU.

They can be "masked" by the Hardware Interrupt Enable Bits (E1-E4) of the status register, which will disable them.

Interrupt inputs are level-sensitive and are sampled at instruction boundaries. Enabled interrupt lines are latched in the pending hardware-exceptions register; disabled maskable lines are ignored and not queued. If an interrupt pin remains asserted after its handler returns, it may be sampled again through the normal interrupt rules.

See the exceptions section for more details.

Non-maskable interrupt inputs (NMI)

- **Direction:** Input
- **Signal Type:** Active Low

When asserted, the CPU samples the Level 5 hardware exception line.

This interrupt cannot be masked, and if this pin is asserted while another level 5 exception is being handled, a "level five hardware exception conflict" fault will be raised.

If NMI remains asserted after its handler returns, it may be sampled again through the normal Level 5 interrupt rules. Therefore, it is important that the level 5 exception handlers are executed quickly and that NMI is only asserted for important non-interruptible events.

Reset input (RSTI)

- **Direction:** Input
- **Signal Type:** Active Low

Can be used by external devices to force a reset of the CPU.

When asserted, the CPU immediately aborts the current instruction or bus wait, stops normal processing, resets to phase 0, and begins the reset sequence described in Chapter 6.

The RSTO pin will also be asserted.

When this pin is deasserted, the CPU completes the reset-output hold, stops asserting the RSTO pin, and the exception unit fetches the reset vector.

If your system needs external devices to be reset for longer than the CPU reset-output hold, reset timing will have to be managed externally to the CPU and RSTI will need to be asserted for however long is needed for the other devices to reset.

Reset output (RSTO)

- **Direction:** Output
- **Signal Type:** Active Low

Asserted while the RSTI pin is asserted and during the reset-output hold before reset-vector fetch.

It will also be asserted for the reset-output hold after a software reset.

A reset can be triggered by asserting the Reset input (RSTI) pin, or in software by a special exception co-processor meta instruction (RSET).

This pin is useful to reset other devices on the bus whenever the CPU is reset or to trigger more complex reset/boot controllers.

2.5.3 Timing Notes

Clocked Bus Model

All bus timing is described relative to the rising edge of CLKI. The CPU presents a bus request during the execution phase that needs external access. If no response is asserted, the CPU repeats the same request and does not advance to the next execution phase.

Signal	Timing rule
A0–A23	Valid whenever BAS is asserted. Stable until the request completes or aborts.
D0–D15	Driven by the CPU only for writes. On reads, the selected device drives the bus and the CPU samples D0–D15 when BACK is accepted.
BRW	Valid whenever BAS is asserted. High requests a read; low requests a write. Stable until the request completes or aborts.
BAT0–BAT2	Valid whenever BAS is asserted. Encodes the access type for the active request and is stable until completion or abort.
PROT	Reflects the status register P bit and may be used with BAS, BAT, and A0–A23 by external protection logic.
BAS	Asserted while a bus request is active. Remains asserted across wait states.
SYNC	Asserted while phase 1 of an instruction or exception-unit dispatch is active.

Table 2.3: CPU bus output timing rules

When BAS is deasserted there is no active bus transaction. External devices must ignore A0–A23, D0–D15, BRW, and BAT0–BAT2 unless BAS is asserted.

Bus Transaction Completion

Response	Effect when sampled while BAS is asserted
BACK	Completes the request successfully. For reads, D0–D15 is sampled on the accepting clock edge. For writes, the write is considered externally committed.
BERR	Aborts the request and raises a bus fault. Read data is ignored and writes are not considered successfully committed by the CPU.
BPER	Aborts the request and raises a bus protection fault. Read data is ignored and writes are not considered successfully committed by the CPU.
No response	Inserts a wait state. The CPU remains in the current execution phase and keeps A0–A23, D0–D15 for writes, BRW, BAT0–BAT2, PROT, and BAS stable.

Table 2.4: Bus response timing rules

BACK, BERR and BPER are logically mutually exclusive. If more than one is asserted for the same request, BERR has priority, then BPER, then BACK.

Bus Cycle Types

Cycle	CPU drives	External device drives	Completion
Read	A0–A23, BRW=1, BAT, BAS, PROT	D0–D15 for the selected address	BACK samples data; BERR/BPER aborts.
Write	A0–A23, D0–D15, BRW=0, BAT, BAS, PROT	No data bus drive	BACK accepts write; BERR/BPER aborts.
Wait state	Same values as the previous request cycle	Same ownership as the active request	Continues until BACK, BERR, or BPER is asserted.
No bus cycle	No active transaction	No selected device should drive D0–D15	CPU advances internally.

Table 2.5: External bus cycle ownership

Bus Timing Diagrams

The timing diagrams below show cycle-level logical timing. Each labelled interval represents one CLKI cycle as observed between rising edges. Waveform rows show physical logic levels; for active-low signals, `L` means asserted and `H` means deasserted. Address, data, `BAT`, and phase rows are value bands rather than electrical setup/hold specifications. The diagrams use `IF` for instruction fetch, `DR` for data read, `DW` for data write, `EV` for exception vector fetch, `EA` for effective address, and `RV` for reset vector.

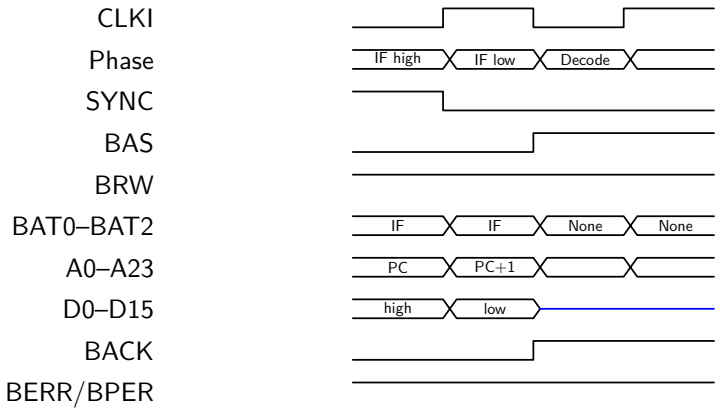

Figure 2.3: Instruction-fetch high-word and low-word reads with no wait states

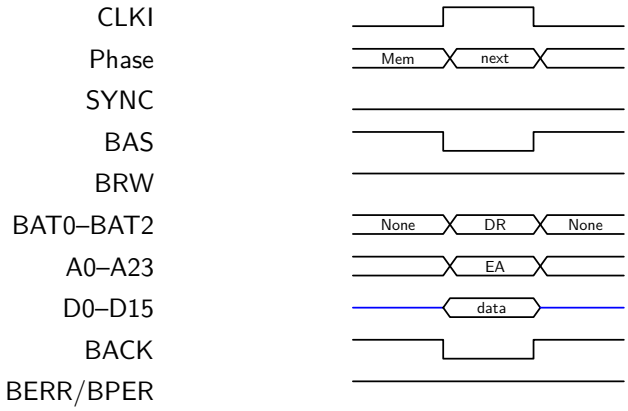

Figure 2.4: Data read cycle with no wait state

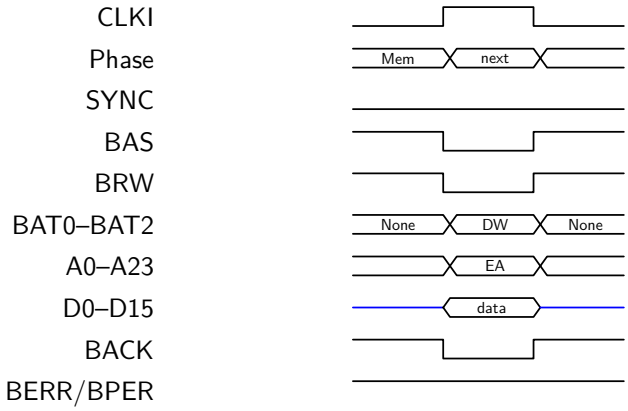

Figure 2.5: Data write cycle with no wait state

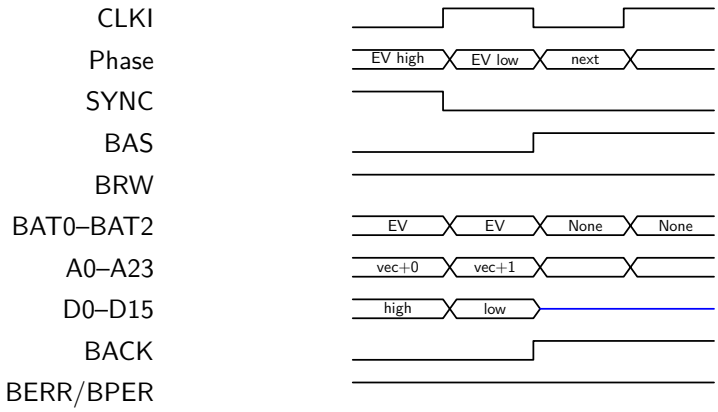

Figure 2.6: Exception-vector fetch with high-word then low-word reads

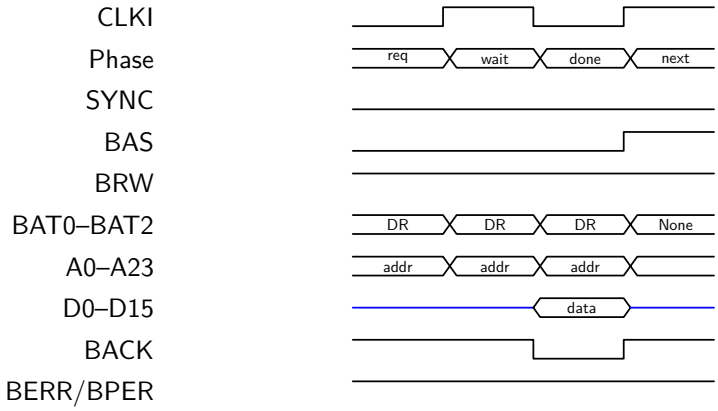

Figure 2.7: Wait-state insertion when no bus response is asserted

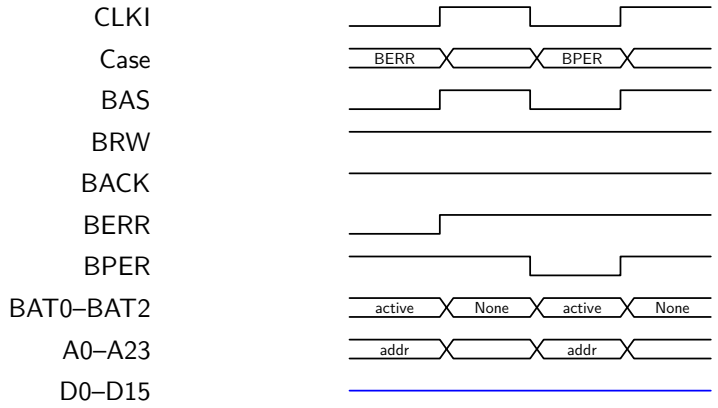

Figure 2.8: Bus-error and bus-protection abort timing

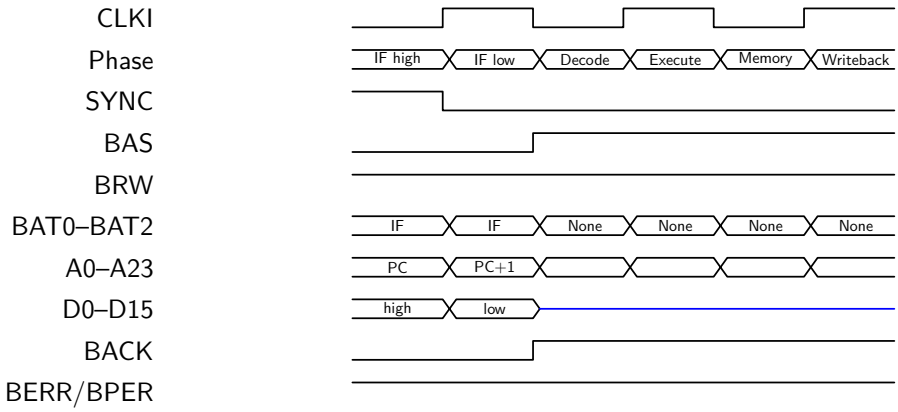

Figure 2.9: Complete register-only ALU instruction with no wait states

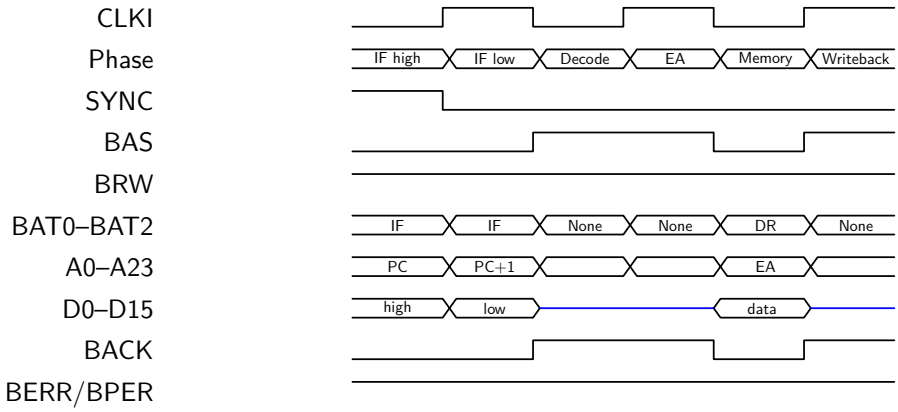

Figure 2.10: Complete LOAD instruction with no wait states

Reset and Interrupt Timing Examples

Figure 2.11 shows a hardware reset where `RSTI` is asserted for one clock. Holding `RSTI` asserted for longer extends the reset-output interval; the reset-vector fetch does not begin until `RSTI` is deasserted and the reset-output hold has completed.

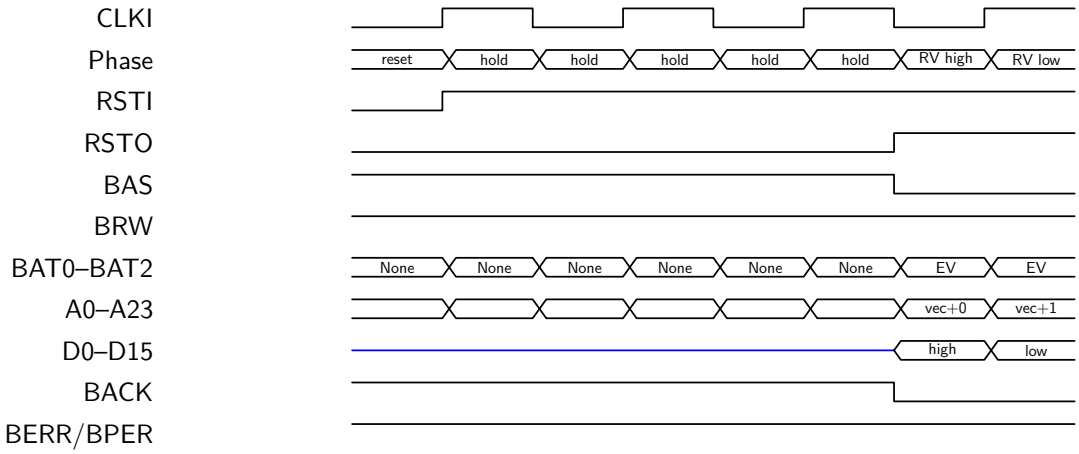

Figure 2.11: Hardware reset with one-cycle RSTI assertion and reset-vector fetch

Figure 2.12 shows an enabled maskable interrupt asserted before an instruction boundary. The CPU samples and latches the interrupt at the boundary. The following exception-unit dispatch fetches the selected hardware-interrupt vector instead of fetching the next processing-unit instruction. If an interrupt line is asserted after an instruction boundary, it is not sampled until the next boundary.

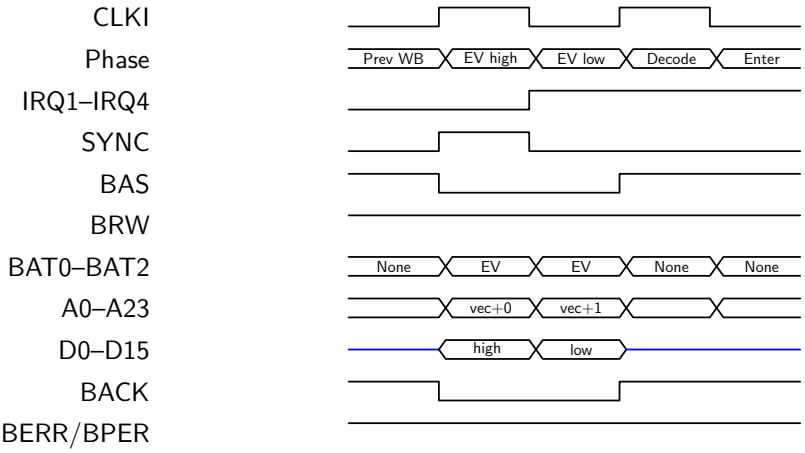

Figure 2.12: Enabled IRQ sampled at an instruction boundary and dispatched with no wait states

Asynchronous and Boundary-Sampled Inputs

Input	Sampling rule
BACK, BERR, BPER	Sampled while BAS is asserted. They complete or abort the active bus request.
IRQ1–IRQ4, NMI	Level-sensitive and sampled at instruction boundaries. Inputs should remain asserted until an instruction boundary if service is required.
HALT	Sampled at instruction boundaries. If asserted, the CPU halts before the next instruction fetch and resumes when deasserted.
TRCE	Sampled at instruction boundaries. If asserted, the instruction being started is traced.
RSTI	May be asserted at any time. It aborts the current instruction, exception dispatch, or bus wait and starts the reset sequence.

Table 2.6: External input sampling rules

This manual defines cycle-level logical timing only. Electrical setup, hold, propagation delay, voltage, fanout, and maximum-board-loading requirements are outside the architectural specification and should be defined by a future datasheet or hardware implementation note.

Chapter 3

Register Model

3.1 Overview

The SIRC-1 CPU provides sixteen 16-bit registers that serve various purposes. These registers can be classified into three categories:

- General purpose registers (`r1`–`r7`)
- Address register pairs (`l`, `a`, `s`, `p`)
- Status register (`sr`)

All programmer-visible registers are 16 bits wide. Address registers can be used in pairs to form 24-bit addresses for memory operations.

3.2 General Purpose Registers

3.2.1 Register Set

The SIRC-1 provides seven general-purpose registers:

Register	Description
`r1`	General purpose register 1
`r2`	General purpose register 2
`r3`	General purpose register 3
`r4`	General purpose register 4
`r5`	General purpose register 5
`r6`	General purpose register 6
`r7`	General purpose register 7

These registers can be used freely for any purpose by the programmer. They hold 16-bit values and can be used as source or destination operands for all ALU instructions.

3.2.2 Usage Conventions

While there are no hardware-enforced conventions, typical usage patterns include:

- `r1`–`r4`: Temporary values and expression evaluation
- `r5`–`r7`: Preserved across function calls (by convention)

These are software conventions only and are not enforced by the CPU.

3.3 Address Register Pairs

3.3.1 Overview

The SIRC-1 provides four address register pairs, each consisting of a high and low 16-bit register. When used together, these pairs form 24-bit addresses by combining the low 8 bits of the high register with the full 16 bits of the low register.

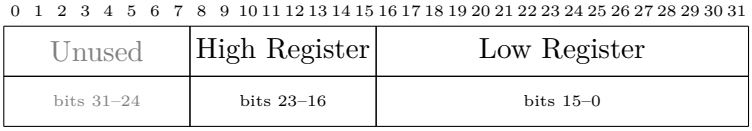

Figure 3.1: Address Register Pair Formation

The upper 8 bits (bits 31–24) of the high register are not used for addressing but remain available for storage, enabling techniques such as tagged pointers.

The exact address composition and stored-address layout are specified in Chapter 4.

Although each address register pair has a specific name, and a suggested usage by convention, they are all treated the same by STOR/LOAD instructions and they can all be used for indirect addressing the same way.

In fact, you can perform a jump by simply loading a value in to the Program Counter registers.

However, it is worth noting that the Link Register pair will be updated automatically through link-writing instructions (**`LDEL`**, **`LJSR`**, and **`BRSR`**) and the Program Counter pair is incremented after every instruction.

3.3.2 Link Register (l)

Component	Register
High Register	`lh` (privileged)
Low Register	`ll`

The link register pair stores return addresses for subroutine calls. When a subroutine is called using **`LDEL`**, **`LJSR`**, or **`BRSR`**, the return address is automatically saved to this register pair.

Typical Use:

- Storing return addresses from subroutine calls
- Alternate storage for general memory addressing

3.3.3 Address Register (a)

Component	Register
High Register	`ah` (privileged)
Low Register	`al`

A general-purpose address register pair used for pointer operations and memory access.
Typical Use:

- Base pointer for data structures
- Array indexing
- General memory addressing

3.3.4 Stack Pointer (s)

Component	Register
High Register	`sh` (privileged)
Low Register	`sl`

The stack pointer register pair points to the current top of the stack. Pre-decrement and post-increment addressing modes are particularly useful with this register.
Typical Use:

- Stack operations (push/pop)
- Local variable allocation
- Function parameter passing

3.3.5 Program Counter (p)

Component	Register
High Register	`ph` (privileged)
Low Register	`pl`

The program counter register pair points to the next instruction to be executed. It is automatically incremented by 1 after each 16-bit word fetch. Since instructions are 32 bits (two words), the PC is incremented twice during a complete instruction fetch, advancing by 2 words total per instruction.
Typical Use:

- Automatically updated during normal execution
- Modified by branch and jump instructions
- Used for position-independent code

3.4 Status Register

The status register (`sr`) is a special 16-bit register that contains CPU status flags and control bits. It is divided into two bytes:

- **Lower Byte (bits 7–0):** Condition flags, accessible in both supervisor and protected modes
- **Upper Byte (bits 15–8):** Control and privileged state

In protected mode, reads of `sr` are redacted to the lower byte (upper byte reads as zero). Direct writes to `sr` raise a privilege violation fault.

The status register is discussed in detail in Chapter 5.

3.5 Privilege Restrictions

3.5.1 High Address Register Restrictions

The high component of each address register pair (`lh`, `ah`, `sh`, `ph`) is considered privileged:

- In **supervisor mode**: Can be read from and written to freely
- In **protected mode**:
  - Reading is allowed
  - Direct writes trigger a privilege exception
  - Address-register-pair write-back is allowed only when the high word would remain unchanged; otherwise it triggers a privilege exception

This restriction provides a form of memory segmentation/protection. User programs running in protected mode cannot modify the segment portion of addresses, preventing them from accessing memory outside their designated segment. Instructions such as **`LDEA`**, **`LDEL`**, **`BRAN`**, **`BRSR`**, **`LJMP`**, **`LJSR`**, and **`RETS`** remain usable in protected mode when they stay within the current segment. If their address-register-pair write-back would alter a high word, the CPU raises a privilege violation fault.

The status register differs slightly from the high address registers:

- High address registers (`lh`, `ah`, `sh`, `ph`) remain readable in protected mode, but direct writes trigger a privilege violation fault.
- The status register's privileged byte is hidden on reads in protected mode, and cannot be modified directly from protected mode writes.

3.6 Register Addressing

Registers are addressed using 4-bit register identifiers in instruction encoding:

ID	Register	Privileged
0x0	`sr`	Upper byte only
0x1	`r1`	No
0x2	`r2`	No
0x3	`r3`	No
0x4	`r4`	No
0x5	`r5`	No
0x6	`r6`	No
0x7	`r7`	No
0x8	`lh`	Yes
0x9	`ll`	No
0xA	`ah`	Yes
0xB	`al`	No
0xC	`sh`	Yes
0xD	`sl`	No
0xE	`ph`	Yes
0xF	`pl`	No

Table 3.1: Register Encoding

3.7 Usage Examples

3.7.1 Basic Register Operations

```
; Load immediate value into r1
LOAD r1, #100

; Add 2 to the value stored in r1 and store the result into r2
LOAD r2, r1
ADDI r2, #2

; Add r1 and r2, store in r3
ADDR r3, r1, r2
```

3.7.2 Address Register Usage

```
; Load value from memory at address in 'a' register pair
LOAD r1, (a)

; Store r1 to memory at address (a + 8)
STOR (#8, a), r1

; Push r1 onto stack (pre-decrement)
```

```
8  STOR -(#2, s), r1
9
10 ; Pop stack into r2 (post-increment)
11 LOAD r2, (s)+
```

3.7.3 Subroutine Call

```
1 ; Call subroutine at address in 'a' register
2 ; Return address saved to 'l' register automatically
3 LJSR a
4
5 ; ...subroutine code...
6
7 ; Return from subroutine
8 ; Copies 'l' register to 'p' register
9 RETS
```

Chapter 4

Data Representation and Memory Model

4.1 Overview

This chapter defines the representation rules used by registers, memory, program images, exception vectors, and multi-word values. These rules are architectural: assemblers, loaders, emulators, hardware implementations, and debuggers must agree on them.

4.2 Word and Byte Ordering

The SIRC-1 is a word-addressed processor. Each architectural memory address names one 16-bit word. Byte addressing is not provided by the CPU instruction set.

When a word is represented outside the CPU as bytes, such as in a ROM image or debugger dump, the word is stored in big-endian byte order: the most significant byte appears first.

Value	First byte	Second byte
`0x1234`	`0x12`	`0x34`
`0xFF80`	`0xFF`	`0x80`

Table 4.1: External Byte Order for 16-bit Words

Multi-word quantities are stored with the most significant word at the lowest word address. This applies to instructions, exception vectors, saved addresses, and any software-defined multi-word integer convention that chooses to use the architectural order.

4.3 Address Values

Architectural addresses are 24-bit word addresses. The four address-register pairs (`l`, `a`, `s`, and `p`) represent an address by combining the low byte of the high register with the full low register:

$$\text{Address} = ((\text{HighRegister} \,\&\, \texttt{0x00FF}) \ll 16) \mid \text{LowRegister} \tag{4.1}$$

Bits 15–8 of the high register are not driven on the address bus and do not participate in address comparisons, effective-address calculation, instruction fetch, or exception vector dispatch. They are preserved as ordinary register storage unless an instruction explicitly writes the high register.

Pair value	High register	Low register	Address bus value
`ah:al`	`0x0012`	`0x3456`	`0x123456`
`ah:al`	`0xAB12`	`0x3456`	`0x123456`
`ph:pl`	`0x0000`	`0x0100`	`0x000100`

Table 4.2: Address Register Pair Interpretation

When a 24-bit address is stored in memory as a 32-bit quantity, the high word is stored first and the low word second. Only bits 7–0 of the high word are significant when the value is loaded as an address.

Stored address	Word at address N	Word at address N+1
`0x123456`	`0x0012`	`0x3456`
`0xFECAFE`	`0x00FE`	`0xCAFE`

Table 4.3: Stored 24-bit Address Format

4.4 Instruction Storage

Every instruction is 32 bits, stored as two consecutive 16-bit words:

- Word at `p`: instruction bits 31–16
- Word at `p+1`: instruction bits 15–0

Instruction fetch reads the high word first, increments `p` by one word, reads the low word, then increments `p` by one word again. Instruction addresses must be even word addresses. Fetching from an odd word address raises an alignment fault.

4.5 Exception Vector Storage

Each exception vector table entry is a stored 24-bit address in the format shown in Table 4.3. Vector entry `V` starts at word address `V * 2`:

- Word at `V * 2`: target address high word
- Word at `V * 2 + 1`: target address low word

For example, vector `0x06` occupies word addresses `0x00000C` and `0x00000D`. If it should jump to `0x123456`, those two words contain `0x0012` and `0x3456`.

4.6 Immediate Values

Immediate fields are encoded as raw two's-complement bit patterns. The consuming instruction defines whether the field is interpreted as signed, unsigned, or bitwise data.

Use	Width	Interpretation
ALU arithmetic immediate	16 bits	Two's-complement operand; result is truncated to 16 bits
ALU arithmetic short immediate	8 bits	Zero-extended operand in the range `0x00`–`0xFF`
ALU logical immediates	16 or 8 bits	Bit mask; no sign interpretation is applied by the logical operation
Memory displacements	16 bits	Two's-complement offset added to the low address word
Coprocessor operands	16 bits	Coprocessor-defined command or parameter bits

Table 4.4: Immediate Field Interpretation

Short immediates occupy an 8-bit field and are zero-extended before use by ALU arithmetic. They are therefore suitable for compact non-negative constants and bit masks, not for encoding negative arithmetic operands. Assemblers must reject values that cannot be represented in the selected immediate field width unless they deliberately offer a documented truncation mode.

Short immediates are not used for memory, effective-address, branch, jump, or subroutine-call displacements. Those forms use 16-bit immediate displacements, register displacements, or assembler-resolved full-width branch offsets.

4.7 Software Layout Conventions

The architectural storage order for multi-word quantities is high word first. The SIRC-1 reference manual does not otherwise define a complete ABI, stack-frame layout, argument-passing convention, or software integer library format. Where examples show structures, arrays, saved registers, or stack use, offsets are word offsets and each push or pop moves the stack pointer by one 16-bit word.

Software standards may build on the architectural high-word-first rule for wider integers and stored addresses, but those conventions are non-normative unless a separate ABI or toolchain document says otherwise.

4.8 Arithmetic and Address Wraparound

General-purpose register arithmetic is performed modulo 2^{16}. Arithmetic condition flags describe the truncated 16-bit result: C records unsigned carry or borrow, and V records signed overflow.

Address calculation for memory, effective-address, branch, jump, and auto-update operations adds offsets to the low word of the selected address register pair. The high address-register word is not changed by ordinary displacement, pre-decrement, or post-increment calculation. If the low word overflows or underflows and `SR.A` (Trap on Address Overflow) is set, the instruction raises a segment-overflow fault. If the bit is clear, the low word wraps modulo 2^{16}.

The program counter follows the same low-word wrap rule during normal instruction fetch. Since instructions are two words long and must begin at even word addresses, a fetch that would begin from an odd `pl` value raises an alignment fault.

4.9 Memory Ordering

The SIRC-1 has no architectural cache, store buffer, memory reordering, or speculative memory access. Bus reads and writes issued by the CPU are externally visible in program order. A bus device may delay completion by withholding bus acknowledgement, but this stalls the current execution phase rather than allowing later bus accesses to pass it.

Chapter 5

Status Register

5.1 Overview

The Status Register (`sr`) is a 16-bit special-purpose register that contains condition flags and control bits for the CPU. It is divided into two bytes with different privilege levels:

- **Lower Byte (bits 7–0):** Condition flags – readable and writable in both supervisor and protected modes
- **Upper Byte (bits 15–8):** Control and mode flags – accessible only in supervisor mode

5.2 Status Register Layout

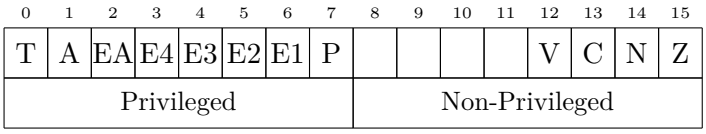

Figure 5.1: Status Register Bit Layout

5.3 Condition Flags (Non-Privileged)

These flags reflect the results of ALU operations and comparisons. They can be read and written by code running in any privilege level.

5.3.1 Zero Flag (Z) – Bit 0

Set when: The result of an operation is zero

Cleared when: The result of an operation is non-zero

Tested by: Equal (`==`), Not Equal (`!=`) condition codes

5.3.2 Negative Flag (N) – Bit 1

Set when: The result of an operation is negative (bit 15 = 1) when interpreted as a signed two's complement value

Cleared when: The result is positive or zero (bit 15 = 0)

Tested by: Negative Set (`NS`), Negative Clear (`NC`), and signed comparison condition codes

5.3.3 Carry Flag (C) – Bit 2

Set when: An addition produces a carry out of bit 15, or a subtraction requires a borrow

Cleared when: No carry/borrow occurs

Tested by: Carry Set (`CS`), Carry Clear (`CC`), Unsigned Higher (`HI`), Unsigned Lower or Same (`LO`) condition codes

Note: For unsigned comparisons, the carry flag acts as an unsigned overflow/borrow indicator

5.3.4 Overflow Flag (V) – Bit 3

Set when: A signed arithmetic operation produces a result that cannot be represented in 16 bits (signed overflow)

Cleared when: No signed overflow occurs

Tested by: Overflow Set (`OS`), Overflow Clear (`OC`), and signed comparison condition codes

Note: Overflow occurs when adding two positive numbers yields a negative result, or adding two negative numbers yields a positive result

5.3.5 Reserved Flags – Bits 4–7

Bits 4 through 7 of the lower byte are reserved for future use. They should not be relied upon by software.

5.4 Control Flags (Privileged)

These flags control CPU behavior and can only be modified directly in supervisor mode. Attempts to write `sr` in protected mode raise a privilege violation fault.

5.4.1 Protected Mode (P) – Bit 8

When Clear (0): CPU operates in supervisor mode with full privileges

When Set (1): CPU operates in protected mode with restricted access to:

- High address-register components (`lh`, `ah`, `sh`, `ph`)
- Upper byte of status register
- Privileged coprocessor operations (opcodes > 0x07xx)

Note: This bit provides basic memory protection by preventing user code from changing memory segments. Performing a restricted operation will cause a "privilege violation" fault.

5.4.2 Hardware Interrupt Enable Bits (E1-E4) – Bits 9–12

These four bits individually control whether each of the four maskable hardware interrupt lines is enabled:

- **E1 (Bit 9):** Enable Hardware Interrupt Line 1 (Level 1, lowest maskable priority)
- **E2 (Bit 10):** Enable Hardware Interrupt Line 2 (Level 2)
- **E3 (Bit 11):** Enable Hardware Interrupt Line 3 (Level 3)
- **E4 (Bit 12):** Enable Hardware Interrupt Line 4 (Level 4, highest maskable priority)

Note: Hardware Interrupt Line 5 (Level 5) is a Non-Maskable Interrupt (NMI) and is always enabled. It cannot be disabled via the status register.

When a bit is set to 1, the corresponding hardware interrupt is enabled and can trigger an exception when asserted. When cleared to 0, interrupts on that line are disabled and will not be serviced.

Important behavior:

- Disabled interrupts are not queued – they are completely ignored while the enable bit is 0
- Hardware interrupts can only interrupt execution if their priority level is higher than the currently executing exception level
- This prevents the same interrupt from re-triggering while its handler is executing
- Lower priority interrupts that occur while handling a higher priority interrupt are queued and serviced after the current handler completes (after RETE)

These bits are typically managed by the operating system's interrupt handling code. At reset, all interrupt enable bits are cleared (interrupts disabled).

Interrupt handling is discussed in detail in Chapter 6.

5.4.3 Exception Active (EA) – Bit 13

When Set (1): The CPU is currently executing an exception handler (`current_exception_level != 0`)

When Clear (0): The CPU is executing normal code (`current_exception_level == 0`)

Automatically Set: When any exception is entered (software, hardware, or fault)

Automatically Cleared: When returning to exception level 0 via `RETE`

Purpose: Allows software to detect whether it's running within an exception handler context

Typical Use: OS code can check this bit to determine appropriate behavior (e.g., avoid re-enabling interrupts during nested exception handling)

Note: This bit reflects the internal `current_exception_level` state. Programs can read this bit but cannot directly modify it – it is managed automatically by the exception handling hardware.

5.4.4 Trap on Address Overflow (A) – Bit 14

When Set (1): Memory address calculations that overflow or underflow will trigger an address overflow exception

When Clear (0): Address overflows wrap around without any side effects

Purpose: Helps detect pointer errors during development

Typical Use: Enabled during debugging, disabled in production for performance

5.4.5 Trace Mode (T) – Bit 15

When Set (1): An exception is generated after every instruction

When Clear (0): Normal execution

Purpose: Allows debuggers to single-step through code

Note: Used by SIRCIT (SIRC Instruction Tracer) for debugging

5.5 Status Register Updates

5.5.1 Automatic Updates

The condition flags (Z, N, C, V) are automatically updated by most ALU instructions, unless explicitly disabled:

- **Arithmetic Instructions** (`ADDI`, `SUBI`, `ADCI`, `SBCI`): Update all four condition flags
- **Logical Instructions** (`ANDI`, `ORRI`, `XORI`): Update Z and N flags; C and V flags are cleared
- **Compare Instructions** (`CMPI`, `CMPR`): Same as Arithmetic Instructions but does not update destination
- **Test Instructions** (`TSAI`, `TSAR`, `TSXI`, `TSXR`): Same as Logical Instructions but does not update destination
- **Load Instructions** (`LOAD`, `LDEA`): Does not update condition flags
- **Shift Operations**: Can optionally update flags based on the instruction's status register update source field

5.5.2 Manual Updates

In supervisor mode, the status register can be directly manipulated like any other register:

```
1 ; Set protected mode bit (only in supervisor mode)
2 ORRI sr, #0x0100
3
4 ; Enable all maskable hardware interrupts (set bits 9-12)
5 ; Note: Hardware interrupt line 5 is an NMI and is always enabled
6 ORRI sr, #0x1E00
7
```

```
; Disable all maskable hardware interrupts (clear bits 9-12)
ANDI sr, #0xE1FF

; Restore status register saved to stack
LOAD sr, (s)+
```

Important: When running in protected mode, any direct write to the status register raises a privilege violation fault. Protected-mode programs can still affect condition flags through ordinary ALU and shift instruction execution; they cannot directly load or store `sr`.

5.6 Reading the Status Register

In supervisor mode, reading `sr` returns the full 16-bit value. In protected mode, reading `sr` returns the lower byte in bits 7–0, with bits 15–8 masked to zero (for security).

```
; In supervisor mode - gets full 16 bits
LOAD r1, sr   ; r1 = 0xXXXX (all bits)

; In protected mode - upper byte masked
LOAD r1, sr   ; r1 = 0x00XX (upper byte forced to 0)
```

5.7 Exception Handling Effects

When an exception occurs:

1. The current status register is saved
2. Protected mode bit (P) is cleared (entering supervisor mode)
3. Trace mode bit (T) is cleared (disabling single-stepping in exception handler)
4. Interrupts may be masked depending on exception type

When returning from an exception with **`RETE`**, the saved status register is restored.

5.8 Usage Examples

5.8.1 Conditional Execution

```
; Compare r1 with r2
CMPR r1, r2

; Branch if equal
BRAN|== @label_equal

; Branch if r1 > r2 (unsigned)
BRAN|HI @label_greater

; Add 0x16 to r1 if r1 >= r2 (signed)
ADDI|>= r1, #0x16
```

5.8.2 Manual Status Flag Control

The SIRC-1 provides syntax to manually control how status flags are updated during ALU and shift operations. This is particularly useful for advanced control flow and flag preservation. See Section 9.12 in Chapter 9 for complete details.

```
; Update flags from shift instead of ALU
ADDI[S] r2, #1, LSL #2          ; Flags from shift, not addition

; Preserve flags across operations
CMPR r1, r2                     ; Set comparison flags
ADDI[N] r3, #1                  ; Increment without changing flags
BRAN|== @still_uses_cmpr_flags  ; Branch uses CMPR flags
```

5.8.3 Testing Bits

```
; Test if bit 5 is set in r1
TSAI r1, #0x0020   ; AND with 0x0020, update flags only

; Branch if bit was set (result non-zero)
BRAN|!= @bit_was_set
```

5.8.4 Enabling Protected Mode

```
; Entering user mode (from supervisor mode)
ORRI sr, #0x0100   ; Set protected mode bit
; Now running in protected mode

; Attempting to write privileged register will fault
; ADDI ah, #0x05   ; This would cause exception!
```

5.8.5 Saving and Restoring Status

```
; Save status register
LOAD r7, sr

; Do some operations that change flags
ADDI r1, r2, r3
CMPI r1, #100

; Restore original status
LOAD sr, r7
```

Chapter 6

Exception and Interrupt Handling

6.1 Overview

An exception is an event that causes the program flow to be diverted to an address stored in a predefined vector.

Some are directly caused by an instruction, some are a side effect of an instruction and some are asynchronously caused by external hardware.

They are managed by the exception coprocessor, which will take control of the processor when there is a pending exception.

6.2 Exception Types

6.2.1 Faults

Faults are priority 7 exceptions raised by the CPU itself in response to error conditions or debug events. They use vectors in the 0x01–0x0F range. Unlike hardware and software exceptions, faults cannot be masked or deferred: when a fault condition is detected, the fault is dispatched at the next instruction boundary regardless of the current exception level.

The T bit (TraceMode) in the status register is always cleared when entering any exception handler, including fault handlers. This ensures that trace faults do not fire inside handlers. RETE restores the saved status register, so if T was set before the exception, tracing resumes after RETE returns.

Fault Categories

Not all faults behave the same way with respect to the faulting instruction and the RETE return address. There are three categories:

Retryable faults The instruction that triggered the fault is cancelled before it takes effect. The link register stores the address of the faulting instruction. RETE returns to that instruction so it can be retried (possibly after the fault handler has corrected the conditions that caused the fault, e.g. mapping a page in response to a bus fault).

Bus Fault, Bus Protection Fault, Alignment Fault, Segment Overflow Fault, Invalid Opcode Fault, Privilege Violation Fault.

Post-instruction faults The instruction completes normally and its effects are committed before the fault fires. The link register stores the address of the *next* instruction. RETE advances to that instruction; the traced instruction is not re-executed.

Instruction Trace Fault.

Exception dispatch faults Raised during the exception handling machinery itself, not during normal instruction execution. The link register stores the program counter at the point the fault was detected within the exception handler. Recovery may require careful state management; these faults often indicate a hardware misconfiguration or a programming error in an exception handler.

Level Five Hardware Exception Conflict, Double Fault.

Fault Descriptions

- **Bus Fault** *(Retryable)* — Raised when an external device asserts the bus error CPU pin.

  This could happen, for example, if an unmapped address is presented by the CPU and another chip detects this and raises an error. It could also be used to implement virtual memory: the instruction is aborted and the program counter is not incremented, so the address can be re-calculated and the operation retried.

- **Bus Protection Fault** *(Retryable)* — Raised when an external device asserts the bus protection error CPU pin.

  This indicates that the requested address and device are valid, but the operation is disallowed. Examples include writing to ROM or accessing a protected memory region while the CPU is in protected mode.

- **Alignment Fault** *(Retryable)* — Raised when fetching instructions if the program counter contains an odd word address.

  This is to simplify fetching: the second word of an instruction is always at `pl | 0x1`, which ensures that instructions can never overflow a segment boundary.

- **Segment Overflow Fault** *(Retryable)* — Raised when a computed address would fall outside the current segment.

  Only raised if `SR.A` (Trap on Address Overflow) is set, since some programs intentionally rely on address wrap-around. Also raised if the program counter overflows; the captured bus address and bus access type in the fault metadata register help identify where the overflow occurred.

- **Invalid Opcode Fault** *(Retryable)* — Raised when a coprocessor call targets a non-existent coprocessor or requests an operation that the selected coprocessor does not implement.

  Can be used for forward compatibility: programs written for a future co-processor (e.g. a floating-point unit) will trap on older hardware, and the handler can emulate the operation in software.

  Normal processing-unit encodings outside the documented instruction set are reserved or architecturally undefined. They are not required to raise an invalid-opcode fault.

- **Privilege Violation Fault** *(Retryable)* — Raised when a privileged operation is attempted while not in supervisor mode:

1. Directly writing to the high word of any address register
2. Directly writing to the status register
3. Address-register-pair write-back that would change a high address-register word
4. Executing a supervisor-only coprocessor operation
5. Triggering a software exception with a vector below 0x60

- **Instruction Trace Fault** *(Post-instruction)* — Raised after every instruction completes when the `TraceMode` SR bit is set.

  Used for single-step debugging. Because the fault fires after the instruction commits, RETE returns to the instruction *following* the traced one. The T bit is cleared on entry to the fault handler (as with all exceptions), so the handler itself is not traced. RETE restores the saved SR, which re-enables tracing unless the handler explicitly clears T from the saved status register via `ETTR` before returning.

- **Level Five Hardware Exception Conflict** *(Exception dispatch)* — Raised when a level five hardware exception arrives while one is already being serviced.

  The windowed link register scheme has no spare register above level 6, so a second L5 cannot be handled normally. Rather than silently dropping it (which could mask a hardware misconfiguration), the CPU raises this fault. Level 5 interrupts should be treated as NMIs.

- **Double Fault** *(Exception dispatch)* — Raised when a fault occurs while another fault is already being handled.

  When the CPU is already at fault level (priority 7) and another fault occurs, there is no additional banked link register available. The fault link register (register 6) is overwritten with the new return address, losing the original fault's return address. The fault metadata register (register 7) is also overwritten; it encodes a double-fault flag along with both the current and original fault types.

  The CPU jumps to vector 0x08. This is typically a last-chance handler that dumps registers and resets. If another fault occurs while the CPU is already at fault level, the CPU attempts to dispatch vector 0x08 again. If the double-fault vector fetch itself fails, the CPU requests reset.

6.2.2 Hardware Exceptions (Interrupts)

A hardware exception occurs when interrupt pins are signalled by external hardware. These are also referred to as "interrupts". When a hardware exception occurs the current instruction is completed, and control is transferred to different vectors based on the interrupt "priority". The vectors for hardware exceptions are in the 0x10-0x50 range.

6.2.3 User Exceptions (Traps)

A user exception occurs when the program executes an EXCP instruction that signals the exception coprocessor that it should trigger an exception. They are usually used by programs running in protected mode to switch to supervisor mode in a controlled way. They are also referred to as "traps". The vectors for user exceptions are in the 0x60-0xFF range.

Software exceptions are only accepted from normal execution. If an **`EXCP`** instruction is executed while any exception or fault handler is already active, the software exception is ignored, is not queued, and does not fetch its vector.

6.3 Exception Registers

The SIRC-1 exception coprocessor maintains several special-purpose registers for exception handling.

6.3.1 Link Registers

The exception unit uses banked link registers to store the processor state when an exception occurs. There are eight link registers in total, indexed 0-7:

- Register 0: Software exceptions (traps)
- Registers 1-5: Hardware exceptions (interrupts), one for each priority level
- Register 6: Faults (abort exceptions)
- Register 7: Fault metadata (stores bus address and bus access type information, see section 6.3.1 for the format)

Each link register stores two pieces of information:

- **Return Address:** The 32-bit address to return to when the exception handler completes
- **Return Status Register:** The 16-bit status register value at the time of the exception

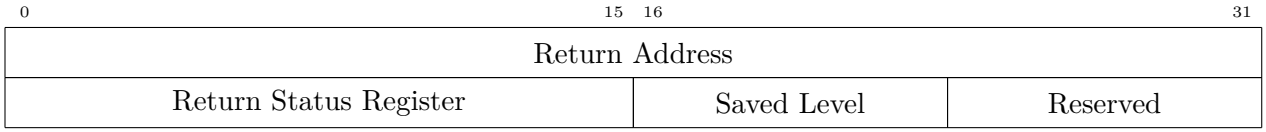

Figure 6.1: Exception Link Register Saved-State Layout

When an exception occurs, the appropriate link register (determined by the exception priority level) is populated with the current program counter and status register values. For abort exceptions (faults), the return address is the address of the faulting instruction so it can be retried. For regular exceptions (hardware and software exceptions), the return address is the address after the current instruction.

Register	Exception Source	Priority Level
0	Software exceptions	1
1	Level 1 hardware	2
2	Level 2 hardware	3
3	Level 3 hardware	4
4	Level 4 hardware	5
5	Level 5 hardware	6
6	Faults	7
7	Fault metadata	n/a

Table 6.1: Exception Link Register Assignment

Link Register Preservation

Important: Link registers can be overwritten when a fault occurs during the handling of another fault (a double fault). In this case, the fault link register (register 6) and the fault metadata register (register 7) is overwritten. This means the original return address is lost.

A valid way of handling this is just to declare the CPU state as invalid, do some last chance error handling (e.g. dumping the registers) and reset.

For robust fault handling that can recover from double faults, fault handlers should save the link register contents early in the handler:

```
:fault_handler
    ; Immediately save link register to memory before doing anything
    ; that might trigger another fault
    ETFR #6                    ; Save level 6 exception registers to a
        and r7
    STOR -(#0, s), r7
    STOR -(#0, s), ah
    STOR -(#0, s), al
    ETFR #7                    ; Save level 7 (metadata) exception
        registers to a and r7
    STOR -(#0, s), r7
    STOR -(#0, s), ah
    STOR -(#0, s), al

    ; Now safe to perform operations that might fault
    ; ...

    ; Restore and return
    LOAD al, (#0, s)+
    LOAD ah, (#0, s)+
    LOAD r7, (#0, s)+
    ETTR #7                   ; Restore level 7 (metadata) exception
        registers from a and r7
    LOAD al, (#0, s)+
    LOAD ah, (#0, s)+
    LOAD r7, (#0, s)+
    ETTR #6                   ; Restore level 6 exception registers from
        a and r7

     ; Now safe to use RETE
     RETE
```

However, if the stack pointer is invalid in this case, the double fault will still cause the CPU state to be corrupted. You could provide an even more robust implementation by storing/loading from hardcoded memory addresses that are known to always be valid.

Fault Metadata Register Format

The fault metadata register (register 7) stores additional information about faults beyond just the return address. The return_status_register field is used to encode fault-specific metadata in the following format:

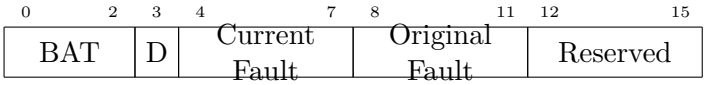

Figure 6.2: Fault Metadata Register Status Field Layout

- **Bits 0-2:** Bus access type (`BAT0`–`BAT2`) captured when the fault was raised (3 bits)
  - 0x0: None
  - 0x1: Instruction fetch
  - 0x2: Data read
  - 0x3: Data write
  - 0x4: Exception vector fetch
  - 0x5: DMA read burst
  - 0x6: DMA write burst
  - 0x7: Reserved
- **Bit 3:** Double fault flag (1 bit) - set to 1 if this is a double fault
- **Bits 4-7:** Current fault type (4 bits) - the type of fault that occurred
- **Bits 8-11:** Original fault type (4 bits) - only meaningful for double faults, indicates the fault type that was being handled when the double fault occurred
- **Bits 12-15:** Unused (reserved for future use)

Fault types use the following values:

- **0x01:** Bus fault
- **0x02:** Alignment fault
- **0x03:** Segment overflow fault
- **0x04:** Invalid opcode fault
- **0x05:** Privilege violation fault
- **0x06:** Instruction trace fault
- **0x07:** Level five hardware exception conflict
- **0x08:** Double fault
- **0x09:** Bus protection fault

The return_address field of register 7 stores the bus address that was being accessed when the fault occurred (for faults that involve memory access). For coprocessor invalid-opcode faults that occur during coprocessor dispatch before bus access, the low word of this field stores the 16-bit coprocessor command that could not be dispatched, and the high word is zero. This lets an invalid-opcode handler emulate missing optional coprocessor operations.

Exception handlers can use the `ETFR` instruction to read the fault metadata register and decode this information to determine how to handle the fault appropriately.

6.3.2 Cause Register

The cause register is a 16-bit internal register used to communicate between different CPU units and coordinate exception handling. It has the following structure:

- **Bits 15-12 (First Nibble):** Coprocessor ID (0x1 for exception unit, 0x0 for processing unit)
- **Bits 11-8 (Second Nibble):** Coprocessor opcode (the operation to execute)
- **Bits 7-0 (Third and Fourth Nibbles):** Vector or parameter value

The cause register is automatically populated by the exception unit when a pending exception needs to be serviced. The priority of exceptions is determined by examining the first nibble of the vector ID, calculated as 7 minus the value of the first nibble (e.g., 0x00 is priority 7, 0x40 is priority 3, 0x60 and above are all priority 1).

6.3.3 Hardware Interrupt Enable Bits

The status register contains 4 individual enable bits (bits 9-12 in the privileged byte) that control whether each maskable hardware interrupt line can trigger an exception:

- Bit 9 (E1): Hardware Interrupt Line 1 (Level 1, lowest maskable priority)
- Bit 10 (E2): Hardware Interrupt Line 2 (Level 2)
- Bit 11 (E3): Hardware Interrupt Line 3 (Level 3)
- Bit 12 (E4): Hardware Interrupt Line 4 (Level 4, highest maskable priority)

Note: Hardware Interrupt Line 5 (Level 5) is a Non-Maskable Interrupt (NMI) and cannot be disabled. It will always trigger an exception when asserted, regardless of the enable bit settings. Bit 13 is used for the Exception Active flag instead.

When an interrupt line is disabled (bit = 0), interrupts on that line are completely ignored and not queued. When enabled (bit = 1), the interrupt can trigger an exception if its priority is higher than the currently executing exception level.

6.3.4 Pending Hardware Interrupts

Hardware interrupt pins are level-sensitive inputs. When an enabled interrupt line is observed, the corresponding bit is set in the pending hardware-exceptions register. Disabled maskable interrupt lines are ignored at this sampling point and are not latched for later service.

The pending register stores one bit per hardware interrupt line, so repeated assertions of the same line coalesce into a single pending interrupt. If the external pin remains asserted after the interrupt is serviced, it may be sampled again and become pending again.

At an instruction boundary, the exception unit selects the highest-priority pending hardware interrupt whose priority is higher than the current exception level. The selected bit is cleared as the handler is dispatched. Pending lower-priority interrupts remain latched and are considered again after the current handler returns with `RETE`.

Faults have priority over all hardware interrupts. If a fault is pending at the same instruction boundary as one or more hardware interrupts, the fault is dispatched first and the hardware interrupts remain pending. This includes instruction trace faults: when trace mode raises a

post-instruction fault, that trace fault is serviced before pending hardware interrupts at the next boundary.

Hierarchical Exception Handling:

Exceptions in SIRC-1 are strictly hierarchical. A hardware interrupt can only interrupt the current execution if its priority level (exception level) is *higher* than the currently active exception level. This prevents:

- An interrupt handler from being interrupted by its own interrupt (same level)
- Lower priority interrupts from interrupting higher priority handlers
- Software exceptions from running during any hardware interrupt (would corrupt link register 0)

The exception unit maintains a `current_exception_level` field that tracks the active exception level (0-7). When not handling any exception, this is 0. When handling an exception at level N, only exceptions at level > N can interrupt.

Level 5 NMI-like Behavior:

Level 5 hardware exceptions behave similarly to a Non-Maskable Interrupt (NMI). They cannot be disabled via the interrupt enable bits in the status register, and will preempt any lower-priority exception handler. However, Level 5 is *not* fully non-maskable. Because faults carry priority 7 (higher than Level 5's priority 6), a L5 interrupt that arrives while a fault is being handled is *queued* in the pending hardware exceptions register rather than serviced immediately.

Once the fault handler returns via `RETE` and the exception level drops back below 6, the pending L5 interrupt fires normally.

This means Level 5 response latency is not deterministic when a fault is in progress. For timing-critical use of the NMI line, fault handlers should be kept as short as possible.

Keep in mind that if a second L5 interrupt arrives *while the CPU is already executing an L5 handler* (exception level 6), it is not queued. It will raise a Level Five Hardware Exception Conflict fault instead. See the fault descriptions above.

These two rules compose in a non-obvious way: if a fault is raised *inside* an L5 handler and a second L5 interrupt arrives while that fault is being serviced, the interrupt is queued (priority 7 blocks priority 6). Once the fault handler returns via `RETE` and the exception level drops back to 6, the queued L5 fires — and since the CPU is already at level 6 at that point, a Level Five Hardware Exception Conflict fault is raised then, not during the inner fault handler.

6.4 Exception Processing

6.4.1 Exception Priority

Exceptions are prioritized as follows (highest to lowest):

1. Priority 7: Faults (abort exceptions)
2. Priority 6: Level 5 hardware exceptions (NMI-like)
3. Priority 5: Level 4 hardware exceptions
4. Priority 4: Level 3 hardware exceptions
5. Priority 3: Level 2 hardware exceptions

6. Priority 2: Level 1 hardware exceptions
7. Priority 1: Software exceptions (traps)

6.4.2 Exception Handling Flow

When an exception occurs, the following steps are executed:

1. For hardware exceptions, the exception unit checks if the interrupt line is enabled in the status register. If disabled, the interrupt is ignored and not queued.
2. The exception unit checks if the exception priority level is higher than the current active exception level. If not, the exception is deferred until the current handler completes.
3. For level 5 hardware exceptions being raised when one is already being handled (both have exception level 6), a Level Five Hardware Exception Conflict fault is raised instead.
4. The current program counter, status register, and current exception level are stored in the appropriate link register based on the exception priority level.
5. The Protected Mode bit in the status register is cleared, switching the CPU to supervisor mode.
6. The current exception level is updated to the level of the exception being serviced.
7. The vector-table address is computed by multiplying the vector ID by 2 (since vectors are 32-bit addresses stored as two 16-bit words), then adding the system RAM offset.
8. The exception unit reads the high word and then the low word of the vector target address from the vector table.
9. The program counter is set to the fetched vector target address, transferring control to the exception handler.

6.4.3 Exception Entry Side Effects

For pending hardware interrupts, priority and mask checks occur before the exception-vector fetch. A hardware interrupt that is disabled or not high enough priority to preempt the current handler does not fetch its vector; if it is already latched as pending, it remains pending for later service according to the pending-interrupt rules above.

Software exceptions are also screened before the exception-vector fetch. A software exception is accepted only when no exception handler is active.

After an exception source is selected for service, the exception unit fetches the high word and low word of the handler target address from the vector table. Architectural entry side effects commit only after the vector target has been fetched and the exception is accepted for service.

When entry commits, the exception unit performs the following state updates as one architectural action:

- Writes the selected link register with the current program counter, current status register, and current exception level.
- Clears `SR.P`, entering supervisor mode.

- Clears `SR.T`, preventing trace faults from firing inside the handler.
- Sets `SR.EA`, indicating that an exception handler is active.
- Preserves the condition flags, reserved lower status bits, interrupt-enable bits, and `SR.A` in the live status register.
- Updates the current exception level to the level of the exception being serviced.
- Loads the program counter from the fetched vector target address.

The saved status register in the link register is the pre-entry value, before `SR.P`, `SR.T`, or `SR.EA` are modified. **`RETE`** restores that saved status register, so a handler can inspect or edit the saved value with **`ETFR`** and **`ETTR`** before returning.

If an exception is not accepted because its priority is not higher than the current exception level, none of these entry side effects occur. Pending hardware interrupts remain subject to the priority and pending-interrupt rules described above.

If a bus or protection fault occurs during an exception-vector fetch before entry commits, the original exception does not enter. The CPU raises the corresponding fault with `BAT0`–`BAT2` set to `ExceptionVectorFetch`. If a fault-vector fetch itself fails, the CPU escalates to double fault. If the double-fault vector fetch fails, the CPU requests reset.

Returning from Exceptions:

The RETE (Return from Exception) instruction restores the saved state from the link register:

1. The return address is loaded from the link register into the program counter
2. The saved status register value is restored (including interrupt enable bits)
3. The saved exception level is restored to current_exception_level
4. Execution resumes at the return address

After RETE, any pending interrupts that are now eligible to run (higher priority than the restored exception level and enabled) will be serviced before the next instruction executes.

6.4.4 Vector Table

The exception vectors are stored starting at address 0x0. Each vector is a 24-bit target address stored as two consecutive 16-bit words in the address format described in Section 4.5. The high word is stored first at `vector * 2`, followed by the low word at `vector * 2 + 1`. The vector table layout is:

- **0x00:** Reset vector
- **0x01:** Bus fault
- **0x02:** Alignment fault
- **0x03:** Segment overflow fault
- **0x04:** Invalid opcode fault
- **0x05:** Privilege violation fault

- **0x06:** Instruction trace fault
- **0x07:** Level five hardware exception conflict
- **0x08:** Double fault
- **0x09:** Bus protection fault
- **0x0A-0x0F:** Reserved for future privileged exceptions
- **0x10:** Level 5 hardware exception vector
- **0x20:** Level 4 hardware exception vector
- **0x30:** Level 3 hardware exception vector
- **0x40:** Level 2 hardware exception vector
- **0x50:** Level 1 hardware exception vector
- **0x60-0xFF:** User exception vectors (128 user-accessible trap vectors)

6.4.5 Exception Quick Reference

Vector	Address	Priority	Source	Maskability	Return Address	Metadata	P Mode
0x00	0x0000–0x0001	n/a	Reset	Not maskable	n/a	None	Yes
0x01	0x0002–0x0003	7	Bus fault	Not maskable	Faulting instruction	Fault metadata	Yes
0x02	0x0004–0x0005	7	Alignment fault	Not maskable	Faulting instruction	Fault metadata	Yes
0x03	0x0006–0x0007	7	Segment overflow	SR.A gated	Faulting instruction	Fault metadata	Yes
0x04	0x0008–0x0009	7	Invalid opcode	Not maskable	Faulting instruction	Fault metadata	Yes
0x05	0x000A–0x000B	7	Privilege fault	Not maskable	Faulting instruction	Fault metadata	Yes
0x06	0x000C–0x000D	7	Trace fault	SR.T gated	Next instruction	Fault metadata	Yes
0x07	0x000E–0x000F	7	L5 conflict	Not maskable	Dispatch point	Fault metadata	Yes
0x08	0x0010–0x0011	7	Double fault	Not maskable	Dispatch point	Fault metadata	Yes
0x09	0x0012–0x0013	7	Bus protection	Not maskable	Faulting instruction	Fault metadata	Yes
0x0A–0x0F	0x0014–0x001F	7	Reserved faults	Reserved	Reserved	Reserved	Reserved
0x10	0x0020–0x0021	6	Level 5 hardware	Not maskable	Next instruction	Link register 5	Yes
0x20	0x0040–0x0041	5	Level 4 hardware	SR.E4	Next instruction	Link register 4	Yes
0x30	0x0060–0x0061	4	Level 3 hardware	SR.E3	Next instruction	Link register 3	Yes
0x40	0x0080–0x0081	3	Level 2 hardware	SR.E2	Next instruction	Link register 2	Yes
0x50	0x00A0–0x00A1	2	Level 1 hardware	SR.E1	Next instruction	Link register 1	Yes
0x60–0xFF	0x00C0–0x01FF	1	Software trap	Priority gated	Next instruction	Link register 0	Yes

Table 6.2: Exception Quick Reference

6.5 Reset and Startup

Reset can be initiated externally by asserting the `RSTI` pin, or in software by executing the privileged **`RSET`** meta-instruction. Both paths converge on the same reset-vector sequence: the CPU holds reset output active, then the exception unit fetches vector 0x00 and loads the fetched target address into the program counter.

6.5.1 Reset State

State	Reset Behavior
Instruction phase	Set to phase 0.
Pending bus request	Cleared; an in-progress stalled bus request is abandoned.
Halt state	Cleared.
Wait-for-exception state	Cleared.
Status register	Cleared to 0x0000 when the reset operation is executed by the exception unit.
Program counter	Loaded from reset vector 0x00.
General registers	Not architecturally cleared by reset. Software should treat their contents as undefined after reset.
Address register pairs	Not architecturally cleared by reset.
Exception link registers	Not architecturally cleared by reset.
Pending hardware exceptions and faults	Cleared. External interrupt pins that remain asserted after reset may be sampled again according to the normal interrupt rules.
Pending coprocessor command	Set internally to the reset operation until the reset vector sequence completes.

Table 6.3: Reset State

6.5.2 Reset Output Hold

While reset is active, the CPU asserts `RSTO` and performs no bus cycles.

For hardware reset, asserting `RSTI` immediately aborts any current instruction, exception handler, exception dispatch, or pending bus wait, then asserts `RSTO`. If `RSTI` remains asserted, `RSTO` remains asserted and the CPU does not advance. After `RSTI` is released, the reset hold completes before the CPU resumes. The reset assertion cycle counts as the first reset-output cycle; after release, five additional countdown cycles occur before the exception unit starts the reset-vector fetch.

For software reset, **`RSET`** completes as a normal coprocessor meta-instruction. When its write-back phase commits the reset command, the CPU requests reset and `RSTO` is asserted for six cycles before the reset-vector fetch begins.

6.5.3 Reset Vector Fetch

The reset vector is vector 0x00 in the exception vector table. After the reset-output hold, the exception unit performs an exception-vector fetch with bus access type `ExceptionVectorFetch`:

1. Read the high word of the reset target address from `system_ram_offset + 0x0000`.
2. Read the low word of the reset target address from `system_ram_offset + 0x0001`.
3. Clear the status register to 0x0000.

4. Load the fetched target address into `p`.

If a bus or protection fault occurs during the reset-vector fetch, the reset target is not loaded. The CPU raises the corresponding fault with `BAT0`–`BAT2` set to `ExceptionVectorFetch`. From that point the normal exception-vector fetch fault rules apply: a fault-vector fetch failure escalates to double fault, and a double-fault vector fetch failure requests reset.

6.6 Return from Exception

To return from an exception handler, the `RETE` (Return from Exception) instruction is used. This instruction performs the following operations:

1. Reads the link register corresponding to the current interrupt mask level (minus 1, since the mask is set to the exception level)
2. Restores the status register from the link register (including the interrupt mask and protected mode bit)
3. Restores the program counter from the link register
4. Execution continues from the restored address

For abort exceptions (faults), `RETE` returns to the faulting instruction, allowing it to be retried. For regular exceptions, `RETE` returns to the instruction following the one that was executing when the exception occurred.

6.6.1 Accessing Link Registers

The exception coprocessor provides instructions to transfer data to and from the link registers:

- **ETFR (Exception Transfer From Register):** Copies the return address and/or status register from a link register to the address register and/or R7
- **ETTR (Exception Transfer To Register):** Copies the address register and/or R7 to a link register's return address and/or status register

These instructions allow exception handlers to examine or modify the saved processor state before returning, enabling features like system calls, context switching, and debuggers.

6.7 Exception Coprocessor Instructions

The exception coprocessor (coprocessor ID 0x1) provides several specialized instructions for exception handling and system control.

Instruction	Opcode	Privilege	Purpose
`EXCP`	0x1	User	Trigger a software exception (trap)
`WAIT`	0x9	Supervisor	Enter wait state until an exception occurs
`RETE`	0xA	Supervisor	Return from an exception handler
`RSET`	0xB	Supervisor	Perform reset sequence and jump to reset vector
`ETFR`	0xC	Supervisor	Copy saved exception state from a link register
`ETTR`	0xD	Supervisor	Write CPU register state back to a link register

Table 6.4: Exception Coprocessor Instruction Summary

For full syntax, operation details, and worked examples, see Chapter 17.

6.7.1 Internal Instructions

The following opcodes are used internally by the exception unit and are not directly accessible via user instructions:

- **Fault (0xE):** Automatically invoked when a fault condition is detected
- **HardwareException (0xF):** Automatically invoked when a hardware interrupt is signaled
- **None (0x0):** No operation, used when no exception is pending

These internal operations follow the standard exception handling flow described in the Exception Processing section.

Part II

Instruction Set Architecture

Chapter 7

Instruction Formats

7.1 Overview

All SIRCIS instructions are exactly 32 bits (4 bytes, or 2 words) in length. This fixed instruction size simplifies instruction fetch and decode logic, though it may result in larger code size compared to variable-length instruction sets.

The SIRC-1 supports three distinct instruction formats:

1. Immediate
2. Short Immediate (with Shift)
3. Register

Note: operandless assembler meta-instructions such as `NOOP` still assemble to one of these three CPU instruction formats.

Each format is optimized for different types of operations and addressing modes.

7.2 Format Overview

Format	**Typical Use**
Immediate	Operations with a 16-bit constant value
Short Immediate	Operations with an 8-bit constant with optional shift
Register	Operations on multiple registers with optional shift

Table 7.1: Instruction Format Summary

7.3 Common Fields

All instruction formats share some common fields:

Opcode (6 bits) Identifies the operation to perform. With 6 bits, up to 64 distinct operations can be encoded.

Condition Flags (4 bits) Specifies under what conditions the instruction should execute. See **Chapter 10** for details.

Additional Flags (2 bits) Used in memory operations to specify address register pairs (a, p, s, l) and in ALU operations to specify how the status register is updated.

7.4 Immediate Format

7.4.1 Structure

The immediate format encodes a full 16-bit immediate value within the instruction.

Note: Due to the large 16-bit operand, shift is not supported in this format.

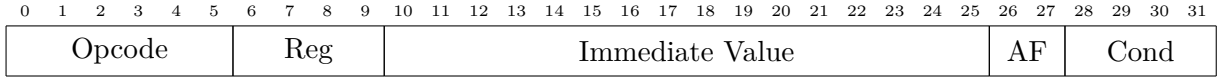

Figure 7.1: Immediate Instruction Format

Opcode (bits 31–26) 6-bit operation code

Register (bits 25–22) 4-bit destination/source register identifier

Immediate (bits 21–6) 16-bit signed or unsigned immediate value

Additional Flags (bits 5–4) 2-bit address register pair selector (for memory operations)

- 00 = l (link register)
- 01 = a (address register)
- 10 = s (stack pointer)
- 11 = p (program counter)

or 2-bit status register update source selector (for ALU operations)

- 00 = Status register not updated
- 01 = Status register updated from ALU operation
- 10 = Status register updated from shift operation
- 11 = Reserved for future use

Condition (bits 3–0) 4-bit condition code

7.4.2 Encoding Examples

Instruction	Opcode	Reg	Immediate	AF	Cond	Hex
`ADDI r1, #100`	0x00 (000000)	0x1 (0001)	0x0064 (0000 0000 0110 0100)	0b01 (01)	0x0 (0000)	`0x00401910`
`CMPI r2, #0x8000`	0x0A (001010)	0x2 (0010)	0x8000 (1000 0000 0000 0000)	0b01 (01)	0x0 (0000)	`0x28A00010`
`LOAD r3, (#16, a)`	0x14 (010100)	0x3 (0011)	0x0010 (0000 0000 0001 0000)	0b01 (01)	0x0 (0000)	`0x50C00410`
`STOR (#-2, s), r4`	0x10 (010000)	0x4 (0100)	0xFFFE (1111 1111 1111 1110)	0b10 (10)	0x0 (0000)	`0x413FFFA0`
`LDEA a, (#4, s)`	0x18 (011000)	0x1 (0001)	0x0004 (0000 0000 0000 0100)	0b10 (10)	0x0 (0000)	`0x60400120`
`LDEA a, -(#0, s)`	0x1A (011010)	0x1 (0001)	0x0000 (0000 0000 0000 0000)	0b10 (10)	0x0 (0000)	`0x68400020`
`LDEL p, (#0, a)`	0x1C (011100)	0x3 (0011)	0x0000 (0000 0000 0000 0000)	0b01 (01)	0x0 (0000)	`0x70C00010`
`LDEL p, (#0, a)+`	0x1E (011110)	0x3 (0011)	0x0000 (0000 0000 0000 0000)	0b01 (01)	0x0 (0000)	`0x78C00010`
`BRAN #4`	0x18 (011000)	0x3 (0011)	0x0004 (0000 0000 0000 0100)	0b11 (11)	0x0 (0000)	`0x60C00130`
`BRSR #4`	0x1C (011100)	0x3 (0011)	0x0004 (0000 0000 0000 0100)	0b11 (11)	0x0 (0000)	`0x70C00130`
`ADDI|== r5, #42`	0x00 (000000)	0x5 (0101)	0x002A (0000 0000 0010 1010)	0b01 (01)	0x1 (0001)	`0x01400A91`
`SUBI|>= r6, #10`	0x02 (000010)	0x6 (0110)	0x000A (0000 0000 0000 1010)	0b01 (01)	0xB (1011)	`0x0980029B`

Table 7.2: Immediate Format Encoding Examples

Notes:

- Condition code examples: 0x0 (0000) = Always, 0x1 (0001) = Equal (==), 0xB (1011) = Signed >=
- AF field for memory ops: 00=l, 01=a, 10=s, 11=p; for ALU ops: 00=no update, 01=update from ALU, 10=update from shift
- Negative immediates use two's complement (#-2 = 0xFFFE)

7.5 Short Immediate Format (with Shift)

7.5.1 Structure

The short immediate format sacrifices immediate value width to include shift information, allowing for more powerful single-instruction operations.

Only supported for ALU instructions - cannot be used with any of the branching, or load/store instructions.

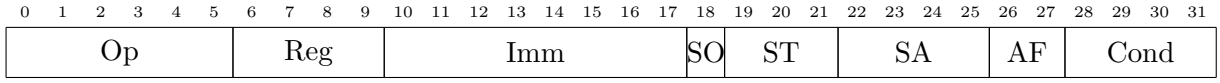

Figure 7.2: Short Immediate with Shift Format

Opcode (bits 31–26) 6-bit operation code

Register (bits 25–22) 4-bit destination/source register identifier

Immediate (bits 21–14) 8-bit signed or unsigned immediate value

Shift Operand (bit 13) Determines how the shift amount field is interpreted

- 0 = Shift amount field contains the literal shift amount
- 1 = Shift amount field refers to register that contains shift amount

Shift Type (bits 12–10) Type of shift operation (see Table 7.6 for encoding details)

Shift Amount (bits 9–6) Shift amount (or register containing shift amount)

Additional Flags (bits 5–4) 2-bit status register update source selector

- 00 = Status register not updated
- 01 = Status register updated from ALU operation
- 10 = Status register updated from shift operation
- 11 = Reserved for future use

Condition (bits 3–0) 4-bit condition code

7.5.2 Encoding Examples

Instruction	Op	Reg	Imm	SO	ST	SA	AF	Cond	Hex
`ADDI r1, #2, LSL #3`	0x20 (100000)	0x1 (0001)	0x02 (00000010)	0 (0)	001 (001)	0x3 (0011)	0b01 (01)	0x0 (0000)	`0x804084D0`
`ORRI r2, #1, LSL #10`	0x25 (100101)	0x2 (0010)	0x01 (00000001)	0 (0)	001 (001)	0xA (1010)	0b01 (01)	0x0 (0000)	`0x94804690`
`ANDI r3, #7, ASR #2`	0x24 (100100)	0x3 (0011)	0x07 (00000111)	0 (0)	100 (100)	0x2 (0010)	0b01 (01)	0x0 (0000)	`0x90C1D090`
`SUBI r4, #5, LSL r2`	0x22 (100010)	0x4 (0100)	0x05 (00000101)	1 (1)	001 (001)	0x2 (0010)	0b01 (01)	0x0 (0000)	`0x89016490`
`XORI|!= r7, #15, LSR #4`	0x26 (100110)	0x7 (0111)	0x0F (00001111)	0 (0)	010 (010)	0x4 (0100)	0b01 (01)	0x2 (0010)	`0x99C3C912`
`CMPI|NS r1, #8, RTL r3`	0x2A (101010)	0x1 (0001)	0x08 (00001000)	1 (1)	101 (101)	0x3 (0011)	0b01 (01)	0x5 (0101)	`0xA84234D5`
`SHFT r3, RTR #5`	0x25 (100101)	0x3 (0011)	0x00 (00000000)	0 (0)	110 (110)	0x5 (0101)	0b10 (10)	0x0 (0000)	`0x94C01960`

Table 7.3: Short Immediate Format Encoding Examples

Notes:

- SO (Shift Operand): 0 = literal shift amount, 1 = register contains shift amount (see rows 4 and 6)
- ST (Shift Type): 000=none, 001=LSL, 010=LSR, 011=ASL, 100=ASR, 101=RTL, 110=RTR
- AF field for ALU: 00=no update, 01=update from ALU, 10=update from shift (see SHFT row 7)
- Condition code examples: 0x0 = Always, 0x2 = Not Equal (!=), 0x5 = Negative (<0)
- SHFT is a meta instruction using ORRI opcode (0x25) with AF=0b10

7.6 Register Format

7.6.1 Structure

The register format allows operations on up to three registers with optional shift.

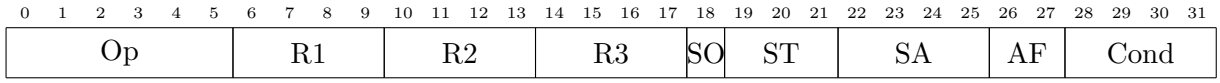

Figure 7.3: Register Instruction Format

Opcode (bits 31–26) 6-bit operation code

Register 1 (bits 25–22) 4-bit destination register

Register 2 (bits 21–18) 4-bit source A register

Register 3 (bits 17–14) 4-bit source B register

Shift Operand (bit 13) Determines how the shift amount field is interpreted

- 0 = Shift amount field contains the literal shift amount
- 1 = Shift amount field refers to register that contains shift amount

Shift Type (bits 12–10) Type of shift operation (see Table 7.6 for encoding details)

Shift Amount (bits 9–6) Shift amount (or register containing shift amount)

Additional Flags (bits 5–4) 2-bit address register pair selector (for memory operations)

- 00 = l (link register)
- 01 = a (address register)
- 10 = s (stack pointer)
- 11 = p (program counter)

or 2-bit status register update source selector (for ALU operations)

- 00 = Status register not updated
- 01 = Status register updated from ALU operation
- 10 = Status register updated from shift operation
- 11 = Reserved for future use

Condition (bits 3–0) Condition code

7.6.2 Register Operand Semantics

For most three-operand instructions:

- R1 = Destination
- R2 = First source operand
- R3 = Second source operand (optional)

Note: If R3 is not specified, the assembler with use R1 as both destination and first source operand when encoding the instruction:

```
1 ADDR r1, r2, r3   ; r1 = r2 + r3
2 ADDR r1, r2       ; r1 = r1 + r2 (R3 implicitly = R1)
```

7.6.3 Encoding Examples

Instruction	Op	R1	R2	R3	SO	ST	SA	AF	Cond	Hex
`ADDR r1, r2, r3`	0x30 (110000)	0x1 (0001)	0x2 (0010)	0x3 (0011)	0 (0)	000 (000)	0x0 (0000)	0b01 (01)	0x0 (0000)	0xC048C010
`ADDR r1, r2, r3, LSL #2`	0x30 (110000)	0x1 (0001)	0x2 (0010)	0x3 (0011)	0 (0)	001 (001)	0x2 (0010)	0b01 (01)	0x0 (0000)	0xC048C490
`LOAD r1, (r2, a)`	0x15 (010101)	0x1 (0001)	0x0 (0000)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b01 (01)	0x0 (0000)	0x54408010
`STOR -(r2, s), r1`	0x13 (010011)	0x0 (0000)	0x1 (0001)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b10 (10)	0x0 (0000)	0x4C048020
`LDEA a, (r2, s)`	0x19 (011001)	0x1 (0001)	0x0 (0000)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b10 (10)	0x0 (0000)	0x64408020
`LDEA a, -(r2, s)`	0x1B (011011)	0x1 (0001)	0x0 (0000)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b10 (10)	0x0 (0000)	0x6C408020
`LDEL p, (r2, a)`	0x1D (011101)	0x3 (0011)	0x0 (0000)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b01 (01)	0x0 (0000)	0x74C08010
`LDEL p, (r2, a)+`	0x1F (011111)	0x3 (0011)	0x0 (0000)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b01 (01)	0x0 (0000)	0x7CC08010
`LJMP a, r2`	0x19 (011001)	0x3 (0011)	0x0 (0000)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b01 (01)	0x0 (0000)	0x64C08010
`LJSR a, r2`	0x1D (011101)	0x3 (0011)	0x0 (0000)	0x2 (0010)	0 (0)	000 (000)	0x0 (0000)	0b01 (01)	0x0 (0000)	0x74C08010
`CMPR r4, r5`	0x3A (111010)	0x4 (0100)	0x4 (0100)	0x5 (0101)	0 (0)	000 (000)	0x0 (0000)	0b01 (01)	0x0 (0000)	0xE9114010
`XORR r2, r3, r4, RTR #1`	0x36 (110110)	0x2 (0010)	0x3 (0011)	0x4 (0100)	0 (0)	110 (110)	0x1 (0001)	0b01 (01)	0x0 (0000)	0xD88D1850
`SUBR|» r1, r2, r3, ASR r4`	0x32 (110010)	0x1 (0001)	0x2 (0010)	0x3 (0011)	1 (1)	100 (100)	0x4 (0100)	0b01 (01)	0xD (1101)	0xC848F11D
`ANDR|CS r5, r6, r7, LSL r1`	0x34 (110100)	0x5 (0101)	0x6 (0110)	0x7 (0111)	1 (1)	001 (001)	0x1 (0001)	0b01 (01)	0x3 (0011)	0xD159E453
`SHFT r1, ASL #3`	0x25 (100101)	0x1 (0001)	0x0 (0000)	0x0 (0000)	0 (0)	011 (011)	0x3 (0011)	0b10 (10)	0x0 (0000)	0x94400CE0

Table 7.4: Register Format Encoding Examples

Notes:

- When R3 is not specified in assembly, it defaults to R1 (e.g., `ADDR r4, r5` uses R1=0x4, R2=0x4, R3=0x5)
- SO (Shift Operand): 0 = literal shift amount, 1 = register contains shift amount (see rows 7 and 8)
- ST (Shift Type): 000=none, 001=LSL, 010=LSR, 011=ASL, 100=ASR, 101=RTL, 110=RTR
- AF for memory ops: 00=l, 01=a, 10=s, 11=p; for ALU ops: 00=no update, 01=update from ALU, 10=update from shift (see SHFT row 9)
- Condition code examples: 0x0 = Always, 0xD = Signed > (»), 0x3 = Carry Set (CS)
- SHFT is a meta instruction using ORRI opcode (0x25) with AF=0b10

7.7 Operand Encoding Details

7.7.1 Condition Codes

Code	Mnemonic	Condition
0000	(none)	Always (unconditional)
0001	==	Equal (Z = 1)
0010	!=	Not Equal (Z = 0)
0011	CS	Carry Set (C = 1)
0100	CC	Carry Clear (C = 0)
0101	NS	Negative Set (N = 1)
0110	NC	Negative Clear (N = 0)
0111	OS	Overflow Set (V = 1)
1000	OC	Overflow Clear (V = 0)
1001	HI	Unsigned Higher (C = 1 AND Z = 0)
1010	LO	Unsigned Lower or Same (C = 0 OR Z = 1)
1011	>=	Signed Greater or Equal (N = V)
1100	<	Signed Less Than (N ≠ V)
1101	»	Signed Greater Than (Z = 0 AND N = V)
1110	«	Signed Less or Equal (Z = 1 OR N ≠ V)
1111	NV	Never (never executes)

Table 7.5: Condition Code Encoding

For more details on condition codes see **Chapter 10**

7.7.2 Shift Types

Code	Type	Mnemonic
000	None (no shift)	–
001	Logical Shift Left	LSL
010	Logical Shift Right	LSR
011	Arithmetic Shift Left	ASL
100	Arithmetic Shift Right	ASR
101	Rotate Left	RTL
110	Rotate Right	RTR
111	Reserved	–

Table 7.6: Shift Type Encoding

For more details on shift operations see **Chapter 9**

7.8 Opcode Encoding Patterns

The SIRCIS instruction set uses systematic opcode patterns to simplify decoding:

7.8.1 ALU Instructions

- **0x0_**: Immediate operand
- **0x1_**: Memory operations
- **0x2_**: Short immediate with shift
- **0x3_**: Register operand

7.8.2 Test vs. Save

All ALU operations have both a "save result" and "test only" variant:

- **0x_0-0x_7**: Save result to destination
- **0x_8-0x_F**: Test only (update flags, don't save result)

To turn a "save result" instruction to a "test only" instruction, you can simply add 0x8 to the opcode.

For example:

- `0x04`: **`ANDI`** – AND immediate, save result
- `0x0C`: **`TSAI`** – Test AND immediate (don't save result)

7.9 Instruction Fetch and Alignment

7.9.1 Fetch Process

Since instructions are 32 bits and the data bus is 16 bits wide, each instruction requires two fetch cycles:

1. Fetch high word (bits 31–16) from address in PC
2. Increment PC by 1
3. Fetch low word (bits 15–0) from address in PC
4. Increment PC by 1

7.9.2 Alignment Requirements

Instructions span two consecutive words and must begin at an even word address. The program counter (`pl`) must always be even.

Attempting to fetch an instruction from an odd word address will trigger an alignment fault exception.

7.10 Format Selection Guidelines

When writing assembly code, the assembler automatically selects the appropriate format based on operands:

Assembly	Format Used	Notes
`ADDI r1, #1000`	Immediate	Full 16-bit immediate
`ADDI r1, #10, LSL #2`	Short Immediate	8-bit immediate + shift
`ADDR r1, r2, r3`	Register	Three register operands

Table 7.7: Format Selection Examples

The choice of format affects instruction encoding but is generally transparent to the programmer, as the assembler handles the details.

Chapter 8

Addressing Modes

8.1 Overview

The SIRC-1 CPU supports seven addressing modes that specify how operands are accessed. These modes determine whether an operand is an immediate value, register contents, or data in memory at a computed address.

Mode	Description
Immediate	Operand is a constant value encoded in the instruction
Register Direct	Operand is the contents of a specified register
Indirect Immediate	Memory address = address register + immediate offset
Indirect Register	Memory address = address register + register offset
Post-Increment	Indirect register, then increment address register
Pre-Decrement	Decrement address register, then use as indirect
Short Immediate	8-bit constant with optional shift

Table 8.1: Addressing Modes Summary

8.2 Addressing Mode Matrix

Mode	Syntax	Effective value or address	Memory access	Legal families
Immediate	`#imm16`	16-bit encoded value	None	ALU, LOAD-immediate, COPI, branch meta-instructions
Short Immediate	`#imm8, shift`	8-bit zero-extended value; shift applies to register source	None	ALU short-immediate
Register Direct	`rN`	16-bit register value	None	ALU, LOAD register-copy, COPR, register-offset forms
Indirect Immediate	`(#offset, addr)`, `(addr)`	`addr.high : (addr.low + offset)`	LOAD reads; STOR writes; LDEA/LDEL none	Memory, LDEA, LDEL
Indirect Register	`(r0, addr)`	`addr.high : (addr.low + r0)`	LOAD reads; STOR writes; LDEA/LDEL none	Memory, LDEA, LDEL
Post-Increment	`(#offset, addr)+`, `(addr)+`, `(r0, addr)+`	Address is calculated from original `addr`; `addr.low += 1` after access	LOAD reads; LDEL none	LOAD, LDEL
Pre-Decrement	`-(#offset, addr)`, `-(addr)`, `-(r0, addr)`	`addr.low -= 1`; address then uses decremented `addr.low + offset`	STOR writes; LDEA none	STOR, LDEA

Table 8.2: Addressing Mode Side Effects and Legal Instruction Families

8.3 Legal Modes by Instruction Family

Instruction family	Legal operand forms	Explicit exclusions and notes
ALU result instructions	`OPI rD, #imm16`; `OPI rD, #imm8, shift`; `OPR rD, rS1, rS2[, shift]`; register shorthand `OPR rD, rS2`.	No memory operands. Full 16-bit immediate forms do not encode shifts.
ALU test instructions	`CMPI/TSAI/TSXI rS, #imm16`; `CMPI/TSAI/TSXI rS, #imm8, shift`; `CMPR/TSAR/TSXR rS1, rS2[, shift]`.	No destination write-back. No memory operands.
LOAD value	`LOAD rD, #imm16`; `LOAD rD, rS`.	No public short-immediate LOAD syntax. Direct register-copy forms do not accept shift suffixes.
LOAD memory	`LOAD rD, (#offset, addr)`; `LOAD rD, (rO, addr)[, shift]`; `LOAD rD, (#offset, addr)+`; `LOAD rD, (rO, addr)+[, shift]`.	No pre-decrement LOAD form. Shift suffixes are only available on register-displacement memory forms.
STOR memory	`STOR (#offset, addr), rS`; `STOR (rO, addr), rS[, shift]`; `STOR -(#offset, addr), rS`; `STOR -(rO, addr), rS[, shift]`.	No post-increment STOR form. Shift suffixes are only available on register-displacement memory forms.
LDEA	`LDEA dest, (#offset, src)`; `LDEA dest, (rO, src)`; `LDEA dest, -(#offset, src)`; `LDEA dest, -(rO, src)`.	No post-increment LDEA form. No shift suffixes.
LDEL	`LDEL dest, (#offset, src)`; `LDEL dest, (rO, src)`; `LDEL dest, (#offset, src)+`; `LDEL dest, (rO, src)+`.	No pre-decrement LDEL form. No shift suffixes.
Control-flow meta-instructions	`BRAN/BRSR #disp`; `BRAN/BRSR @label`; `RETS`; `LJMP src[, #offset|rO]`; `LJSR src[, #offset|rO]`; `LJSR (#offset, src)+`.	`LJMP (src)` and `LJSR (src)` are not public syntax; use `LJMP src` or `LJSR src` for zero displacement.
Coprocessor	`COPI #imm16`; `COPR rS`; standard coprocessor meta-instructions as documented in Chapter 16.	No hidden destination register syntax, no memory operands, no shifts, and no status-update override suffixes.

Table 8.3: Addressing Mode Legality by Instruction Family

8.4 Zero-Displacement Shorthand

Immediate-displacement indirect forms may omit the displacement when it is zero. The following forms are exact assembler-level equivalents:

Shorthand	Canonical form	Meaning
`(addr)`	`(#0, addr)`	Use the selected address register
`(addr)+`	`(#0, addr)+`	Use the address, then increment
`-(addr)`	`-(#0, addr)`	Decrement, then use the address

Table 8.4: Zero-Displacement Shorthand

The shorthand does not change which instruction families are legal. For example, `LOAD r1, (a)+` is legal because **`LOAD`** has a post-increment immediate-displacement form, but `STOR (a)+, r1` is not legal because **`STOR`** has no post-increment form. Likewise, `STOR -(s), r1` is legal because **`STOR`** has a pre-decrement form, but the corresponding pre-decrement **`LOAD`** form is not legal.

Register-displacement forms do not have a corresponding shorthand: `(rO, addr)`, `(rO, addr)+`, and `-(rO, addr)` always name the displacement register explicitly.

8.5 Operand Categories

Instruction descriptions use these operand categories:

Register value A 16-bit general-purpose, status, or address-component register operand used directly by an ALU, load, store, or coprocessor instruction.

Immediate value A constant encoded in the instruction. Full immediates are 16 bits; short immediates are 8 bits.

Memory source An effective address read by `LOAD`.

Memory destination An effective address written by `STOR`.

Branch target An effective address written to the program-counter pair `p`.

Address destination An address-register pair written by `LDEA` or `LDEL`.

Alterable destination Any destination register, address-register pair, or memory location that the instruction can modify when its condition is true.

8.6 Common Address Calculation Rules

For indirect, post-increment, pre-decrement, branch, jump, and effective-address forms, address arithmetic is performed on the low word of the selected address-register pair. The high word supplies the segment and is preserved unless the instruction explicitly writes an address-register pair destination.

Post-increment and pre-decrement change the selected address register by exactly one word. The displacement or index register participates only in the effective address calculation; it does not change the auto-update amount.

When `p` is used as the source address register, the value used for address calculation is the program counter after the current instruction has been fetched. Since every instruction is two words, PC-relative branch displacements are relative to the next instruction address, not to the address of the branch instruction itself.

8.7 Immediate Addressing

8.7.1 Description

The operand is a constant value 16-bit value encoded directly in the instruction.

8.7.2 Syntax

```
1 ADDI r1, #100          ; 16-bit immediate
2 ADDI r2, #-50          ; Signed 16-bit immediate
```

8.7.3 Effective Value

16-bit value encoded into the instruction. Arithmetic instructions interpret this field according to Chapter 4.

8.7.4 Usage

- Loading constant values
- Arithmetic with known values
- Bit manipulation with masks

8.8 Register Direct Addressing

8.8.1 Description

The operand is the contents of a specified register. This is the most common mode for ALU operations in a load/store architecture.

8.8.2 Syntax

```
1 ADDR r1, r2, r3    ; r1 = r2 + r3
2 ADDR r4, r5        ; r4 = r4 + r5
```

8.8.3 Effective Value

The 16-bit value currently stored in the specified register.

8.8.4 Usage

- All ALU operations between registers
- Register-to-register data movement
- Testing register values

8.9 Indirect Immediate Addressing

8.9.1 Description

The effective address is computed by adding a 16-bit signed immediate offset to an address register pair low word. Memory instructions use the computed address for a load or store; effective-address instructions write the computed address to an address-register pair.

8.9.2 Syntax

```
1 LOAD r1, (#8, a)       ; Load from address (a + 8)
2 STOR (#-4, s), r2      ; Store to address (s - 4)
3 LDEA p, (#100, p)      ; Jump relative (pc = pc + 100)
```

8.9.3 Effective Address

$$\text{EA} = \text{AddressRegister.high} : (\text{AddressRegister.low} + \text{Offset}) \tag{8.1}$$

The address register can be: `l` (link), `a` (address), `s` (stack), or `p` (program counter).

8.9.4 Usage

- Accessing structure fields (fixed offset from base)
- Array element access (if index is constant)
- Stack frame access (local variables)
- Position-independent code (PC-relative addressing)

8.10 Indirect Register Addressing

8.10.1 Description

Similar to indirect immediate, but the offset is taken from a register instead of being encoded as an immediate value. This enables runtime-computed offsets.

8.10.2 Syntax

```
1 LOAD r1, (r2, a)       ; Load from address (a + r2)
2 STOR (r4, s), r3       ; Store to address (s + r4)
```

8.10.3 Effective Address

$$\text{EA} = \text{AddressRegister.high} : (\text{AddressRegister.low} + \text{IndexRegister}) \tag{8.2}$$

8.10.4 Usage

- Array indexing with variable index
- Table lookups
- Dynamic offset calculations

8.11 Post-Increment Addressing

8.11.1 Description

The effective address is computed using indirect register addressing, then the address register pair is incremented after the memory operation.

This mode is particularly useful for iterating through arrays or implementing stack operations.

8.11.2 Syntax

```
1 LOAD r1, (r2, s)+      ; Load from (s + r2), then s = s + 1
2 LOAD r1, (#2, s)+      ; Load from (s + 2), then s = s + 1
```

8.11.3 Operation Sequence

1. Compute EA = AddressRegister.high : (AddressRegister.low + Offset)
2. Perform memory operation (load)
3. AddressRegister.low = AddressRegister.low + 1

8.11.4 Usage

- Stack pop operations
- Forward array traversal
- Buffer reading

8.12 Pre-Decrement Addressing

8.12.1 Description

The address register pair is decremented before computing the effective address for the memory operation.

This mode is the complement of post-increment and is useful for stack push operations.

8.12.2 Syntax

```
STOR -(r2, s), r1      ; s = s - 1, then store to (s + r2)
STOR -(#2, s), r3      ; s = s - 1, then store to (s + 2)
```

8.12.3 Operation Sequence

1. AddressRegister.low = AddressRegister.low - 1
2. Compute EA = AddressRegister.high : (AddressRegister.low + Offset)
3. Perform memory operation (store)

8.12.4 Usage

- Stack push operations
- Backward array traversal
- Buffer writing

8.13 Short Immediate Addressing

8.13.1 Description

Similar to immediate addressing, but limited to 8-bit values so shift information can fit into the instruction encoding.

8.13.2 Syntax

```
ADDI r1, #10, LSL #2   ; r1 = (r1 << 2) + 10
SUBI r2, #1, LSL #8    ; r2 = (r2 << 8) - 1
```

8.13.3 Effective Value

8-bit value encoded into the instruction and zero-extended before ALU arithmetic.

The shift is applied to the source operand, not the constant.

8.13.4 Usage

- Loading constant values
- Arithmetic with known values
- Bit manipulation with masks

8.14 Address Register Selection

Many addressing modes require specifying which address register pair to use. The 2-bit address register field encodes:

Bits	Symbol	Register Pair
00	l	Link Register (lh, ll)
01	a	Address Register (ah, al)
10	s	Stack Pointer (sh, sl)
11	p	Program Counter (ph, pl)

Table 8.5: Address Register Encoding

8.15 Addressing Mode Examples

8.15.1 Structure Access

```
; Assume 'a' points to a struct:
; struct Point {
;   word 0: int16_t x
;   word 1: int16_t y
; }

LOAD r1, (#0, a)       ; r1 = point.x
LOAD r2, (#1, a)       ; r2 = point.y
```

8.15.2 Array Iteration

```
; Iterate array of 16-bit values
; Assume 'a' points to array start
; r7 = counter
loop:
  LOAD r1, (#0, a)+   ; Load element, advance by 1
  ; ... process r1 ...
  SUBI r7, #1
  BRAN|!= @loop
```

8.15.3 Stack Frame

```
:begin_subroutine

; Function prologue - save registers
STOR -(#0, s), r1
STOR -(#0, s), r2
STOR -(#0, s), r3

; Access parameters (above saved regs)
LOAD r4, (#3, s)      ; First parameter
LOAD r5, (#4, s)      ; Second parameter

; Function epilogue - restore registers
LOAD r3, (#0, s)+
LOAD r2, (#0, s)+
LOAD r1, (#0, s)+
RETS
```

8.16 Addressing Mode Restrictions

8.16.1 Privilege Mode Restrictions

Protected mode restricts writes to the high word of an address register:

- **Direct high-register writes**: Trigger a privilege violation fault
- **Address-register-pair write-back**: Allowed only when the high word would remain unchanged; otherwise triggers a privilege violation fault
- **Post-increment/Pre-decrement**: Allowed in protected mode only when the high word would remain unchanged

8.16.2 Alignment

- All memory addresses refer to 16-bit words
- There are no alignment restrictions for general memory I/O
- All instructions span 2 words and must begin at an even word address

- Unaligned instruction fetches (odd word address) trigger an alignment fault exception

Chapter 9

Shift Operations

9.1 Overview

The SIRC-1 CPU supports powerful shift operations that can be applied to operands in many instructions. Shifts can multiply or divide values by powers of 2, extract bit fields, or perform bit rotations.

Shift operations are encoded within Short Immediate and Register format instructions, allowing a value to be shifted before being used in an ALU operation—all within a single instruction.

Shift operations are only applied to source operands. A shift cannot be applied to the result of an operation before register writeback.

Note: By default, when shifts are used with ALU instructions (apart from the SHFT meta instruction), the status register flags are updated based on the ALU operation result, not the shift. However, you can manually control this behavior using the status override syntax (see Section 9.12).

9.2 Shift Types

9.2.1 Shift Type Encoding

Code	Mnemonic	Type	Description
000	NUL (or none)	None	No shift performed
001	LSL	Logical Shift Left	Shift left, fill with zeros
010	LSR	Logical Shift Right	Shift right, fill with zeros
011	ASL	Arithmetic Shift Left	Same as LSL (shifts in zeros)
100	ASR	Arithmetic Shift Right	Shift right, preserving sign bit
101	RTL	Rotate Left	Rotate left within the operand
110	RTR	Rotate Right	Rotate right within the operand
111	–	Reserved	Reserved for future use

Table 9.1: Shift Type Encoding

9.3 Shift Syntax

A shift postamble can come after any short immediate or register instruction. It is applied in the "Decode and Register Fetch" phase of instruction execution, before any ALU operations or memory address calculation occurs.

It is applied to the first source operand of an instruction. If using the three operand form of instruction, it will be the second operand. If using the two operand form of instruction, the second operand is inferred to be the same as the destination.

See this table for some examples of which operand is shifted:

Instruction	How the CPU Interprets It	First Source Operand
ADDR r1, r2, r3, LSL #1	ADDR r1, r2, r3, LSL #1	r2
ADDR r1, r2, LSL #1	ADDR r1, r1, r2, LSL #1	r1
ADDI r1, #2, LSL #1	ADDI r1, r1, #2, LSL #1	r1

Table 9.2: Examples of first source operands

9.4 Shift Type Details

9.4.1 Logical Shift Left (LSL)

Operation: Shift bits to the left, filling vacated bits with zeros.

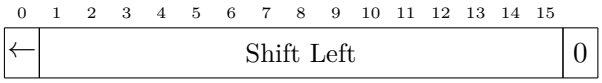

Figure 9.1: Logical Shift Left

- Bit 15 is shifted into the carry flag
- Bit 0 receives 0
- Equivalent to multiplying by 2^n where n is the shift count

Example:

```
; Multiply r1 by 4
ADDI r1, #0, LSL #2   ; r1 = (r1 << 2) = r1 * 4

; Set bit 10
ORRI r3, #1, LSL #10      ; r3 |= (r3 << 10) with immediate 1 = 0x0400
```

9.4.2 Logical Shift Right (LSR)

Operation: Shift bits to the right, filling vacated bits with zeros.

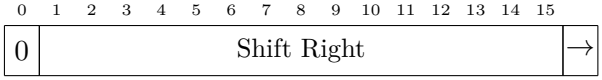

Figure 9.2: Logical Shift Right

- Bit 0 is shifted into the carry flag
- Bit 15 receives 0
- Equivalent to unsigned division by 2^n
- Use for unsigned values only

Example:

```
; Divide unsigned value by 8
ADDI r1, #0, LSR #3   ; r1 = (r1 >> 3) = r1 / 8 (unsigned)

; Extract upper byte
ADDI r3, #0, LSR #8   ; r3 = (r3 >> 8)
```

9.4.3 Arithmetic Shift Left (ASL)

Operation: Identical to LSL. Shift bits to the left, filling with zeros.

- Equivalent to signed multiplication by 2^n
- Can cause signed overflow if sign bit changes
- Overflow flag is set if sign bit changes during shift

Example:

```
; Multiply signed value by 2
ADDI r1, #0, ASL #1   ; r1 = (r1 << 1) = r1 * 2 (signed)
```

9.4.4 Arithmetic Shift Right (ASR)

Operation: Shift bits to the right, preserving the sign bit (bit 15).

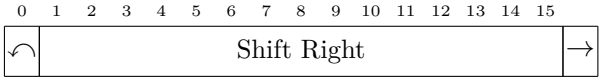

Figure 9.3: Arithmetic Shift Right

- Bit 0 is shifted into the carry flag
- Bit 15 (sign bit) is copied into bit 15 and bit 14
- Equivalent to signed division by 2^n (rounds toward negative infinity)
- Use for signed values only

Example:

```
; Divide signed value by 4
ADDI r1, #0, ASR #2   ; r1 = (r1 >> 2) = r1 / 4 (signed)

; Average two signed values
ADDR r3, r4, r5
ADDI r3, #0, ASR #1   ; r3 = (r3 >> 1) = (r4 + r5) / 2
```

9.4.5 Rotate Left (RTL)

Operation: Rotate bits to the left within the 16-bit operand. The bit that wraps from bit 15 to bit 0 is also copied into the carry flag. The incoming carry flag is not used as an input.

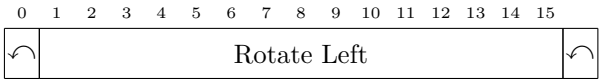

Figure 9.4: Rotate Left

- Bit 15 wraps into bit 0 and is copied into the carry flag
- The previous carry flag value is ignored
- No data is lost
- Useful for bit manipulation, circular buffers, and cyclic bit patterns

Example:

```
; Rotate r1 left by 1
SHFT r1, RTL #1

; Circular rotate of one word
SHFT r3, RTL #4
```

9.4.6 Rotate Right (RTR)

Operation: Rotate bits to the right within the 16-bit operand. The bit that wraps from bit 0 to bit 15 is also copied into the carry flag. The incoming carry flag is not used as an input.

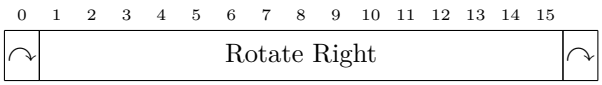

Figure 9.5: Rotate Right

- Bit 0 wraps into bit 15 and is copied into the carry flag
- The previous carry flag value is ignored
- No data is lost

Example:

```
; Rotate r1 right by 1
SHFT r1, RTR #1

; Check if bit 0 is set
SHFT r2, RTR #1       ; Bit 0 -> carry
; Now branch on carry to test the old bit 0
```

9.5 Shift Operands

The Shift Operand (SO) bit determines what value is shifted:

SO Bit	Mode	Description
0	Shift by Immediate	R2 (first source operand) is shifted by a literal amount
1	Shift Count in Register	R2 is shifted by the amount in the shift count register

Table 9.3: Shift Operand Modes

9.5.1 Mode 0: Shift by Immediate

The first source operand (R2) is shifted by a literal amount before the ALU operation.

```
; Add (r1 * 4) + 1 to r1
ADDI r1, #1, LSL #2   ; r1 = r1 + (r2 << 2) + 1
                      ; SO=0: r2 is shifted by literal 2
```

9.5.2 Mode 1: Shift by Register Count

The shift count is taken from the shift amount field (interpreted as a register), allowing variable-length shifts. R2 (first source) is still what gets shifted.

```
; Variable shift
; Shift amount register contains shift count
ADDR r1, r2, r3, LSL r4   ; r1 = (r2 << r4) + r3
                          ; SO=1: shift r2 by value in r4

; Barrel shifter emulation
ADDR r4, r5, r6, ASR r7   ; r4 = (r5 >> r7) + r6 (arithmetic)
```

9.6 Shift Count

The shift count is a 4-bit field, allowing shifts of 0–15 positions.

- Shift count of 0 means no shift is performed
- Maximum shift count is 15
- Shift counts $\geq$ 16 would be meaningless for 16-bit values

9.7 Status Flag Effects

9.7.1 Default Behavior

When shift operations are used with ALU instructions, the status flags are **by default** updated based on the **ALU operation result**, not the shift operation. For example, in `CMPI r2, #1, LSL #2`, the flags reflect the comparison result, not the shift.

To update flags based on the shift operation instead, use the `[S]` status override modifier (see Section 9.12), or use the **`SHFT`** meta-instruction documented in Chapter 13.

9.7.2 Carry Flag

For all shifts and rotates:

- The last bit shifted out is placed in the carry flag
- For shifts of more than 1 position, only the final bit shifted out matters
- If shift count is 0, carry flag is not modified

9.7.3 Zero Flag

Set if the result after shifting is zero.

9.7.4 Negative Flag

Set if bit 15 of the result is 1 (result is negative when interpreted as signed).

9.7.5 Overflow Flag

- For **ASL**: Set if the sign bit changes during the shift (signed overflow)
- For other shifts: Behavior is undefined; typically unchanged or cleared

9.8 Common Use Cases

9.8.1 Multiplication by Constants

```
; Multiply by 2
ADDI r1, #0, LSL #1    ; r1 = (r1 << 1) = r1 * 2

; Multiply by 3
ADDR r1, r2, r2, LSL #1    ; r1 = (r2 << 1) + r2 = r2 * 3

; Multiply by 5
ADDR r1, r2, r2, LSL #2    ; r1 = (r2 << 2) + r2 = r2 * 5
```

9.8.2 Division by Powers of 2

```
; Unsigned divide by 4
LOAD r1, r2          ; r1 = r2
ADDI r1, #0, LSR #2   ; r1 = (r1 >> 2) = r1 / 4 (unsigned)

; Signed divide by 8
LOAD r1, r2          ; r1 = r2
ADDI r1, #0, ASR #3   ; r1 = (r1 >> 3) = r1 / 8 (signed)
```

9.8.3 Bit Field Extraction

```
; Extract bits 8-11 from r1
LOAD r2, r1          ; r2 = r1
ADDI r2, #0, LSR #8   ; Shift down
ANDI r2, #0x0F            ; Mask to 4 bits

; Extract and sign-extend bit 7
LOAD r2, r1          ; r2 = r1
ADDI r2, #0, LSL #8   ; Shift bit 7 to bit 15
ADDI r2, #0, ASR #8   ; Arithmetic shift to sign-extend
```

9.8.4 Bit Manipulation

```
; Set bit N (where N is in r2)
LOAD r3, #1               ; r3 = 1
ADDI r3, #0, LSL r2       ; r3 = (r3 << r2) = 1 << N
ORRR r1, r3               ; r1 |= (1 << N)

; Clear bit N
LOAD r3, #1
ADDI r3, #0, LSL r2       ; r3 = (r3 << r2) = 1 << N
LOAD r4, #0xFFFF
XORR r3, r4, r3           ; r3 = 0xFFFF ^ (1 << N) = ~(1 << N)
ANDR r1, r1, r3           ; r1 = r1 & ~(1 << N)

; Test bit N
LOAD r3, #1
ADDI r3, #0, LSL r2       ; r3 = (r3 << r2) = 1 << N
TSAR r1, r3               ; Test r1 & (1 << N)
BRAN|!= @bit_was_set      ; Branch if bit set
```

9.9 Multi-Word Shifts

For operations on values larger than 16 bits, shifts and rotates can be combined:

9.9.1 32-bit Left Shift

```
; Shift 32-bit value in r1:r2 left by 1
; r1 = high word, r2 = low word
```

```
SHFT r2, LSL #1          ; Shift low, MSB -> carry
ADCI r1, #0, LSL #1      ; Shift high and add carry into bit 0
```

9.9.2 32-bit Right Shift (Unsigned)

```
; Shift 32-bit value in r1:r2 right by 1 (unsigned)
SHFT r1, LSR #1          ; Shift high, LSB -> carry
SHFT[N] r2, LSR #1       ; Shift low while preserving carry from high word
ORRI|CS r2, #0x8000      ; If old high bit 0 was set, insert it into low
    bit 15
```

9.9.3 32-bit Right Shift (Signed)

```
; Shift 32-bit value in r1:r2 right by 1 (signed)
SHFT r1, ASR #1          ; Arithmetic shift high, LSB -> carry
SHFT[N] r2, LSR #1       ; Shift low while preserving carry from high word
ORRI|CS r2, #0x8000      ; If old high bit 0 was set, insert it into low
    bit 15
```

9.10 Performance Considerations

- All shift operations complete in the same number of cycles regardless of shift count
- Shifts can be performed "for free" as part of another instruction (no extra cycles)
- Using shifts for multiplication/division by powers of 2 is much faster than using a multiply/divide instruction (if one existed)
- Barrel shifter implementation in hardware allows any shift count in constant time

9.11 Limitations

- Maximum shift count is 15 (4-bit field)
- For shifts > 15, multiple shift instructions must be used
- Rotates do not consume the carry flag; `RTL` and `RTR` rotate only within the 16-bit operand
- Some shift types (e.g., reserved type 111) may have undefined behavior

9.12 Status Flag Update Control

By default, ALU instructions with shift operations update the status register flags based on the **ALU operation result**, not the shift operation. However, the SIRC-1 provides manual control over which operation updates the flags.

9.12.1 Status Override Syntax

A status override modifier can be added immediately after an instruction mnemonic using square brackets:

Modifier	Meaning	Description
[A]	ALU	Update flags from ALU result (default)
[S]	Shift	Update flags from shift result, not ALU
[N]	None	Do not update status flags at all

Table 9.4: Status Flag Override Modifiers

9.12.2 Examples

Default Behavior (ALU flags):

```
; Without override, flags reflect the ALU addition result
ADDI r2, #1, LSL #2           ; r2 = (r2 << 2) + 1
                              ; Flags: based on addition result
```

Shift Flag Override:

```
; Update flags based on shift result instead of ALU
ADDI[S] r2, #1, LSL #2        ; r2 = (r2 << 2) + 1
                              ; Flags: based on (r2 << 2), not the +1

; Check if shifted value is zero
CMPI[S] r3, #0, LSR #4        ; Compare (r3 >> 4) with 0
                              ; Flags: based on shift, not comparison
```

No Flag Update:

```
; Perform operation without affecting status flags
ADDI[N] r1, #5, LSL #1        ; r1 = (r1 << 1) + 5
                              ; Flags: unchanged

; Useful when preserving flags from previous operation
CMPR r4, r5                   ; Set flags based on comparison
ADDI[N] r6, #1                ; Increment r6 without changing flags
BRAN|== @equal_branch         ; Branch using flags from CMPR
```

9.12.3 Use Cases

Testing Shift Results

When you need to know properties of a shifted value without performing a subsequent comparison:

```
; Check if r1 << 3 would be negative
ADDI[S] r1, #0, LSL #3        ; r1 = r1 << 3, flags from shift
BRAN|NS @overflow_detected    ; Branch if sign bit set
```

Preserving Flags

When performing operations that should not affect condition codes:

```
; Test condition
CMPR r1, r2                    ; Compare r1 and r2

; Do some work without affecting flags
ADDI[N] r3, #1                 ; Increment counter (flags unchanged)
LOAD[N] r4, (#0, a)+           ; Load next value (flags unchanged)

; Branch using original comparison flags
BRAN|>= @r1_was_greater
```

SHFT Meta-Instruction

The **SHFT** meta-instruction is the ALU-family shorthand for this common case:

```
SHFT r1, LSL #3                ; ORRI[S] r1, #0, LSL #3
```

See Chapter 13 for the full **SHFT** instruction entry.

9.12.4 Assembler Syntax Notes

- The status override modifier must appear immediately after the instruction mnemonic with no spaces: `ADDI[S]`
- The modifier is optional; omitting it defaults to [A] (ALU flag updates)
- The override applies to the entire instruction, including any conditional execution
- Explicit [A] is rarely needed but can improve code readability

Chapter 10

Condition Codes

10.1 Overview

Most SIRCIS instructions can be executed conditionally based on the state of the condition flags in the status register. This allows for efficient implementation of conditional logic without requiring explicit branch instructions.

The 4-bit condition code field in each instruction specifies under what conditions the instruction should execute. If the condition is not met, the instruction behaves as a `NOOP` (no operation).

10.2 Condition Code Encoding

Code	Mnemonic	Condition (when instruction executes)
0000	AL (or none)	Always (unconditional)
0001	==	Equal (Z = 1)
0010	!=	Not Equal (Z = 0)
0011	CS	Carry Set (C = 1)
0100	CC	Carry Clear (C = 0)
0101	NS	Negative Set (N = 1)
0110	NC	Negative Clear (N = 0)
0111	OS	Overflow Set (V = 1)
1000	OC	Overflow Clear (V = 0)
1001	HI	Unsigned Higher (C = 1 AND Z = 0)
1010	LO	Unsigned Lower or Same (C = 0 OR Z = 1)
1011	>=	Signed Greater or Equal (N = V)
1100	«	Signed Less Than (N ≠ V)
1101	»	Signed Greater Than (Z = 0 AND N = V)
1110	<=	Signed Less or Equal (Z = 1 OR N ≠ V)
1111	NV	Never (never executes)

Table 10.1: Condition Code Encoding

10.3 Condition Code Categories

10.3.1 Simple Conditions

These conditions test a single status flag:

Always (default) No condition code specified; instruction always executes.

== (Equal) Executes when Z = 1. Typically used after `CMP` to test equality.

!= (Not Equal) Executes when Z = 0. Typical use: test inequality.

CS (Carry Set) Executes when C = 1. Can indicate unsigned overflow or borrow.

CC (Carry Clear) Executes when C = 0. No unsigned overflow/borrow.

NS (Negative Set) Executes when N = 1. Tests if result is negative (signed).

NC (Negative Clear) Executes when N = 0. Tests if result is positive or zero (signed).

OS (Overflow Set) Executes when V = 1. Signed overflow occurred.

OC (Overflow Clear) Executes when V = 0. No signed overflow.

10.3.2 Unsigned Comparisons

These conditions are used after comparing unsigned integers:

HI (Unsigned Higher) Executes when C = 1 AND Z = 0. Tests if first operand > second operand (unsigned).

LO (Unsigned Lower or Same) Executes when C = 0 OR Z = 1. Tests if first ≤ second (unsigned).

Note: For unsigned comparisons:

- Use `HI` and `LO` after `CMP`
- `HI` tests if first > second (unsigned)
- `LO` tests if first ≤ second (unsigned)
- `CS` is equivalent to unsigned ≥
- `CC` is equivalent to unsigned <

10.3.3 Signed Comparisons

These conditions are used after comparing signed integers:

>= (Signed Greater or Equal) Executes when N = V. First operand ≥ second operand (signed).

« (Signed Less Than) Executes when N ≠ V. First operand < second operand (signed).

» (Signed Greater Than) Executes when Z = 0 AND N = V. First operand > second operand (signed).

<= (Signed Less or Equal) Executes when Z = 1 OR N ≠ V. First operand ≤ second operand (signed).

10.3.4 Special Conditions

Never (NV) Never executes; equivalent to `NOOP`. Can be used for instruction padding or to temporarily disable instructions during debugging.

10.4 Assembly Syntax

Condition codes are specified by appending a pipe character (`|`) followed by the condition mnemonic to the instruction:

```
; Unconditional
ADDI r1, #10

; Conditional - execute only if equal
ADDI|== r2, #20

; Conditional branch
BRAN|!= @loop_start

; Multiple conditions
ADDR|HI r3, r4, r5
STOR|NS (#0, s), r6
```

10.5 Usage with Compare Instructions

The most common use of condition codes is with compare (`CMP`) instructions:

10.5.1 Unsigned Comparison

```
; if (r1 > r2) { r3 = r4; } (unsigned)
CMPR r1, r2           ; Compare r1 and r2
LOAD|HI r3, r4        ; r3 = r4 if r1 > r2 (unsigned)

; if (r1 <= 100) { branch } (unsigned)
CMPI r1, #100
BRAN|LO @target
```

10.5.2 Signed Comparison

```
; if (r1 >= r2) { r3 = r4; } (signed)
CMPR r1, r2           ; Compare r1 and r2
LOAD|>= r3, r4        ; r3 = r4 if r1 >= r2 (signed)

; if (r1 < -10) { branch } (signed)
CMPI r1, #-10
BRAN|<< @negative_case
```

10.5.3 Equality Testing

```
; if (r1 == r2) { branch }
CMPR r1, r2
BRAN|== @equal_case

; if (r1 != 0) { branch }
CMPI r1, #0
BRAN|!= @non_zero_case
```

10.6 Condition Code Truth Tables

10.6.1 Unsigned Comparison (CMPR rA, rB)

Relationship	Z	C	Condition	Code	Meaning
rA = rB	1	*	==	0001	Equal
rA ≠ rB	0	*	!=	0010	Not Equal
rA > rB (unsigned)	0	1	HI	1001	Higher
rA < rB (unsigned)	0	0	CC	0100	Lower
rA ≥ rB (unsigned)	*	1	CS	0011	Higher or Same
rA ≤ rB (unsigned)	*	*	LO	1010	Lower or Same

Table 10.2: Unsigned Comparison Results

10.6.2 Signed Comparison (CMPR rA, rB)

Relationship	Z	N	V	Condition	Code	Meaning
rA = rB	1	*	*	==	0001	Equal
rA ≠ rB	0	*	*	!=	0010	Not Equal
rA > rB (signed)	0	0	0	»	1101	Greater
rA > rB (signed)	0	1	1	»	1101	Greater
rA < rB (signed)	*	0	1	«	1100	Less
rA < rB (signed)	*	1	0	«	1100	Less
rA ≥ rB (signed)	*	0	0	>=	1011	Greater or Equal
rA ≥ rB (signed)	*	1	1	>=	1011	Greater or Equal
rA ≤ rB (signed)	1	*	*	<=	1110	Less or Equal
rA ≤ rB (signed)	0	0	1	<=	1110	Less or Equal
rA ≤ rB (signed)	0	1	0	<=	1110	Less or Equal

Table 10.3: Signed Comparison Results

10.7 Advanced Usage

10.7.1 Conditional Assignment

```
; max = (a > b) ? a : b (unsigned)
CMPR r1, r2               ; Compare a and b
LOAD|HI r3, r1            ; r3 = a if a > b
LOAD|LO r3, r2            ; r3 = b if a <= b
```

10.7.2 Conditional Increment

```
; if (condition) count++
CMPI r1, #0
ADDI|!= r2, #1            ; Increment r2 if r1 != 0
```

10.7.3 Conditional Function Call

```
; if (r1 > 0) call_function()
CMPI r1, #0
LJSR|>> a                 ; Call function if r1 > 0 (signed)
```

10.7.4 Loop Construction

```
; for (i = 10; i > 0; i--)
ADDI r1, #0, #10          ; i = 10
loop:
    ; ... loop body ...
    SUBI r1, #1           ; i--
    BRAN|HI @loop         ; Continue if i > 0 (unsigned)
```

10.8 Condition Code Selection Guide

Intent	After CMP	Condition Code
If equal	`CMPR a, b`	`==`
If not equal	`CMPR a, b`	`!=`
If a > b (unsigned)	`CMPR a, b`	`HI`
If a ≥ b (unsigned)	`CMPR a, b`	`CS`
If a < b (unsigned)	`CMPR a, b`	`CC`
If a ≤ b (unsigned)	`CMPR a, b`	`LO`
If a > b (signed)	`CMPR a, b`	`»`
If a ≥ b (signed)	`CMPR a, b`	`>=`
If a < b (signed)	`CMPR a, b`	`«`
If a ≤ b (signed)	`CMPR a, b`	`<=`
If negative	`CMPI a, #0`	`NS`
If positive or zero	`CMPI a, #0`	`NC`
If overflow occurred	After arithmetic	`OS`
If no overflow	After arithmetic	`OC`

Table 10.4: Condition Code Selection Guide

10.9 Performance Considerations

- Conditional instructions do not branch, avoiding pipeline flushes
- When a condition is false, the instruction still takes 6 cycles but performs no operation
- For short conditional sequences, conditional execution can be faster than branching
- For longer conditional blocks, explicit branches may be more efficient

10.10 Common Pitfalls

- **Signed vs. Unsigned**: Using unsigned conditions (`HI`, `LO`) for signed values or vice versa produces incorrect results
- **Flag Clobbering**: Flags are modified by most ALU instructions. Ensure the condition flags haven't been modified between the comparison and conditional instruction:

```
; WRONG - flags clobbered
CMPR r1, r2
ADDI r3, #100          ; Modifies flags!
BRAN|HI @target        ; Tests wrong flags

; CORRECT - flags preserved
CMPR r1, r2
BRAN|HI @target
ADDI r3, #100          ; After the branch
```

- **Inverted Logic**: Remember that `CMP A, B` computes A - B, so condition codes test A relative to B, not B relative to A

Part III

SIRCIS Instruction Reference

Chapter 11

Reading Instruction Descriptions

This part of the manual describes each SIRCIS instruction in a consistent format. The goal is to make each instruction entry usable as an implementation contract, not only as programmer guidance.

11.1 Instruction Entry Fields

Mnemonic The assembly mnemonic or mnemonic family. Immediate and register variants are listed together when they perform the same operation.

Opcodes The 6-bit opcode values for each supported instruction format.

Syntax The accepted assembler forms. Meta-instructions are called out separately from real CPU instruction encodings.

Operands The permitted operand kinds and addressing modes for the instruction.

Operation Pseudocode for the architectural effect of the instruction. Arithmetic is performed on 16-bit word values unless otherwise stated.

Write-back Whether the computed result is written to a register, discarded, or used to update control state.

Status Flags The effect on the condition flags in the status register.

Condition Codes All normal instructions may be conditional. If the condition is false, the instruction has no architectural side effects unless a later instruction description explicitly states otherwise.

Exceptions Faults or traps that can be raised by the instruction. When an exception can occur, the entry should state whether partial side effects are possible.

Timing The base execution time and any circumstances that add wait states.

Privilege Whether the instruction is available in protected mode, supervisor mode, or both.

11.2 Notation

Notation	Meaning
`rD`	Destination general-purpose register
`rS`	Source general-purpose register
`rS1`	First source register, usually the left-hand operand
`rS2`	Second source register, usually the right-hand operand
`#imm16`	16-bit immediate word encoded in immediate format
`#imm8`	8-bit immediate encoded in short immediate format
`shift`	Optional SIRCIS shift definition
`SR.X`	Status register field `X`

Table 11.1: Instruction Description Notation

11.3 Status Flag Effect Symbols

Flag tables use the following symbols:

Symbol	Meaning
`*`	Updated from the instruction result
`0`	Cleared to zero
`1`	Set to one
`-`	Preserved
`S`	Updated from the shifter result when `[S]` is used

Table 11.2: Status Flag Effect Symbols

11.4 Conditional Execution

Every encoded instruction contains a condition-code field. If the condition evaluates true, the instruction executes normally. If the condition evaluates false, the instruction is skipped: destination registers, address registers, memory, the program counter, link register, coprocessor state, and status flags are not modified by that instruction.

11.5 Status Update Overrides

ALU instructions can select the source of status flag updates with the status update field:

Syntax	Encoded AF	Effect
`[N]`	`00`	Preserve status flags
`[A]`	`01`	Update flags from the ALU result
`[S]`	`10`	Update flags from the shifter result
Reserved	`11`	Reserved for future use

Table 11.3: Status Update Source Encoding

When no override is written, ALU instructions use `[A]`. Instructions that do not support status updates must not use status update override syntax.

Chapter 12

Instruction Summary

12.1 Overview

The SIRCIS instruction set consists of 64 possible opcodes (6-bit opcode field), of which approximately 52 are documented and assigned. The instructions are organized into logical groups:

- **ALU Instructions (0x00–0x0F, 0x20–0x2F, 0x30–0x3F)**: Arithmetic, logical, and comparison operations
- **Memory Instructions (0x10–0x17)**: Load and store operations
- **Address and Control Flow (0x18–0x1F)**: Effective-address calculation, jumps, and calls
- **Coprocessor (0x0F, 0x3F)**: Coprocessor interface

12.2 Complete Instruction List

Opcode	Mnemonic	Format	Flags	Operation
0x00	ADDI	Immediate	NZCV	Add immediate
0x01	ADCI	Immediate	NZCV	Add immediate with carry
0x02	SUBI	Immediate	NZCV	Subtract immediate
0x03	SBCI	Immediate	NZCV	Subtract immediate with carry
0x04	ANDI	Immediate	NZ	AND immediate
0x05	ORRI	Immediate	NZ	OR immediate
0x06	XORI	Immediate	NZ	XOR immediate
0x07	LOAD	Immediate	–	Load immediate (move)
0x08	–	–	–	Undocumented
0x09	–	–	–	Undocumented
0x0A	CMPI	Immediate	NZCV	Compare immediate
0x0B	–	–	–	Undocumented
0x0C	TSAI	Immediate	NZ	Test AND immediate
0x0D	–	–	–	Undocumented
0x0E	TSXI	Immediate	NZ	Test XOR immediate
0x0F	COPI	Immediate	–	Coprocessor call immediate
0x10	STOR	Indirect Imm	–	Store to (addr + imm)
0x11	STOR	Indirect Reg	–	Store to (addr + reg)
0x12	STOR	Pre-Dec	–	Store with pre-decrement
0x13	STOR	Pre-Dec Reg	–	Store with pre-dec (reg)
0x14	LOAD	Indirect Imm	–	Load from (addr + imm)
0x15	LOAD	Indirect Reg	–	Load from (addr + reg)
0x16	LOAD	Post-Inc	–	Load with post-increment
0x17	LOAD	Post-Inc Reg	–	Load with post-inc (reg)
0x18	LDEA	Indirect Imm	–	Load effective address
0x19	LDEA	Indirect Reg	–	Load effective address (reg)
0x1A	LDEA	Pre-Dec Imm	–	Load effective address, pre-dec
0x1B	LDEA	Pre-Dec Reg	–	Load effective address, pre-dec (reg)
0x1C	LDEL	Indirect Imm	–	Load effective address and link
0x1D	LDEL	Indirect Reg	–	Load effective address and link (reg)
0x1E	LDEL	Post-Inc Imm	–	Load effective address and link, post-inc
0x1F	LDEL	Post-Inc Reg	–	Load effective address and link, post-inc (reg)
0x20	ADDI	Short Imm+Shift	NZCV	Add short immediate
0x21	ADCI	Short Imm+Shift	NZCV	Add short imm with carry
0x22	SUBI	Short Imm+Shift	NZCV	Subtract short immediate
0x23	SBCI	Short Imm+Shift	NZCV	Subtract short imm with carry
0x24	ANDI	Short Imm+Shift	NZ	AND short immediate
0x25	ORRI	Short Imm+Shift	NZ	OR short immediate
0x26	XORI	Short Imm+Shift	NZ	XOR short immediate
0x27	LOAD	Short Imm+Shift	–	Undocumented
0x28	–	–	–	Undocumented
0x29	–	–	–	Undocumented
0x2A	CMPI	Short Imm+Shift	NZCV	Compare short immediate
0x2B	–	–	–	Undocumented
0x2C	TSAI	Short Imm+Shift	NZ	Test AND short immediate
0x2D	–	–	–	Undocumented
0x2E	TSXI	Short Imm+Shift	NZ	Test XOR short immediate
0x2F	COPI	Short Imm+Shift	–	Undocumented
0x30	ADDR	Register	NZCV	Add register
0x31	ADCR	Register	NZCV	Add register with carry
0x32	SUBR	Register	NZCV	Subtract register
0x33	SBCR	Register	NZCV	Subtract register with carry
0x34	ANDR	Register	NZ	AND register
0x35	ORRR	Register	NZ	OR register
0x36	XORR	Register	NZ	XOR register
0x37	LOAD	Register	–	Load from register (move)
0x38	–	–	–	Undocumented
0x39	–	–	–	Undocumented
0x3A	CMPR	Register	NZCV	Compare register
0x3B	–	–	–	Undocumented
0x3C	TSAR	Register	NZ	Test AND register
0x3D	–	–	–	Undocumented
0x3E	TSXR	Register	NZ	Test XOR register
0x3F	COPR	Register	–	Coprocessor call register

Table 12.1: Complete SIRCIS Instruction Set

12.3 Instruction Grouping Patterns

12.3.1 Opcode Organization

The opcode space is systematically organized:

- **0x0_**: Immediate operand format
- **0x1_**: Memory and control flow operations
- **0x2_**: Short immediate with shift format
- **0x3_**: Register operand format

12.3.2 Save vs. Test Variants

Many ALU instructions have two variants:

- **+0x00 to +0x07**: Save result to destination register
- **+0x08 to +0x0F**: Test only (update flags, discard result)

Examples:

- 0x04 = `ANDI` (save result), 0x0C = `TSAI` (test only)
- 0x06 = `XORI` (save result), 0x0E = `TSXI` (test only)

12.4 Meta-Instructions

Several commonly-used operations are implemented as assembler meta-instructions that assemble to specific opcodes:

Meta-Instruction	Assembles To	Purpose
`NOOP`	`ADDI[N] r1, #0`	No operation
`RETS`	`LDEA p, (#0, l)`	Return from subroutine
`SHFT`	`ORRI[S] rD, #0, shift`	Shift with status update
`BRAN`	`LDEA p, ...`	PC-relative branch
`BRSR`	`LDEL p, ...`	PC-relative call
`LJMP`	`LDEA p, ...`	Long jump
`LJSR`	`LDEL p, ...`	Long call
`EXCP`	`COPI #0x11vv`	User exception
`WAIT`	`COPI #0x1900`	Wait for interrupt
`RETE`	`COPI #0x1A00`	Return from exception
`RSET`	`COPI #0x1B00`	System reset
`ETFR`	`COPI #0x1Cxy`	Read exception state
`ETTR`	`COPI #0x1Dxy`	Write exception state
`DMAR`	`COPI #0x28xx`	DMA read to registers
`DMAW`	`COPI #0x29xx`	DMA write from registers
`DMAT`	`COPI #0x2Ann`	DMA memory transfer
`MULU`	`COPI #0x3000`	Unsigned multiply
`MULS`	`COPI #0x3100`	Signed multiply
`DIVU`	`COPI #0x3200`	Unsigned divide
`DIVS`	`COPI #0x3300`	Signed divide

Table 12.2: Meta-Instructions

12.5 Instruction Timing

All instructions use exactly 6 execution phases and complete in 6 clock cycles when no bus wait states are required:

1. Instruction Fetch (High Word)
2. Instruction Fetch (Low Word)
3. Decode and Register Fetch
4. Execute/Address Calculate
5. Memory Access (or NOP)
6. Write Back

This fixed sequence simplifies hardware design. Delayed bus acknowledgement may hold a phase and add wait cycles.

12.6 Next Chapters

The following chapters provide detailed documentation for each instruction group:

- **Chapter 13**: ALU instructions (arithmetic, logical, compare)
- **Chapter 14**: Load and store operations
- **Chapter 15**: Branches, jumps, and address operations
- **Chapter 16**: Coprocessor interface
- **Chapter 17**: Meta-instruction cross-reference

Chapter 13

ALU Instructions

13.1 Overview

The ALU (Arithmetic Logic Unit) instructions perform arithmetic and logical operations on register operands. All ALU instructions follow the load/store architecture principle: they operate only on registers, not directly on memory.

ALU instructions are available in three format variants:

- **Immediate**: Operation with 16-bit constant
- **Short Immediate with Shift**: Operation with 8-bit constant and optional shift
- **Register**: Operation between registers with optional shift

13.2 ALU Instruction Families

13.2.1 Arithmetic Instructions

- `ADD` – Addition
- `ADC` – Add with Carry
- `SUB` – Subtraction
- `SBC` – Subtract with Carry (Borrow)

13.2.2 Logical Instructions

- `AND` – Bitwise AND
- `ORR` – Bitwise OR
- `XOR` – Bitwise XOR (Exclusive OR)

13.2.3 Data Movement

- `LOAD` – Load immediate or copy register

13.2.4 Test and Compare

- `CMP` – Compare (subtract without saving result)
- `TSA` – Test AND (AND without saving result)
- `TSX` – Test XOR (XOR without saving result)

13.3 Legal Forms

ALU instructions are register-only operations. Immediate operands are encoded in the instruction word; memory operands are not legal for this instruction family.

Instruction	Syntax	Encoding	Notes
ADD	`ADDI rD, #imm16`	0x00	Writes result
ADD	`ADDI rD, #imm8, shift`	0x20	Writes result
ADD	`ADDR rD, rS1, rS2[, shift]`	0x30	Writes result
ADC	`ADCI rD, #imm16`	0x01	Writes result
ADC	`ADCI rD, #imm8, shift`	0x21	Writes result
ADC	`ADCR rD, rS1, rS2[, shift]`	0x31	Writes result
SUB	`SUBI rD, #imm16`	0x02	Writes result
SUB	`SUBI rD, #imm8, shift`	0x22	Writes result
SUB	`SUBR rD, rS1, rS2[, shift]`	0x32	Writes result
SBC	`SBCI rD, #imm16`	0x03	Writes result
SBC	`SBCI rD, #imm8, shift`	0x23	Writes result
SBC	`SBCR rD, rS1, rS2[, shift]`	0x33	Writes result
AND	`ANDI rD, #imm16`	0x04	Writes result
AND	`ANDI rD, #imm8, shift`	0x24	Writes result
AND	`ANDR rD, rS1, rS2[, shift]`	0x34	Writes result
ORR	`ORRI rD, #imm16`	0x05	Writes result
ORR	`ORRI rD, #imm8, shift`	0x25	Writes result
ORR	`ORRR rD, rS1, rS2[, shift]`	0x35	Writes result
XOR	`XORI rD, #imm16`	0x06	Writes result
XOR	`XORI rD, #imm8, shift`	0x26	Writes result
XOR	`XORR rD, rS1, rS2[, shift]`	0x36	Writes result
LOAD	`LOAD rD, #imm16`	0x07	Writes result; preserves flags
LOAD	`LOAD rD, rS`	0x37	Writes result; preserves flags
CMP	`CMPI rS, #imm16`	0x0A	Updates flags only
CMP	`CMPI rS, #imm8, shift`	0x2A	Updates flags only
CMP	`CMPR rS1, rS2[, shift]`	0x3A	Updates flags only
TSA	`TSAI rS, #imm16`	0x0C	Updates flags only
TSA	`TSAI rS, #imm8, shift`	0x2C	Updates flags only
TSA	`TSAR rS1, rS2[, shift]`	0x3C	Updates flags only
TSX	`TSXI rS, #imm16`	0x0E	Updates flags only
TSX	`TSXI rS, #imm8, shift`	0x2E	Updates flags only
TSX	`TSXR rS1, rS2[, shift]`	0x3E	Updates flags only

Table 13.1: Legal ALU Instruction Forms

Register shorthand: When a register-form ALU instruction omits `rS1`, the assembler encodes `rD` as both the destination and first source operand. For example, `ADDR r1, r2` is encoded as `ADDR r1, r1, r2`.

Short-immediate LOAD: Opcode `0x27` has no public assembly syntax and is undocumented.

13.4 Common ALU Semantics

Topic	Definition
Condition false	The instruction is skipped and has no side effects. Registers and status flags are preserved.
Immediate format	The left operand is the named register. The right operand is the encoded 16-bit immediate.
Short immediate format	The left operand is the named register after the encoded shift is applied. The right operand is the encoded 8-bit immediate.
Register format	The left operand is `rS1` after the encoded shift is applied. The right operand is `rS2`.
Test variants	`CMP`, `TSA`, and `TSX` compute a result for flag updates but do not write it back.
LOAD	`LOAD` writes the selected operand to `rD` and does not update status flags.
Timing	ALU instructions complete in 6 cycles, plus any instruction-fetch wait states.
Exceptions	ALU execution raises no instruction-specific faults; see the notes below.

Table 13.2: Common ALU Semantics

ALU instruction execution does not perform data-memory access and does not raise data bus, data bus-protection, data alignment, segment-overflow, privilege-violation, or invalid-opcode faults. Instruction fetch can still raise the normal fetch faults before an ALU instruction executes. If trace mode was enabled when the instruction began, an instruction-trace fault is raised after the instruction commits. Processing-unit encodings outside the documented ALU forms are reserved or architecturally undefined; they are not required to raise an invalid-opcode fault.

13.5 Status Flag Updates

Most ALU instructions update the status register flags:

Arithmetic (ADD, ADC, SUB, SBC, CMP) Update all four flags: N, Z, C, V

Logical (AND, ORR, XOR, TSA, TSX) Update N and Z; clear C and V

Load (LOAD) Do not update flags

Family	N	Z	C	V	Default AF	Meaning
ADD	*	*	*	*	[A]	C is carry out; V is signed overflow.
ADC	*	*	*	*	[A]	Adds incoming C; C is carry out.
SUB	*	*	*	*	[A]	C is set when a borrow occurs.
SBC	*	*	*	*	[A]	Subtracts incoming C as borrow input; C is set on outgoing borrow.
CMP	*	*	*	*	[A]	Same flags as SUB; result is discarded.
AND	*	*	0	0	[A]	C and V are cleared.
ORR	*	*	0	0	[A]	C and V are cleared.
XOR	*	*	0	0	[A]	C and V are cleared.
TSA	*	*	0	0	[A]	Same flags as AND; result is discarded.
TSX	*	*	0	0	[A]	Same flags as XOR; result is discarded.
LOAD	-	-	-	-	[N]	Status flags are preserved.

Table 13.3: ALU Status Flag Effects

See Table 11.2 for the flag-effect symbol definitions. The table shows the effect when the instruction uses its default status update source. If `[N]` is encoded, all status flags are preserved. If `[A]` is encoded, flags are updated from the ALU result according to the instruction family. If `[S]` is encoded, flags are updated from the shifter result instead of the ALU result; see Chapter 9 for the shifter flag rules. Public **`LOAD`** forms do not support status update override syntax and always preserve the status flags.

13.5.1 Manual Flag Update Control

ALU instructions except **`LOAD`** support manual control over how status flags are updated using the status override syntax. See Section 9.12 in Chapter 9 for complete details.

- `INSTR[A]` – Update flags from ALU result (default behavior)
- `INSTR[S]` – Update flags from shift result instead of ALU
- `INSTR[N]` – Do not update status flags

Example:

```
; Update flags from shift operation
ADDI[S] r1, #0, LSL #3          ; Flags reflect (r1 << 3), not the ALU

; Preserve flags during operation
CMPR r2, r3                     ; Set comparison flags
```

```
ADDI[N] r4, #1                  ; Increment without changing flags
BRAN|>= @branch_if_r2_gte_r3    ; Use flags from CMPR
```

SHFT – Shift with Status Update Meta-Instruction

Assembles to: `ORRI[S] rD, #0, shift`
Syntax:

```
SHFT rD, shift                  ; rD = shifted rD; flags from shift
```

Operands: `rD` is both the destination and shifted source register. `shift` may use any standard immediate or register shift count supported by short-immediate ALU instructions.
Operation:

```
shifted = shift(rD)
rD = shifted OR 0
SR = flags_from_shift_result
```

Description:
Provides concise syntax for shifting a register value while taking status flags from the shifter result. It is not a separate CPU operation. The zero OR operation preserves the shifted source value, while `[S]` makes the status flags reflect the shift result instead of the OR result.
Flags: Updated from the shifter result by default.
Write-back: Inherits `ORRI` write-back to `rD`.
Exceptions: Does not access memory and raises no instruction-specific faults.
Condition codes: If the condition is false, no shift result is written and status flags are preserved.
Timing: Same as `ORRI`: 6 cycles, plus instruction-fetch wait states.
Privilege: Available in protected mode and supervisor mode.
Examples:

```
; Shift r1 left by 3 and update flags
SHFT r1, LSL #3                 ; ORRI[S] r1, #0, LSL #3

; Shift right and test result
SHFT r3, LSR #2                 ; r3 = r3 >> 2
BRAN|== @zero_result            ; Branch if shifted result is zero
```

Notes:

- Because the lowering uses the short-immediate format, `SHFT` has the same shift field limits as other short-immediate ALU instructions.
- All shift types are supported: LSL, LSR, ASL, ASR, RTL, and RTR.

13.6 ADD – Addition

ADDI / ADDR – Add Immediate / Add Register

Opcodes: 0x00 (Imm), 0x20 (Short Imm), 0x30 (Reg)
Syntax:

```
ADDI rD, #imm16                 ; rD = rD + imm16
ADDI rD, #imm8, shift           ; rD = (rD << shift) + imm8
ADDR rD, rS1, rS2               ; rD = rS1 + rS2
ADDR rD, rS1, rS2, shift        ; rD = (rS1 << shift) + rS2
```

Operands: `rD` is the destination and first source in immediate forms. Register forms use `rS1` as the first source and `rS2` as the second source; assembler shorthand may omit `rS1`. Short-immediate and register forms may apply a shift to the first source operand.
Operation:

```
result = operand1 + operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = carry_out
SR.V = signed_overflow
```

Description:
Adds two values and stores the result in the destination register. In short immediate format with shift, the destination register is shifted before adding the immediate. The carry flag is set if an unsigned overflow occurs (carry out of bit 15). The overflow flag is set if a signed overflow occurs (the sign of the result is incorrect for the operation).
Flags: N Z C V
Write-back: If the condition is true, writes the 16-bit result to `rD`.
Exceptions: Does not access memory and raises no instruction-specific faults.
Condition codes: If the condition is false, no result is written and status flags are preserved.
Timing: 6 cycles, plus instruction-fetch wait states.
Privilege: Available in protected mode and supervisor mode.
Example:

```
; Simple addition
ADDI r1, #100               ; r1 = r1 + 100

; Add two registers
ADDR r3, r1, r2             ; r3 = r1 + r2

; Scaled addition (multiply by 3)
ADDR r4, r5, r5, LSL #1     ; r4 = r5 + (r5 * 2) = r5 * 3

; Build large constant
ADDI r1, #1, LSL #12        ; r1 = (r1 << 12) + 1
```

Notes:

- For multi-precision arithmetic, use `ADC` to propagate carry
- Adding zero (`ADDI r1, #0`) can be used to move values or test flags
- Shift operations allow efficient constant multiplication

13.7 ADC – Add with Carry

ADCI / ADCR – Add with Carry

Opcodes: 0x01 (Imm), 0x21 (Short Imm), 0x31 (Reg)
Syntax:

```
ADCI rD, #imm16                 ; rD = rD + imm16 + C
ADCI rD, #imm8, shift           ; rD = (rD << shift) + imm8 + C
ADCR rD, rS1, rS2               ; rD = rS1 + rS2 + C
ADCR rD, rS1, rS2, shift        ; rD = (rS1 << shift) + rS2 + C
```

Operands: Same operand forms as `ADD`. The incoming `C` flag is read as an additional least-significant carry input.
Operation:

```
result = operand1 + operand2 + SR.C
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = carry_out
SR.V = signed_overflow
```

Description:
Adds two values plus the carry flag and stores the result. In short immediate format with shift, the destination register is shifted before adding the immediate. This instruction is used for multi-precision (32-bit, 48-bit, etc.) arithmetic to propagate the carry from lower-order additions to higher-order additions.
Flags: N Z C V
Write-back: If the condition is true, writes the 16-bit result to `rD`.
Exceptions: Does not access memory and raises no instruction-specific faults.
Condition codes: If the condition is false, no result is written and status flags are preserved.
Timing: 6 cycles, plus instruction-fetch wait states.
Privilege: Available in protected mode and supervisor mode.
Example:

```
; 32-bit addition (r1:r2 = r3:r4 + r5:r6)
; Low words
ADDR r2, r4, r6             ; r2 = r4 + r6, set carry

; High words (with carry propagation)
ADCR r1, r3, r5             ; r1 = r3 + r5 + carry
```

Notes:

- Always use after `ADD` for multi-word arithmetic
- The carry flag must be in the correct state before `ADC`
- Can be used to add 1 conditionally based on carry flag

13.8 SUB – Subtraction

SUBI / SUBR – Subtract Immediate / Subtract Register

Opcodes: 0x02 (Imm), 0x22 (Short Imm), 0x32 (Reg)
Syntax:

```
SUBI rD, #imm16                 ; rD = rD - imm16
SUBI rD, #imm8, shift           ; rD = (rD << shift) - imm8
SUBR rD, rS1, rS2               ; rD = rS1 - rS2
SUBR rD, rS1, rS2, shift        ; rD = (rS1 << shift) - rS2
```

Operands: Same operand forms as `ADD`. The second operand is subtracted from the first operand.
Operation:

```
result = operand1 - operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = borrow
SR.V = signed_overflow
```

Description:
Subtracts the second operand from the first and stores the result. In short immediate format with shift, the destination register is shifted before subtracting the immediate. The carry flag is set if a borrow occurs (operand2 > operand1 for unsigned). The overflow flag indicates signed overflow.
Flags: N Z C V
Write-back: If the condition is true, writes the 16-bit result to `rD`.
Exceptions: Does not access memory and raises no instruction-specific faults.
Condition codes: If the condition is false, no result is written and status flags are preserved.
Timing: 6 cycles, plus instruction-fetch wait states.
Privilege: Available in protected mode and supervisor mode.
Example:

```
; Decrement by 1
SUBI r1, #1                 ; r1 = r1 - 1

; Subtract registers
SUBR r3, r1, r2             ; r3 = r1 - r2

; Loop counter
SUBI r7, #1                 ; Decrement counter
BRAN|!= @loop               ; Branch if not zero
```

Notes:

- Carry is set when the subtraction requires a borrow
- For multi-precision subtraction, use `SBC`
- Subtracting a value from itself (`SUBR r1, r1, r1`) clears the register

13.9 SBC – Subtract with Carry (Borrow)

SBCI / SBCR – Subtract with Carry

Opcodes: 0x03 (Imm), 0x23 (Short Imm), 0x33 (Reg)
Syntax:

```
SBCI rD, #imm16                 ; rD = rD - imm16 - C
SBCI rD, #imm8, shift           ; rD = (rD << shift) - imm8 - C
SBCR rD, rS1, rS2               ; rD = rS1 - rS2 - C
SBCR rD, rS1, rS2, shift        ; rD = (rS1 << shift) - rS2 - C
```

Operands: Same operand forms as `SUB`. The incoming `C` flag is read as the borrow input.
Operation:

```
result = operand1 - operand2 - SR.C
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = borrow
SR.V = signed_overflow
```

Description:
Subtracts the second operand and borrow from the first operand. In short immediate format with shift, the destination register is shifted before subtracting the immediate. Used for multi-precision subtraction to propagate borrows from lower-order to higher-order subtractions.
Flags: N Z C V
Write-back: If the condition is true, writes the 16-bit result to `rD`.
Exceptions: Does not access memory and raises no instruction-specific faults.
Condition codes: If the condition is false, no result is written and status flags are preserved.
Timing: 6 cycles, plus instruction-fetch wait states.
Privilege: Available in protected mode and supervisor mode.
Example:

```
; 32-bit subtraction (r1:r2 = r3:r4 - r5:r6)
; Low words
SUBR r2, r4, r6             ; r2 = r4 - r6, set/clear carry

; High words (with borrow propagation)
SBCR r1, r3, r5             ; r1 = r3 - r5 - borrow
```

Notes:

- Always use after `SUB` for multi-word subtraction
- Carry is treated as the borrow input for multi-word subtraction

13.10 AND – Bitwise AND

ANDI / ANDR – AND Immediate / AND Register

Opcodes: 0x04 (Imm), 0x24 (Short Imm), 0x34 (Reg)
Syntax:

```
ANDI rD, #imm16                 ; rD = rD & imm16
ANDI rD, #imm8, shift           ; rD = (rD << shift) & imm8
ANDR rD, rS1, rS2               ; rD = rS1 & rS2
ANDR rD, rS1, rS2, shift        ; rD = (rS1 << shift) & rS2
```

Operands: Same operand forms as `ADD`. Logical operations interpret operands as 16-bit bit patterns; short immediates are zero-extended.
Operation:

```
result = operand1 AND operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Description:
Performs bitwise AND operation. Each bit in the result is 1 only if the corresponding bits in both operands are 1. In short immediate format with shift, the destination register is shifted before ANDing with the immediate. Commonly used for bit masking and clearing specific bits.
Flags: N Z (C and V cleared)
Write-back: If the condition is true, writes the 16-bit result to `rD`.
Exceptions: Does not access memory and raises no instruction-specific faults.
Condition codes: If the condition is false, no result is written and status flags are preserved.
Timing: 6 cycles, plus instruction-fetch wait states.
Privilege: Available in protected mode and supervisor mode.
Example:

```
; Mask lower byte
ANDI r1, #0x00FF            ; r1 = r1 & 0xFF

; Clear bit 5
ANDI r2, #~(1<<5)           ; r2 = r2 & 0xFFDF

; Check if two registers share any set bits
ANDR r3, r1, r2             ; r3 = r1 & r2
CMPI r3, #0
BRAN|!= @common_bits        ; Branch if any bits in common
```

Notes:

- ANDing with 0xFFFF leaves value unchanged
- ANDing with 0x0000 clears the register
- Use `TSA` to test bits without modifying the register

13.11 ORR – Bitwise OR

ORRI / ORRR – OR Immediate / OR Register

Opcodes: 0x05 (Imm), 0x25 (Short Imm), 0x35 (Reg)
Syntax:

```
ORRI rD, #imm16                 ; rD = rD | imm16
ORRI rD, #imm8, shift           ; rD = (rD << shift) | imm8
ORRR rD, rS1, rS2               ; rD = rS1 | rS2
ORRR rD, rS1, rS2, shift        ; rD = (rS1 << shift) | rS2
```

Operands: Same operand forms as **AND**. Logical operations interpret operands as 16-bit bit patterns; short immediates are zero-extended.
Operation:

```
result = operand1 OR operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Description:
Performs bitwise OR operation. Each bit in the result is 1 if either corresponding bit in the operands is 1. In short immediate format with shift, the destination register is shifted before ORing with the immediate. Commonly used for setting specific bits.
Flags: N Z (C and V cleared)
Write-back: If the condition is true, writes the 16-bit result to **rD**.
Exceptions: Does not access memory and raises no instruction-specific faults.
Condition codes: If the condition is false, no result is written and status flags are preserved.
Timing: 6 cycles, plus instruction-fetch wait states.
Privilege: Available in protected mode and supervisor mode.
Example:

```
; Set bit 7
ORRI r1, #0x0080            ; r1 = r1 | 0x80

; Set multiple bits
ORRI r2, #0xF000            ; Set upper nibble

; Combine two registers
ORRR r3, r1, r2             ; r3 = r1 | r2

; Set bit N (variable)
ADDI r4, #1
SHFT r4, LSL r5             ; r4 = 1 << r5
ORRR r1, r4                 ; r1 |= (1 << r5)
```

Notes:

- ORing with 0x0000 leaves value unchanged
- ~~ORing with 0xFFFF sets all bits~~
- Use for enabling flag bits or combining bit fields

13.12 XOR – Bitwise Exclusive OR

XORI / XORR – XOR Immediate / XOR Register

Opcodes: 0x06 (Imm), 0x26 (Short Imm), 0x36 (Reg)
Syntax:

```
XORI rD, #imm16                 ; rD = rD ^ imm16
XORI rD, #imm8, shift           ; rD = (rD << shift) ^ imm8
XORR rD, rS1, rS2               ; rD = rS1 ^ rS2
XORR rD, rS1, rS2, shift        ; rD = (rS1 << shift) ^ rS2
```

Operands: Same operand forms as **AND**. Logical operations interpret operands as 16-bit bit patterns; short immediates are zero-extended.
Operation:

```
result = operand1 XOR operand2
destination = result
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Description:
Performs bitwise XOR (exclusive OR) operation. Each bit in the result is 1 if the corresponding bits in the operands are different. In short immediate format with shift, the destination register is shifted before XORing with the immediate. Used for toggling bits, bitwise comparison, and simple encryption.
Flags: N Z (C and V cleared)
Write-back: If the condition is true, writes the 16-bit result to **rD**.
Exceptions: Does not access memory and raises no instruction-specific faults.
Condition codes: If the condition is false, no result is written and status flags are preserved.
Timing: 6 cycles, plus instruction-fetch wait states.
Privilege: Available in protected mode and supervisor mode.
Example:

```
; Toggle bit 3
XORI r1, #0x0008            ; r1 = r1 ^ 0x08

; Invert all bits
XORI r2, #0xFFFF            ; r2 = ~r2

; Check if registers are equal
XORR r3, r1, r2             ; r3 = r1 ^ r2
; If r3 == 0, then r1 == r2

; Swap using XOR (no temporary)
XORR r1, r1, r2             ; r1 = r1 ^ r2
XORR r2, r1, r2             ; r2 = r1 ^ r2 = (r1 ^ r2) ^ r2 = r1
XORR r1, r1, r2             ; r1 = (r1 ^ r2) ^ r1 = r2
```

Notes:

- XORing with 0x0000 leaves value unchanged
- XORing with 0xFFFF inverts all bits (bitwise NOT)
- XORing a value with itself always gives 0

13.13 LOAD – Load Immediate / Move Register

LOAD – Load/Move

Opcodes: 0x07 (Imm), 0x37 (Reg). Opcode 0x27 is undocumented.
Syntax:

```
LOAD rD, #imm16                 ; rD = imm16 (via add)
LOAD rD, rS                     ; rD = rS (via add)
```

Operands: Immediate form writes a 16-bit immediate value to `rD`. Register form copies `rS` to `rD`. No short-immediate, memory, or shift forms are public assembly syntax.
Operation:

```
; Implemented as: rD = 0 + operand
destination = operand
; Flags NOT updated
```

Description:
Loads an immediate value into a register or copies a register value. This is actually implemented as an ADD with an implicit zero operand, but flags are not updated. This is the primary way to load constants or move data between registers. Opcode `0x27` has no public assembly syntax and is undocumented.
Flags: Preserved.
Write-back: If the condition is true, writes the loaded or copied value to `rD`.
Exceptions: Does not access memory in these forms and raises no instruction-specific faults.
Condition codes: If the condition is false, no result is written and status flags are preserved.
Timing: 6 cycles, plus instruction-fetch wait states.
Privilege: Available in protected mode and supervisor mode.
Example:

```
; Load constant
LOAD r1, #1234              ; r1 = 1234

; Copy register
LOAD r2, r1                 ; r2 = r1

; Load with arithmetic
LOAD r3, #1
ADDI r3, #0, LSL #8         ; r3 = (r3 << 8) + 0 = 256

; Zero a register
LOAD r4, #0                 ; r4 = 0

; Load large constant (combine)
LOAD r5, #0x12
ADDI r5, #0, LSL #8         ; r5 = (r5 << 8) + 0 = 0x1200
ORRI r5, #0x34              ; r5 = 0x1234
```

Notes:

- Does not update flags, unlike **`ADD`**
- Most efficient way to move data between registers
- For 16-bit constants, use immediate form
- To build shifted or composite constants, combine **`LOAD`** with ALU instructions such as **`ADDI`** or **`ORRI`**

13.14 CMP – Compare

CMPI / CMPR – Compare Immediate / Compare Register

Opcodes: 0x0A (Imm), 0x2A (Short Imm), 0x3A (Reg)
Syntax:

```
CMPI rS, #imm16                 ; Compare rS with imm16
CMPI rS, #imm8, shift           ; Compare (rS << shift) with imm8
CMPR rS1, rS2                   ; Compare rS1 with rS2
CMPR rS1, rS2, shift            ; Compare (rS1 << shift) with rS2
```

Operands: Same source operand forms as **SUB**. The first operand is compared with the second operand; no destination register is written.
Operation:

```
result = operand1 - operand2
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = borrow
SR.V = signed_overflow
```

Description:
Performs subtraction but discards the result, only updating the status flags. In short immediate format with shift, the source register is shifted before comparing with the immediate. This is the primary instruction for implementing conditional execution and branches. The flags can be tested with condition codes to determine the relationship between the operands.
Flags: N Z C V
Write-back: None. The subtraction result is discarded after status flags are updated.
Exceptions: Does not access memory and raises no instruction-specific faults.
Condition codes: If the condition is false, no status flags are updated.
Timing: 6 cycles, plus instruction-fetch wait states.
Privilege: Available in protected mode and supervisor mode.
Example:

```
; Test if r1 is zero
CMPI r1, #0
BRAN|== @is_zero            ; Branch if r1 == 0

; Test if r1 > r2 (unsigned)
CMPR r1, r2
BRAN|HI @r1_greater

; Test if r1 >= r2 (signed)
CMPR r1, r2
BRAN|>= @r1_greater_or_equal

; Range check: if (r1 >= 10 && r1 < 20)
CMPI r1, #10
BRAN|<< @out_of_range       ; Branch if r1 < 10
CMPI r1, #20
BRAN|>= @out_of_range       ; Branch if r1 >= 20
; r1 is in range [10, 20)
```

Notes:

- Does not modify any register, only flags

13.15 TSA – Test AND

TSAI / TSAR – Test AND Immediate / Test AND Register

Opcodes: 0x0C (Imm), 0x2C (Short Imm), 0x3C (Reg)
Syntax:

```
TSAI rS, #imm16                 ; Test rS & imm16
TSAI rS, #imm8, shift           ; Test (rS << shift) & imm8
TSAR rS1, rS2                   ; Test rS1 & rS2
TSAR rS1, rS2, shift            ; Test (rS1 << shift) & rS2
```

Operands: Same source operand forms as **AND**. The result is used only for status flag updates; no destination register is written.
Operation:

```
result = operand1 AND operand2
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Description:
Performs bitwise AND but discards the result, only updating flags. In short immediate format with shift, the source register is shifted before ANDing with the immediate. Used to test if specific bits are set without modifying the register. The zero flag indicates whether any tested bits were set.
Flags: N Z (C and V cleared)
Write-back: None. The AND result is discarded after status flags are updated.
Exceptions: Does not access memory and raises no instruction-specific faults.
Condition codes: If the condition is false, no status flags are updated.
Timing: 6 cycles, plus instruction-fetch wait states.
Privilege: Available in protected mode and supervisor mode.
Example:

```
; Test if bit 5 is set
TSAI r1, #0x0020            ; Test r1 & 0x20
BRAN|!= @bit5_set           ; Branch if bit 5 was set

; Test if any upper byte bits are set
TSAI r2, #0xFF00
BRAN|!= @has_upper_bits

; Test if r1 and r2 have any bits in common
TSAR r1, r2
BRAN|== @no_common_bits     ; Branch if no bits in common

; Test multiple bits
TSAI r3, #0xF000            ; Test upper nibble
BRAN|== @upper_clear        ; Branch if all tested bits clear
```

Notes:

- Does not modify any register, only flags
- Z=1 means all tested bits were 0
- Z=0 means at least one tested bit was 1

13.16 TSX – Test XOR

TSXI / TSXR – Test XOR Immediate / Test XOR Register

Opcodes: 0x0E (Imm), 0x2E (Short Imm), 0x3E (Reg)
Syntax:

```
TSXI rS, #imm16                 ; Test rS ^ imm16
TSXI rS, #imm8, shift           ; Test (rS << shift) ^ imm8
TSXR rS1, rS2                   ; Test rS1 ^ rS2
TSXR rS1, rS2, shift            ; Test (rS1 << shift) ^ rS2
```

Operands: Same source operand forms as **XOR**. The result is used only for status flag updates; no destination register is written.
Operation:

```
result = operand1 XOR operand2
; Result is discarded, only flags updated
SR.N = result[15]
SR.Z = (result == 0)
SR.C = 0
SR.V = 0
```

Description:
Performs bitwise XOR but discards the result, only updating flags. In short immediate format with shift, the source register is shifted before XORing with the immediate. Used to test if bits match a specific pattern or to compare values for equality. Z=1 indicates the values are identical.
Flags: N Z (C and V cleared)
Write-back: None. The XOR result is discarded after status flags are updated.
Exceptions: Does not access memory and raises no instruction-specific faults.
Condition codes: If the condition is false, no status flags are updated.
Timing: 6 cycles, plus instruction-fetch wait states.
Privilege: Available in protected mode and supervisor mode.
Example:

```
; Test if r1 equals specific value
TSXI r1, #0x1234            ; Test r1 ^ 0x1234
BRAN|== @is_0x1234          ; Branch if r1 == 0x1234

; Test if two registers are equal
TSXR r1, r2                 ; Test r1 ^ r2
BRAN|== @registers_equal    ; Branch if r1 == r2

; Test if bit pattern matches
TSAI r3, #0xFF              ; Mask to lower byte
TSXI r3, #0xAA              ; Test if lower byte == 0xAA
BRAN|== @pattern_match
```

Notes:

- Does not modify any register, only flags
- Z=1 means values are identical (all bits match)
- Can be used as equality test (alternative to CMP for == only)
- More efficient than XOR + CMP for simple equality tests

13.17 Summary

The ALU instructions form the computational core of the SIRCIS instruction set. Key points:

- All ALU operations work only on registers (load/store architecture)
- Three format variants provide flexibility (immediate, short+shift, register)
- Test variants (CMP, TSA, TSX) allow non-destructive testing
- Shift operations can be combined with ALU ops for powerful single-cycle operations
- Consistent flag behavior makes conditional execution predictable

Chapter 14

Memory Access Instructions

14.1 Overview

Memory access instructions transfer data between registers and memory. Following the load/store architecture, these are the only instructions that can access memory—all arithmetic and logical operations must work on registers.

The SIRC-1 provides two primary memory operations:

- `LOAD` – Read from memory into a register
- `STOR` – Write from a register to memory

Both instructions support four memory addressing forms:

- Indirect with immediate offset: `(#offset, addr)`
- Indirect with register offset: `(reg, addr)`
- Post-increment load: `(#offset, addr)+` or `(reg, addr)+`
- Pre-decrement store: `-(#offset, addr)` or `-(reg, addr)`

Important: Register-offset memory forms support optional shift operations on the data being loaded or stored. Immediate-offset memory forms do not encode a shift field. See Section 14.8 for details. Shifts are also **not** supported when loading immediate values or copying register values directly.

14.2 Effective Address Calculation

For all memory operations, the effective address (EA) is calculated as:

$$\text{EA} = \text{AddressRegister} + \text{Offset} \tag{14.1}$$

Where:

- Address Register is one of: `l`, `a`, `s`, or `p`
- Offset is either an immediate value or register contents

The resulting 24-bit address is used to access memory.

14.3 Legal Forms

Instruction	Syntax	Encoding	Notes
LOAD	`LOAD rD, (#offset, addr)`	0x14	Immediate offset
LOAD	`LOAD rD, (r0, addr)[, shift]`	0x15	Register offset; optional load-result shift
LOAD	`LOAD rD, (#offset, addr)+`	0x16	Immediate offset; post-increment
LOAD	`LOAD rD, (r0, addr)+[, shift]`	0x17	Register offset; optional shift; post-increment
STOR	`STOR (#offset, addr), rS`	0x10	Immediate offset
STOR	`STOR (r0, addr), rS[, shift]`	0x11	Register offset; optional source shift
STOR	`STOR -(#offset, addr), rS`	0x12	Immediate offset; pre-decrement
STOR	`STOR -(r0, addr), rS[, shift]`	0x13	Register offset; optional shift; pre-decrement

Table 14.1: Legal Memory Instruction Forms

Zero displacement: Immediate-displacement memory forms may omit `#0`. Thus `LOAD rD, (addr)`, `LOAD rD, (addr)+`, `STOR (addr), rS`, and `STOR -(addr), rS` assemble as the matching `#0` forms. This shorthand does not add new addressing modes: `LOAD` has no pre-decrement form and `STOR` has no post-increment form.

14.4 Common Memory Semantics

Topic	Definition
Condition false	The instruction is skipped and has no side effects. Registers, memory, address registers, and status flags are preserved.
Address calculation	Immediate-offset forms add the 16-bit encoded offset to the selected address register low word. Register-offset forms add `r0` to the selected address register low word. The selected address register high word supplies the segment.
Post-increment LOAD	The memory address is calculated from the original address register value. The address register low word is incremented by one word in write-back after the memory access.
Pre-decrement STOR	The memory address uses the selected address register low word decremented by one word, plus the offset. The address register low word is decremented by one word in write-back after the memory access.
Status flags	Memory instructions preserve all status flags. Shifts used by memory instructions cannot update the status register.
Timing	Memory instructions complete in 6 cycles when the bus access is acknowledged without wait states. Bus wait states extend the memory-access phase.
Exceptions	Memory instructions can raise data bus, data bus-protection, and segment-overflow faults; see the notes below.
Fault side effects	Destination register writes and address-register auto-updates occur in write-back after the memory-access phase. If a fault aborts execution before write-back, those write-back effects do not occur.

Table 14.2: Common Memory Instruction Semantics

Memory instructions can raise data bus and data bus-protection faults during the memory-access phase. Effective-address calculation can raise a segment-overflow fault when `SR.A` (Trap on Address Overflow) is set and the 16-bit low-word address calculation wraps. Documented memory forms do not raise privilege-violation or invalid-opcode faults. Instruction fetch can still raise the normal fetch faults before a memory instruction executes. If trace mode was enabled when the instruction began, an instruction-trace fault is raised after the instruction commits. Processing-unit encodings outside the documented memory forms are reserved or architecturally undefined; they are not required to raise an invalid-opcode fault.

14.5 LOAD Instructions

LOAD – Load from Memory

Opcodes: 0x14 (Indirect Imm), 0x15 (Indirect Reg), 0x16 (Post-Inc Imm), 0x17 (Post-Inc Reg)
Syntax:

```
LOAD rD, (#offset, addr)       ; rD = memory[addr + offset]
LOAD rD, (addr)                ; equivalent to LOAD rD, (#0, addr)
LOAD rD, (rS, addr)            ; rD = memory[addr + rS]
LOAD rD, (#offset, addr)+      ; rD = memory[addr + offset]; addr += 1
LOAD rD, (addr)+               ; equivalent to LOAD rD, (#0, addr)+
LOAD rD, (rS, addr)+           ; rD = memory[addr + rS]; addr += 1

; With optional shift (register-offset memory forms only)
LOAD rD, (rS, addr), shift            ; Load and shift
LOAD rD, (rS, addr)+, shift           ; Load, shift, then increment
```

Operands: `rD` is the destination general-purpose register. `addr` is the address-register pair that supplies the segment and base low word. Immediate-offset forms use a 16-bit displacement. Register-offset forms use `rS` as the displacement register and may apply a shift to the loaded memory value before write-back.
Operation:

```
if post_increment then
    effective_address = addr + offset
    loaded_value = memory[effective_address]
    destination = shift_if_encoded(loaded_value)
    addr.low = addr.low + 1
else
    effective_address = addr + offset
    loaded_value = memory[effective_address]
    destination = shift_if_encoded(loaded_value)
endif
; Flags are preserved
```

Description:
Loads a 16-bit value from memory into the destination register. Post-increment forms update the address register after the load, useful for array traversal.
Write-back: If the condition is true, writes the loaded value to `rD`. Post-increment forms also write the incremented address-register low word.
Flags: Preserved.
Exceptions: May raise data bus or data bus-protection faults during the memory read. May raise a segment-overflow fault during effective-address calculation when `SR.A` is set.
Timing: 6 cycles when the memory read is acknowledged without wait states; wait states extend the memory-access phase.
Condition codes: If the condition is false, no memory read occurs, no destination register is written, no address register is incremented, and status flags are preserved.
Privilege: Available in protected mode and supervisor mode. Protected-mode data-memory access is subject to bus protection.
Examples:

```
; Load from fixed offset
LOAD r1, (#4, a)                ; r1 = memory[a + 4]
```

14.6 STOR Instructions

STOR – Store to Memory

Opcodes: 0x10 (Indirect Imm), 0x11 (Indirect Reg), 0x12 (Pre-Dec Imm), 0x13 (Pre-Dec Reg)
Syntax:

```
STOR (#offset, addr), rS        ; memory[addr + offset] = rS
STOR (addr), rS                 ; equivalent to STOR (#0, addr), rS
STOR (rD, addr), rS             ; memory[addr + rD] = rS
STOR -(#offset, addr), rS       ; addr -= 1; memory[addr + offset] =
    rS
STOR -(addr), rS                ; equivalent to STOR -(#0, addr), rS
STOR -(rD, addr), rS            ; addr -= 1; memory[addr + rD] = rS

; With optional shift (register-offset memory forms only)
STOR (rD, addr), rS, shift              ; Shift and store
STOR -(rD, addr), rS, shift             ; Decrement, shift, and store
```

Operands: `rS` is the source general-purpose register. `addr` is the address-register pair that supplies the segment and base low word. Immediate-offset forms use a 16-bit displacement. Register-offset forms use `rD` as the displacement register and may apply a shift to the source value before storing it.
Operation:

```
if pre_decrement then
    decremented_low = addr.low - 1
    effective_address = (addr.high : decremented_low) + offset
    memory[effective_address] = shift_if_encoded(source)
    addr.low = decremented_low
else
    effective_address = addr + offset
    memory[effective_address] = shift_if_encoded(source)
endif
; Source register and flags are preserved
```

Description:
Stores a 16-bit value from a register into memory. Pre-decrement forms use the decremented address for the store and write the decremented address register value back after the memory access, useful for stack push operations.
Write-back: If the condition is true, writes the source value to memory. Pre-decrement forms also write the decremented address-register low word.
Flags: Preserved.
Exceptions: May raise data bus or data bus-protection faults during the memory write. May raise a segment-overflow fault during effective-address calculation when `SR.A` is set.
Timing: 6 cycles when the memory write is acknowledged without wait states; wait states extend the memory-access phase.
Condition codes: If the condition is false, no memory write occurs, no address register is decremented, and status flags are preserved.
Privilege: Available in protected mode and supervisor mode. Protected-mode data-memory access is subject to bus protection.
Examples:

```
; Store to fixed offset
STOR (#8, a), r1                ; memory[a + 8] = r1
```

14.7 Common Memory Access Patterns

14.7.1 Structure Field Access

```
; struct { int16_t x, y, z; } point;
; 'a' points to point
LOAD r1, (#0, a)                ; r1 = point.x
LOAD r2, (#1, a)                ; r2 = point.y
LOAD r3, (#2, a)                ; r3 = point.z
```

14.7.2 Array Iteration

```
; Process array of 16-bit values
; 'a' = array base, r7 = count
loop:
    LOAD r1, (#0, a)+           ; Load element, advance
    ; ... process r1 ...
    SUBI r7, #1
    BRAN|!= @loop
```

14.7.3 Stack Operations

```
; Push registers
STOR -(#0, s), r1
STOR -(#0, s), r2
STOR -(#0, s), r3

; Pop registers (reverse order)
LOAD r3, (#0, s)+
LOAD r2, (#0, s)+
LOAD r1, (#0, s)+
```

14.8 Shift Operations with Memory Instructions

Register-offset memory forms of **LOAD** and **STOR** support optional shift operations. This feature allows data to be shifted during the load or store operation, which is particularly useful for bit manipulation and working with the CPU's word-addressed memory architecture.

14.8.1 LOAD with Shift

When loading from memory, the data read from memory is shifted **after** being loaded but **before** being written to the destination register:

```
LOAD r3, (r2, a), LSL #2        ; r3 = (memory[a + r2]) << 2
LOAD r5, (r2, a), ASL #3        ; r5 = (memory[a + r2]) << 3
LOAD r6, (r4, a), ASR #2        ; r6 = (memory[a + r4]) >> 2 (sign-extend)
```

14.8.2 STOR with Shift

When storing to memory, the source register value is shifted **before** being written to memory. The original register value is **not** modified:

```
STOR (r2, a), r1, LSL #1        ; memory[a + r2] = r1 << 1 (r1 unchanged)
STOR (r2, a), r1, ASL #2        ; memory[a + r2] = r1 << 2 (r1 unchanged)
STOR (r4, a), r1, ASR #4        ; memory[a + r4] = r1 >> 4 (r1 unchanged)
```

14.8.3 Supported Shift Types

All standard shift types are supported:

- **LSL** – Logical Shift Left (fill with zeros)
- **LSR** – Logical Shift Right (fill with zeros)
- **ASL** – Arithmetic Shift Left (same as LSL)
- **ASR** – Arithmetic Shift Right (sign-extend)
- **RTL** – Rotate Left within the operand
- **RTR** – Rotate Right within the operand

14.8.4 Shift Limitations

Important: Shift operations are **only** supported when using register-offset memory forms. The following modes do **not** support shifts:

- Loading immediate values: `LOAD r1, #0x1234` – No shift allowed
- Copying registers: `LOAD r1, r2` – No shift allowed
- Immediate-offset memory load: `LOAD r1, (#0, a)` – No shift allowed
- Immediate-offset memory store: `STOR (#0, a), r1` – No shift allowed

For unsupported shift forms, load or store the unshifted value and use an ALU instruction or `SHFT` for the shift operation.

14.8.5 Use Cases

Packed Data

Shifting during load/store can save cycles when extracting or placing fields in packed words:

```
; Load a packed word and move the high byte into the low byte position
LOAD r3, (r2, a), LSR #8
```

Bit Manipulation

Shifting during memory access enables efficient bit manipulation:

```
; Extract upper byte from memory
LOAD r1, (r3, a), LSR #8       ; r1 = (memory[a + r3] >> 8) & 0xFF

; Store scaled value
STOR (r3, a), r2, LSL #2       ; memory[a + r3] = r2 * 4
```

Combined with Auto-Increment/Decrement

Shifts work seamlessly with post-increment and pre-decrement modes:

```
; Load, shift, and advance pointer
LOAD r3, (r2, a)+, LSL #1      ; r3 = (memory[a + r2]) << 1; a += 1

; Decrement, shift, and store
STOR -(r2, a), r1, LSR #2      ; a -= 1; memory[a + r2] = r1 >> 2
```

14.8.6 Status Flags

Note: Unlike shifts in ALU instructions, shifts performed during **LOAD** and **STOR** operations can **not** update the status register flags. If you need flag updates based on a shift result, use an ALU instruction or the **SHFT** meta-instruction (see Chapter 17).

Chapter 15

Control Flow Instructions

15.1 Overview

Control flow in SIRC-1 is built from effective-address instructions. The real CPU opcodes compute an address into an address-register pair; jumps and calls are produced by targeting the program-counter pair `p`.

The SIRC-1 provides these control-flow and address-calculation operations:

- **`LDEA`** – Load Effective Address into an address-register pair
- **`LDEL`** – Load Effective Address and Link; also saves the return address in `l`
- **`BRAN`** – PC-relative assembler meta-instruction for `LDEA p, ...`
- **`BRSR`** – PC-relative assembler meta-instruction for `LDEL p, ...`
- **`RETS`** – Return meta-instruction for `LDEA p, (#0, l)`
- **`LJMP`** – Absolute-jump assembler meta-instruction for `LDEA p, ...`
- **`LJSR`** – Absolute-call assembler meta-instruction for `LDEL p, ...`

All encoded control-flow instructions can be conditional. If the condition is false, the instruction is skipped and has no side effects, as described in Chapter 11.

15.2 Legal Forms

Instruction	Syntax	Encoding	Notes
LDEA	`LDEA dest, (#offset, src)`	0x18	Immediate-offset effective address
LDEA	`LDEA dest, (rS, src)`	0x19	Register-offset effective address
LDEA	`LDEA dest, -(#offset, src)`	0x1A	Pre-decrement source, immediate offset
LDEA	`LDEA dest, -(rS, src)`	0x1B	Pre-decrement source, register offset
LDEL	`LDEL dest, (#offset, src)`	0x1C	Immediate-offset effective address and link
LDEL	`LDEL dest, (rS, src)`	0x1D	Register-offset effective address and link
LDEL	`LDEL dest, (#offset, src)+`	0x1E	Post-increment source, immediate offset
LDEL	`LDEL dest, (rS, src)+`	0x1F	Post-increment source, register offset
BRAN	`BRAN #disp`	`LDEA` meta	Assembles to `LDEA p, (#disp, p)`
BRAN	`BRAN @label`	`LDEA` meta	PC-relative linker-computed displacement
BRSR	`BRSR #disp`	`LDEL` meta	Assembles to `LDEL p, (#disp, p)`
BRSR	`BRSR @label`	`LDEL` meta	PC-relative linker-computed call
RETS	`RETS`	`LDEA` meta	Assembles to `LDEA p, (#0, l)`
LJMP	`LJMP src`	`LDEA` meta	Assembles to `LDEA p, (#0, src)`
LJMP	`LJMP src, #offset`	`LDEA` meta	Assembles to `LDEA p, (#offset, src)`
LJMP	`LJMP src, rS`	`LDEA` meta	Assembles to `LDEA p, (rS, src)`
LJSR	`LJSR src`	`LDEL` meta	Assembles to `LDEL p, (#0, src)`
LJSR	`LJSR src, #offset`	`LDEL` meta	Assembles to `LDEL p, (#offset, src)`
LJSR	`LJSR src, rS`	`LDEL` meta	Assembles to `LDEL p, (rS, src)`
LJSR	`LJSR (#offset, src)+`	`LDEL` meta	Assembles to `LDEL p, (#offset, src)+`
LJSR	`LJSR (rS, src)+`	`LDEL` meta	Assembles to `LDEL p, (rS, src)+`

Table 15.1: Legal Control-Flow and Effective-Address Forms

Address registers: `src` and `dest` are address-register pairs: `l`, `a`, `s`, or `p`. The branch meta-instructions always use `p` as both the source and destination address-register pair.

Zero displacement: Immediate-displacement forms may omit `#0`. Thus `LDEA dest, (src)`, `LDEA dest, -(src)`, `LDEL dest, (src)`, and `LDEL dest, (src)+` assemble as the matching `#0` forms. For non-auto-update absolute jumps and calls, use the meta-instruction syntax `LJMP src` or `LJSR src`; `LJMP (src)` and `LJSR (src)` are not public syntax. The post-increment call shorthand `LJSR (src)+` is legal and assembles as `LDEL p, (#0, src)+`.

Aliased register writes: Instructions that write the same address-register pair through more than one write-back path have architecturally undefined behavior. Avoid forms such as `LDEA a, -(#0, a)` and `LDEL l, (#0, l)+`. The assembler rejects known hazardous aliased forms.

Shift suffixes: Control-flow and effective-address displacements are not shifted. Register-offset forms do not accept shift suffixes in the assembler.

15.3 Common Semantics

Topic	Definition
Condition false	The instruction is skipped and has no side effects. The program counter advances to the next instruction normally.
Address calculation	The selected source address-register high word supplies the segment. The target low word is the selected source low word plus the immediate or register displacement.
Pre-decrement LDEA	The source low word is decremented by one word before the displacement is applied. The decremented source low word is written back.
Post-increment LDEL	The target address is calculated from the original source value. The source low word is incremented by one word in write-back.
Subroutine link	`LDEL`, `BRSR`, and `LJSR` save the normal next-instruction address in `l`.
Status flags	Preserved. Control-flow instructions do not support status update override syntax.
Timing	6 cycles, plus any instruction-fetch wait states. These instructions do not perform a data-memory bus access.
Exceptions	Effective-address calculation can raise segment-overflow faults when `SR.A` is set; see the notes below.
Privilege	In protected mode, address-register-pair write-back is allowed only when every written high word would remain unchanged. If any high word would change, the instruction raises a privilege-violation fault instead of completing write-back. Direct writes to high address registers also raise a privilege-violation fault.
Fault side effects	Program-counter, destination address-register, source auto-update, and link-register updates occur after condition evaluation. If a fault aborts execution before write-back, those effects do not complete.

Table 15.2: Common Control-Flow Instruction Semantics

Control-flow and effective-address instructions do not perform data-memory access and do not raise data bus, data bus-protection, or invalid-opcode faults in documented forms. Effective-address calculation can raise a segment-overflow fault when `SR.A` (Trap on Address Overflow) is set and the 16-bit low-word address calculation wraps. Instruction fetch can still raise the normal fetch faults before the instruction executes. If trace mode was enabled when the instruction began, an instruction-trace fault is raised after the instruction commits.

15.4 Branch Meta-Instructions

BRAN – Branch Meta-Instruction

Assembles to: `LDEA` with `p` as the source and destination
Syntax:

```
BRAN #disp                      ; p = p + disp
BRAN @label                     ; p = p + label offset
BRAN|cond #disp                 ; Conditional PC-relative branch
```

Operands: `#disp` is a 16-bit PC-relative word displacement. `@label` is resolved by the linker to a PC-relative displacement from the next instruction address.
Operation:

```
; Assembles to LDEA p, (#disp, p)
if condition:
    p = p + displacement
```

Description:
Branches relative to the current program counter. `BRAN` is an assembler meta-instruction, not a distinct CPU opcode. Use `LDEA p, (#offset, src)` or `LDEA p, (rS, src)` for non-PC-relative computed jumps.
Write-back: Inherits `LDEA` write-back to `p`.
Flags: Preserved.
Exceptions: Same as `LDEA`; may raise segment-overflow or protected-mode privilege faults.
Condition codes: If the condition is false, no branch occurs and status flags are preserved.
Timing: Same as `LDEA`: 6 cycles, plus instruction-fetch wait states.
Privilege: Same as `LDEA`. Protected-mode branches must preserve the high word of `p`.
Examples:

```
; Unconditional branch forward
BRAN #20                        ; Jump ahead 20 words

; Backward branch (loop)
loop:
    ; ... loop body ...
    BRAN|!= @loop               ; Jump back if not zero
```

BRSR – Branch to Subroutine Meta-Instruction

Assembles to: `LDEL` with `p` as the source and destination
Syntax:

```
BRSR #disp                      ; l = next PC; p = p + disp
BRSR @label                     ; l = next PC; p = p + label offset
BRSR|cond #disp                 ; Conditional PC-relative call
```

Operands: `#disp` is a 16-bit PC-relative word displacement. `@label` is resolved by the linker to a PC-relative displacement from the next instruction address.
Operation:

```
; Assembles to LDEL p, (#disp, p)
if condition:
    l = address_of_next_instruction
    p = p + displacement
```

Description:
Calls a PC-relative subroutine by saving the return address in `l` and branching to the computed target. Return with `RETS`, which assembles to `LDEA p, (#0, l)`.
Write-back: Inherits `LDEL` write-back to `l` and `p`.
Flags: Preserved.
Exceptions: Same as `LDEL`; may raise segment-overflow or protected-mode privilege faults.
Condition codes: If the condition is false, no call occurs, `l` is not written, and status flags are preserved.
Timing: Same as `LDEL`: 6 cycles, plus instruction-fetch wait states.
Privilege: Same as `LDEL`. Protected-mode calls must preserve the high words written to `l` and `p`.
Examples:

```
; PC-relative call
BRSR @function

; Return from the subroutine
RETS
```

15.5 Return Meta-Instruction

RETS – Return from Subroutine Meta-Instruction

Assembles to: `LDEA p, (#0, l)`
Syntax:

```
RETS                                ; p = l
RETS|cond                           ; Conditional return
```

Operands: None. The source address-register pair is implicitly `l`; the destination is implicitly `p`.
Operation:

```
; Assembles to LDEA p, (#0, l)
if condition:
    p = l
```

Description:
Returns from a subroutine by copying the link register to the program counter. Used after **`LJSR`** or **`BRSR`**.
Write-back: Inherits **`LDEA`** write-back to `p`.
Flags: Preserved.
Exceptions: Same as **`LDEA`**; may raise protected-mode privilege faults if the return would change `p`'s high word.
Condition codes: If the condition is false, no return occurs and status flags are preserved.
Timing: Same as **`LDEA`**: 6 cycles, plus instruction-fetch wait states.
Privilege: Available in protected mode and supervisor mode, subject to protected-mode address-register write-back restrictions.
Examples:

```
my_function:
    ; ... function body ...
    RETS                            ; Return to caller
```

15.6 Load Effective Address

LDEA – Load Effective Address

Opcodes: 0x18 (Immediate), 0x19 (Register), 0x1A (Pre-Dec Immediate), 0x1B (Pre-Dec Register)
Syntax:

```
LDEA dest, (#offset, src)      ; dest = src + offset
LDEA dest, (src)               ; equivalent to LDEA dest, (#0, src)
LDEA dest, (rS, src)           ; dest = src + rS
LDEA dest, -(#offset, src)     ; src -= 1; dest = src + offset
LDEA dest, -(src)              ; equivalent to LDEA dest, -(#0, src)
LDEA dest, -(rS, src)          ; src -= 1; dest = src + rS
LDEA|cond dest, (#offset, src)
```

Operands: `dest` and `src` are address-register pairs. Immediate forms use a 16-bit displacement. Register forms use `rS` as a displacement. Pre-decrement forms also update `src`.
Operation:

```
if condition:
    if pre_decrement:
        src.low = src.low - 1
    dest.high = src.high
    dest.low  = src.low + displacement
```

Description:
Computes an effective address and loads it into the destination address-register pair. This is useful for pointer arithmetic and for control flow when the destination is `p`. Pre-decrement forms also update the source address register pair.
Write-back: If the condition is true, writes the computed address to `dest`. Pre-decrement forms also write the decremented source address back to `src`.
Flags: Preserved.
Exceptions: May raise a segment-overflow fault during effective-address calculation when `SR.A` is set. In protected mode, raises a privilege-violation fault if write-back would change a written address-register high word.
Condition codes: If the condition is false, no destination or source address register is written and status flags are preserved.
Timing: 6 cycles, plus instruction-fetch wait states. No data-memory bus access is performed.
Privilege: Available in protected mode and supervisor mode, subject to protected-mode address-register write-back restrictions.
Examples:

```
; Pointer arithmetic
LDEA a, (#16, a)               ; a = a + 16

; Computed jump
LDEA p, (r1, a)                ; p = a + r1

; Pre-decrement source pointer and copy nearby address
LDEA a, -(#0, s)               ; s -= 1; a = s
```

LDEL – Load Effective Address and Link

Opcodes: 0x1C (Immediate), 0x1D (Register), 0x1E (Post-Inc Immediate), 0x1F (Post-Inc Register)
Syntax:

```
LDEL dest, (#offset, src)      ; l = next PC; dest = src + offset
LDEL dest, (src)               ; equivalent to LDEL dest, (#0, src)
LDEL dest, (rS, src)           ; l = next PC; dest = src + rS
LDEL dest, (#offset, src)+     ; l = next PC; dest = src + offset;
    src += 1
LDEL dest, (src)+              ; equivalent to LDEL dest, (#0, src)+
LDEL dest, (rS, src)+          ; l = next PC; dest = src + rS; src
    += 1
LDEL|cond dest, (#offset, src)
```

Operands: `dest` and `src` are address-register pairs. Immediate forms use a 16-bit displacement. Register forms use `rS` as a displacement. Post-increment forms also update `src`.
Operation:

```
if condition:
    l = address_of_next_instruction
    dest.high = src.high
    dest.low  = src.low + displacement
    if post_increment:
        src.low = src.low + 1
```

Description:
Computes an effective address, saves the return address to `l`, and writes the computed address to the destination address-register pair. It is the real link-writing primitive behind `BRSR` and `LJSR`.
Write-back: If the condition is true, writes the return address to `l` and the computed address to `dest`. Post-increment forms also write the incremented source address back to `src`.
Flags: Preserved.
Exceptions: May raise a segment-overflow fault during effective-address calculation when `SR.A` is set. In protected mode, raises a privilege-violation fault if write-back would change a written address-register high word.
Condition codes: If the condition is false, no destination, link, or source address register is written and status flags are preserved.
Timing: 6 cycles, plus instruction-fetch wait states. No data-memory bus access is performed.
Privilege: Available in protected mode and supervisor mode, subject to protected-mode address-register write-back restrictions.
Examples:

```
; Absolute call through a
LDEL p, (#0, a)                ; l = next PC; p = a

; Explicit destination form
LDEL a, (r1, s)                ; l = next PC; a = s + r1

; Post-increment source pointer after call target calculation
LDEL p, (#0, a)+               ; l = next PC; p = old a; a += 1
```

15.7 Long Jump Meta-Instructions

LJMP – Long Jump Meta-Instruction

Assembles to: `LDEA` with `p` as the destination
Syntax:

```
LJMP  src                       ; p = src
LJMP  src, #offset              ; p = src + offset
LJMP  src, rS                   ; p = src + rS
LJMP|cond src                   ; Conditional long jump
```

Operands: `src` is an address-register pair. Optional `#offset` or `rS` operands select immediate or register displacement forms.
Operation:

```
; Assembles to LDEA p, ...
if condition:
    p = src + displacement
```

Description:
Performs an absolute jump through an address register. `LJMP` is an assembler meta-instruction, not a separate CPU opcode.
Write-back: Inherits `LDEA` write-back to `p`.
Flags: Preserved.
Exceptions: Same as `LDEA`; may raise segment-overflow or protected-mode privilege faults.
Condition codes: If the condition is false, no jump occurs and status flags are preserved.
Timing: Same as `LDEA`: 6 cycles, plus instruction-fetch wait states.
Privilege: Same as `LDEA`. Protected-mode jumps must preserve the high word of `p`.

LJSR – Long Jump to Subroutine Meta-Instruction

Assembles to: `LDEL` with `p` as the destination
Syntax:

```
LJSR src                          ; l = next PC; p = src
LJSR src, #offset                 ; l = next PC; p = src + offset
LJSR src, rS                      ; l = next PC; p = src + rS
LJSR (#offset, src)+              ; l = next PC; p = src + offset; src
    += 1
LJSR (src)+                       ; equivalent to LJSR (#0, src)+
LJSR (rS, src)+                   ; l = next PC; p = src + rS; src += 1
LJSR|cond src                     ; Conditional long call
```

Operands: `src` is an address-register pair. Optional `#offset` or `rS` operands select immediate or register displacement forms. Post-increment forms also update `src`.
Operation:

```
; Assembles to LDEL p, ...
if condition:
    l = address_of_next_instruction
    p = src + displacement
    if post_increment:
        src.low = src.low + 1
```

Description:
Calls a subroutine through an address register. The return address is saved in `l`; return with `RETS`. Post-increment forms advance `src` after computing the target. Because `LJSR` implicitly writes `l` and `p`, post-increment source registers cannot be `l` or `p`.
Write-back: Inherits `LDEL` write-back to `l`, `p`, and any post-increment source register.
Flags: Preserved.
Exceptions: Same as `LDEL`; may raise segment-overflow or protected-mode privilege faults.
Condition codes: If the condition is false, no call occurs, `l` is not written, any post-increment source register is preserved, and status flags are preserved.
Timing: Same as `LDEL`: 6 cycles, plus instruction-fetch wait states.
Privilege: Same as `LDEL`. Protected-mode calls must preserve the high words written to `l` and `p`.
Examples:

```
; Call function at address in a
LOAD ah, #0x0040
LOAD al, #0x0000                  ; a = 0x00400000
LJSR a                            ; Call 0x00400000

; Call with offset
LJSR a, #function_offset          ; p = a + function_offset

; Call with register displacement
LOAD r1, #8
LJSR a, r1                        ; p = a + r1

; Call through a table entry and advance to the next entry
LJSR (#0, a)+                     ; p = old a; a += 1
LJSR (r1, a)+                     ; p = a + r1; a += 1
```

15.8 Control Flow Patterns

15.8.1 If-Then-Else

```
; if (r1 > r2) { ... } else { ... }
CMPR r1, r2
BRAN|<= @else_part
; Then part
; ...
BRAN @end
else_part:
; Else part
; ...
end:
```

15.8.2 While Loop

```
; while (r1 > 0) { ... }
while_start:
    CMPI r1, #0
    BRAN|<= @while_end
    ; Loop body
    ; ...
    BRAN @while_start
while_end:
```

15.8.3 Switch/Jump Table

```
; Switch on r1 (0-3)
LOAD r2, r1                    ; r2 = r1
ADDI r2, #0, LSL #1            ; r2 = (r2 << 1) = r2 * 2 (word size)
LDEA p, (r2, p)                ; Jump via table

jump_table:
    DW case0 - jump_table
    DW case1 - jump_table
    DW case2 - jump_table
    DW case3 - jump_table
```

Chapter 16

Coprocessor Instructions

16.1 Overview

The SIRC-1 CPU includes a coprocessor interface that allows delegation of certain operations to specialized coprocessors. The primary coprocessor is the Exception Unit (Coprocessor 1), which handles exceptions, interrupts, and privileged operations.

Coprocessor calls are ordinary conditional instructions. If the instruction's condition is true, the processing unit computes a command operand and writes it to the 16-bit pending coprocessor-command latch during write-back. The selected coprocessor then interprets the command value. If the selected coprocessor is not present in the CPU model or does not implement the requested operation, the exception unit raises an invalid-opcode fault so the operation can be emulated or rejected by software.

16.2 Legal Forms

Instruction	Syntax	Encoding	Notes
COPI	`COPI #imm16`	0x0F	16-bit coprocessor command operand
COPR	`COPR rS`	0x3F	Command operand is read from `rS`

Table 16.1: Legal Coprocessor Instruction Forms

The CPU encoding reserves register and shift fields inherited from the immediate and register instruction formats. The programmer-facing `COPI` and `COPR` syntax hides those fields; assemblers emit canonical values and coprocessors must not interpret them as destination-register writes. Opcode 0x2F has no public assembly syntax and is undocumented.

16.3 Common Semantics

Topic	Definition
Condition false	The instruction is skipped and has no side effects. The pending coprocessor command, registers, memory, address registers, program counter, and status flags are preserved.
Command operand	`COPI` uses the 16-bit immediate value. `COPR` uses its source register value as the command operand.
Command format	For standard coprocessor commands, bits 15–12 select the coprocessor ID, bits 11–8 select the coprocessor operation, and bits 7–0 are coprocessor-defined operand bits.
Write-back	The processing unit writes the command operand to the pending coprocessor-command latch. It does not write general-purpose registers, address registers, or data memory as part of the call. Hidden encoding fields are ignored.
Status flags	Preserved by the processing-unit coprocessor-call write-back path. A coprocessor operation may define its own later CPU-state effects; exception-unit effects are documented in Chapter 6.
Privilege	In protected mode, coprocessor-operation nibbles 0x0–0x7 are user-callable and 0x8–0xF are supervisor-only. Privilege checks only apply when the instruction condition is true.
Timing	The coprocessor-call instruction itself uses the normal 6 execution phases and performs no data-memory bus access. The selected coprocessor may perform follow-up work after the command is latched.
Exceptions	Supervisor-only operations can raise privilege-violation faults; missing coprocessors and unimplemented operations can raise invalid-opcode faults. See the notes below.
Fault side effects	If a fault is raised while dispatching the command, software must not depend on the contents of the pending command latch after the fault.

Table 16.2: Common Coprocessor Instruction Semantics

Coprocessor-call instructions do not perform data-memory access and do not raise data bus, data bus-protection, data alignment, or segment-overflow faults in documented forms. Instruction fetch can still raise the normal fetch faults before the call executes. If trace mode was enabled when

the instruction began, an instruction-trace fault is raised after the processing-unit call commits, unless an earlier fault prevents the instruction from completing. In protected mode, supervisor-only coprocessor operation nibbles are rejected before the command is dispatched. Missing coprocessors and unimplemented coprocessor operations are detected after the command has been latched for dispatch and raise invalid-opcode faults through the exception unit.

COPI/COPR – Coprocessor Call

Opcodes: 0x0F (Immediate), 0x3F (Register). Opcode 0x2F is undocumented.
Syntax:

```
COPI #opcode                    ; Call coprocessor with 16-bit command
COPR rS                         ; Call coprocessor with command in rS
```

Operands: `COPI` takes a 16-bit immediate command operand. `COPR` takes a general-purpose source register whose 16-bit value is used as the command operand. Hidden encoded register and shift fields are canonicalized by the assembler and are not programmer-visible operands.
Description:
Transfers a command to a coprocessor. For the standard command layout, the high nibble selects the coprocessor and the next nibble selects that coprocessor's operation. The remaining low byte is defined by the selected operation.
Operation:

```
if condition_true then
    pending_coprocessor_command = command_operand
endif
```

Write-back: Writes the command operand to the pending coprocessor-command latch.
Flags: Preserved by the coprocessor-call instruction.
Exceptions: May raise privilege-violation faults for supervisor-only operations in protected mode. May raise invalid-opcode faults for missing coprocessors and unimplemented coprocessor operations.
Condition codes: If the condition is false, no command is dispatched, the pending command latch is preserved, no coprocessor privilege or opcode checks are performed, and status flags are preserved.
Timing: 6 cycles, plus any instruction-fetch wait states.
Privilege: Available in protected mode and supervisor mode. In protected mode, coprocessor-operation nibbles 0x8–0xF are supervisor-only and raise privilege-violation faults when executed.
Examples:

```
; Wait for interrupt (Exception Unit)
COPI #0x1900                    ; Coprocessor 1, operation 0x9
; Prefer the WAIT meta-instruction for normal assembly code.

; Return from exception
COPI #0x1A00                    ; Coprocessor 1, operation 0xA
; Prefer the RETE meta-instruction for normal assembly code.

; User exception (system call)
COPI #0x1180                    ; Coprocessor 1, operation 0x1, vector 0x80
; Prefer EXCP #0x80 for normal assembly code.
```

16.4 Coprocessor Compatibility

Standard coprocessor IDs are stable across SIRC-1 models. A CPU model may omit an optional coprocessor, but it must not assign that coprocessor ID to an incompatible unit. If software calls an absent coprocessor or an operation that the selected coprocessor does not implement, the CPU raises an invalid-opcode fault.

ID	Name	Required	Operations	Compatibility rule
0x0	Processing Unit	Yes	Internal	Executes SIRCIS processing-unit instructions
0x1	Exception Unit	Yes	0x0–0xD	Handles reset, faults, interrupts, traps, and links
0x2	DMA Unit	Yes	0x8–0xA	Required standard DMA transfer unit
0x3	Integer Maths Unit	No	0x0–0x3	Optional standard integer multiply/divide unit
0x4–0xF	Reserved/model space	No	Model-defined	Reserved unless a CPU model documents the assignment

Table 16.3: Standard Coprocessor IDs

16.4.1 Model and Revision Policy

- Required coprocessors and documented operations must remain backward compatible within the SIRC-1 family.
- Optional standard coprocessors keep their assigned IDs even on models that omit them.
- Reserved coprocessor IDs, reserved operation nibbles, and reserved operand values must raise invalid-opcode faults when they are not implemented by the selected CPU model.
- Software may probe optional coprocessors by executing the documented operation and handling an invalid-opcode fault, or by using a model-specific capability table if one is provided by system firmware.
- Processing-unit undocumented opcodes are a separate case: they are accepted by the processing unit and have architecturally undefined effects unless a future CPU revision documents them.

16.5 Exception Unit (Coprocessor 1)

The Exception Unit handles:

- Faults (abort exceptions, highest priority)
- Hardware exceptions (interrupts, multiple priority levels)
- Software exceptions (traps)
- Privileged operations

16.5.1 Common Operations

Opcode	Mnemonic	Operation
0x1100-0x11FF	EXCP	User exception (trap) at vector 0x60-0xFF
0x1900	WAIT	Wait for interrupt
0x1A00	RETE	Return from exception
0x1B00	RSET	System reset
0x1C00-0x1C7F	ETFR	Transfer from exception link register
0x1D00-0x1D7F	ETTR	Transfer to exception link register

Table 16.4: Exception Unit Operations

16.6 Exception Unit Meta-Instructions

EXCP – User Exception Meta-Instruction

Assembles to: `COPI #0x1100 | vector`
Syntax: `EXCP #vector`
Operands: `#vector` is the low 8-bit user exception vector selector.
Description: Triggers a software exception at the specified user vector, normally 0x60–0xFF. Used by user-mode programs to invoke system calls or request supervisor-mode services.
Write-back: Inherits `COPI` command dispatch to the exception unit.
Flags: Preserved by the coprocessor-call instruction; exception entry effects are defined by the exception unit.
Exceptions: Dispatches a software exception through the exception unit.
Condition codes: If the condition is false, no trap is dispatched and status flags are preserved.
Timing: Same as `COPI`: 6 cycles for the coprocessor call, plus exception-unit follow-up work.
Privilege: User-callable in protected mode and supervisor mode.
Example:

```
1 LOAD r1, #'A'                    ; Character to print
2 EXCP #0x80                       ; Trap to OS print routine
```

WAIT – Wait for Interrupt Meta-Instruction

Assembles to: `COPI #0x1900`
Syntax: `WAIT`
Operands: None.
Description: Puts the CPU into wait state until an exception occurs. Only valid in supervisor mode.
Write-back: Inherits `COPI` command dispatch to the exception unit.
Flags: Preserved by the coprocessor-call instruction.
Exceptions: Raises a privilege-violation fault if executed in protected mode.
Condition codes: If the condition is false, the CPU does not enter wait state and status flags are preserved.
Timing: Same as `COPI`: 6 cycles before the wait-state request is dispatched.
Privilege: Supervisor mode only.

RETE – Return from Exception Meta-Instruction

Assembles to: `COPI #0x1A00`
Syntax: `RETE`
Operands: None.
Description: Returns from an exception handler by restoring the saved program counter and status register.
Write-back: Inherits `COPI` command dispatch to the exception unit.
Flags: Restored by the exception unit as part of exception return.
Exceptions: Raises a privilege-violation fault if executed in protected mode.
Condition codes: If the condition is false, no exception return occurs and current status flags are preserved.
Timing: Same as `COPI`: 6 cycles before exception-unit return processing.
Privilege: Supervisor mode only.

RSET – System Reset Meta-Instruction

Assembles to: `COPI #0x1B00`
Syntax: `RSET`
Operands: None.
Description: Performs a software reset of the processor by clearing the status register and jumping to the reset vector.
Write-back: Inherits `COPI` command dispatch to the exception unit.
Flags: Reset by the exception unit as part of reset processing.
Exceptions: Raises a privilege-violation fault if executed in protected mode.
Condition codes: If the condition is false, no reset occurs and current status flags are preserved.
Timing: Same as `COPI`: 6 cycles before reset processing begins.
Privilege: Supervisor mode only.

ETFR/ETTR – Exception Link Transfer Meta-Instructions

Assembles to: `COPI #0x1Cxy` or `COPI #0x1Dxy`
Syntax:

```
ETFR #n                          ; Transfer saved address and status
    to a and r7
ETFR a, #n                       ; Transfer saved address to a only
ETFR r7, #n                      ; Transfer saved status to r7 only
ETTR #n                          ; Transfer a and r7 to saved address
    and status
ETTR #n, a                       ; Transfer a to saved address only
ETTR #n, r7                      ; Transfer r7 to saved status only
```

Operands: `#n` selects an exception link register. Optional operands select whether to transfer the saved return address, saved status register, or both.
Description: `ETFR` copies saved exception state from an exception link register to CPU registers. `ETTR` copies CPU register state back to the selected exception link register before returning.
Write-back: `ETFR` writes selected saved state to `a`, `r7`, or both. `ETTR` writes `a`, `r7`, or both to selected saved exception state.
Flags: Preserved by the coprocessor-call instruction.
Exceptions: Raises a privilege-violation fault if executed in protected mode.
Condition codes: If the condition is false, no transfer occurs and status flags are preserved.
Timing: Same as `COPI`: 6 cycles before exception-unit transfer processing.
Privilege: Supervisor mode only.
Example:

```
alignment_fault_handler:
    ETFR a, #6                   ; Get faulting return address
    LOAD al, @fixed_address      ; Correct the address
    ETTR #6, a                   ; Write corrected address back
    RETE
```

See Chapter 6 for detailed documentation of exception handling and all exception coprocessor instructions.

16.7 DMA Unit (Coprocessor 2)

The required DMA unit provides bulk word transfers using the existing address register pairs and the general-purpose register file. It is intentionally simple: it has no descriptor table, hidden parameter registers, or autonomous background execution. When a DMA command is accepted, the processing unit stops fetching further instructions until the DMA command completes or faults.

DMA operations are supervisor-only because they perform memory access using supervisor-visible address state. In protected mode, they raise a privilege-violation fault before dispatch.

16.7.1 DMA Operations

Mnemonic	Command	Count	Registers	Operation
`DMAR`	0x2800 | operand	`#-7`–`#7`	`a`/`l`/`s`, `r1`–`r7`	Read memory into registers
`DMAW`	0x2900 | operand	`#-7`–`#7`	`a`/`l`/`s`, `r1`–`r7`	Write registers to memory
`DMAT`	0x2A00 | n	0–255	`a`, `l`, `r1`–`r7`	Copy memory at `a` to memory at `l`

Table 16.5: DMA Unit Operations

For `DMAR` and `DMAW`, the command operand byte is encoded as follows: bits 7–6 select the address register (`00` = `a`, `01` = `l`, `10` = `s`, `11` reserved), bit 5 selects direction (`0` = forward, `1` = backward), bits 2–0 hold the count magnitude, and bits 4–3 are reserved and written as zero. Negative counts are encoded as the backward direction bit plus the positive magnitude, not as a two's-complement count field. A zero count is encoded with the forward direction bit.

For all DMA operations, a count of zero is a no-op: no memory access occurs, registers are preserved, and address registers are not advanced. CPU status flags are preserved. If the condition is false, no DMA command is dispatched, no coprocessor privilege or opcode checks are performed, and all CPU state is preserved.

DMA memory reads and writes can raise data bus and bus-protection faults using DMA read/write bus access types. If an address-register low word overflows or underflows and `SR.A` is set, the DMA operation raises a segment-overflow fault; otherwise the low word wraps modulo 2^{16}. Partial side effects are possible if a fault occurs mid-transfer, and software must not assume that re-executing the same DMA instruction restarts the original operation.

DMAR – DMA Read To Registers Meta-Instruction

Assembles to: `COPI #0x2800 | operand`
Syntax: `DMAR addr, #n`
Operands: `addr` must be `a`, `l`, or `s`. `#n` is a literal count operand in the range `#-7`–`#7`. Counts outside that range are invalid DMA operations.
Description: Reads sequential words from the selected address register into `r1` through `rn`. Positive counts read from the current address and advance it. Negative counts first move the address register backward by the count magnitude, then read the ascending memory block from that new address.
Operation:

```
count = abs(n)
if n < 0:
    addr.low = addr.low - count
transfer = addr
for i in 1..count:
    r[i] = memory[transfer]
    transfer.low = transfer.low + 1
if n > 0:
    addr.low = addr.low + count
```

Write-back: A successful positive-count `DMAR` advances the selected address register by the count magnitude. A successful negative-count `DMAR` leaves the selected address register at its decremented start address. Address-register high words are not changed by the transfer increments or decrements.
Clobbers: Writes `r1` through `rn`.
Flags: Preserved.
Exceptions: Unsupported DMA operations, reserved operand encodings, and invalid counts raise invalid-opcode faults. In protected mode, raises a privilege-violation fault. DMA reads can raise data bus, bus-protection, and segment-overflow faults.
Condition codes: If the condition is false, no DMA command is dispatched and all CPU state is preserved.
Timing: The `COPI` command dispatch uses the normal 6 execution phases. The DMA unit then performs DMA read bus cycles and may add wait states through normal bus acknowledgement.
Privilege: Supervisor mode only.

DMAW – DMA Write From Registers Meta-Instruction

Assembles to: `COPI #0x2900 | operand`
Syntax: `DMAW addr, #n`
Operands: `addr` must be `a`, `l`, or `s`. `#n` is a literal count operand in the range `#-7`–`#7`. Counts outside that range are invalid DMA operations.
Description: Writes `r1` through `rn` to sequential words at the selected address register. Positive counts write from the current address and advance it. Negative counts first move the address register backward by the count magnitude, then write the ascending memory block from that new address. Register order always corresponds to ascending memory addresses, so `DMAW s, #-n` followed by `DMAR s, #n` restores `r1` through `rn`.
Operation:

```
count = abs(n)
if n < 0:
    addr.low = addr.low - count
transfer = addr
for i in 1..count:
    memory[transfer] = r[i]
    transfer.low = transfer.low + 1
if n > 0:
    addr.low = addr.low + count
```

Write-back: A successful positive-count **DMAW** advances the selected address register by the count magnitude. A successful negative-count **DMAW** leaves the selected address register at its decremented start address. Address-register high words are not changed by the transfer increments or decrements.
Clobbers: General-purpose registers are preserved.
Flags: Preserved.
Exceptions: Unsupported DMA operations, reserved operand encodings, and invalid counts raise invalid-opcode faults. In protected mode, raises a privilege-violation fault. DMA writes can raise data bus, bus-protection, and segment-overflow faults.
Condition codes: If the condition is false, no DMA command is dispatched and all CPU state is preserved.
Timing: The `COPI` command dispatch uses the normal 6 execution phases. The DMA unit then performs DMA write bus cycles and may add wait states through normal bus acknowledgement.
Privilege: Supervisor mode only.

DMAT – DMA Memory Transfer Meta-Instruction

Assembles to: `COPI #0x2A00 | n`
Syntax: `DMAT a, l, #n`
Operands: The source and destination address registers must be written explicitly as `a` and `l`. `#n` is an 8-bit literal count operand in the range 0–255. A count of zero is a no-op.
Description: Copies sequential words from the address in `a` to the address in `l`, using `r1`–`r7` as an internal transfer window in chunks of up to seven words.
Operation:

```
while remaining > 0:
    chunk = min(remaining, 7)
    read chunk words from memory[a] into r1..r[chunk]
    write chunk words from r1..r[chunk] to memory[l]
    a.low = a.low + chunk
    l.low = l.low + chunk
    remaining = remaining - chunk
```

Write-back: A successful non-zero `DMAT` advances `a` and `l` by the number of words transferred. Address-register high words are not changed by the transfer increments.
Clobbers: Clobbers `r1`–`r7`; after completion they contain the final transfer window.
Flags: Preserved.
Exceptions: Unsupported DMA operations and reserved operand encodings raise invalid-opcode faults. In protected mode, raises a privilege-violation fault. DMA reads and writes can raise data bus, bus-protection, and segment-overflow faults.
Fault side effects: Registers, memory, and address registers reflect the last completed word transfer if a fault occurs mid-transfer.
Condition codes: If the condition is false, no DMA command is dispatched and all CPU state is preserved.
Timing: The `COPI` command dispatch uses the normal 6 execution phases. The DMA unit then performs DMA read and write bus cycles and may add wait states through normal bus acknowledgement. Instruction fetch resumes only after the DMA operation completes or faults.
Privilege: Supervisor mode only.
Overlap: Overlapping source and destination ranges are architecturally undefined unless the source and destination addresses are identical.

16.8 Integer Maths Unit (Coprocessor 3)

The optional integer maths unit provides a stable multiply/divide convention. CPU models without the hardware raise an invalid-opcode fault for these commands, allowing supervisor software to emulate the operation and return as if the coprocessor call had completed.

Reference source listings for software emulation routines are supplied with the SIRC-1 distribution media. Those listings are examples of a supervisor compatibility handler, not additional instruction encodings.

16.8.1 Maths Operations

Mnemonic	Command	Inputs	Outputs	Operation
`MULU`	0x3000	`r1`, `r2`	`r1:r2`, `r3`	Unsigned 16 x 16 -> 32 multiply
`MULS`	0x3100	`r1`, `r2`	`r1:r2`, `r3`	Signed 16 x 16 -> 32 multiply
`DIVU`	0x3200	`r1:r2`, `r3`	`r1`, `r2`, `r3`	Unsigned divide
`DIVS`	0x3300	`r1:r2`, `r3`	`r1`, `r2`, `r3`	Signed divide

Table 16.6: Integer Maths Unit Operations

Bit	Name	Meaning
0	E	Any maths error
1	DZ	Divide by zero
2	OV	Quotient overflow
3–15	Reserved	Written as zero, ignore read

Table 16.7: Maths Status Word in `r3`

Integer maths operations preserve CPU status flags. Missing maths hardware and unsupported maths operations raise invalid-opcode faults. Divide by zero and quotient overflow are reported in `r3`, not as arithmetic exceptions. If the condition is false, no maths command is dispatched, no coprocessor opcode check is performed, and registers and status flags are preserved.

MULU/MULS – Integer Multiply Meta-Instructions

Assembles to: `COPI #0x3000` or `COPI #0x3100`
Syntax:

```
MULU                                ; Unsigned multiply
MULS                                ; Signed multiply
```

Operands: None in assembly syntax. The operations multiply `r1` by `r2`.
Description: Multiplies two 16-bit operands and writes the 32-bit product to `r1`:`r2`, high word first. **`MULU`** treats both inputs as unsigned. **`MULS`** treats both inputs as two's-complement signed values.
Operation:

```
MULU:
    product = unsigned(r1) * unsigned(r2)
    r1 = product[31:16]
    r2 = product[15:0]
    r3 = 0

MULS:
    product = signed(r1) * signed(r2)
    r1 = product[31:16]
    r2 = product[15:0]
    r3 = 0
```

Write-back: Writes `r1`, `r2`, and `r3`.
Flags: CPU status flags are preserved. `r3` is written as zero.
Exceptions: Missing maths hardware and unsupported multiply operations raise invalid-opcode faults. These operations perform no data-memory bus access.
Condition codes: If the condition is false, no maths command is dispatched and registers and status flags are preserved.
Timing: The **`COPI`** command dispatch uses the normal 6 execution phases. Hardware maths operations may take additional implementation-defined cycles before instruction fetch resumes.
Privilege: User-callable in protected mode and supervisor mode.

DIVU/DIVS – Integer Divide Meta-Instructions

Assembles to: `COPI #0x3200` or `COPI #0x3300`
Syntax:

```
DIVU                                ; Unsigned divide
DIVS                                ; Signed divide
```

Operands: None in assembly syntax. The dividend is `r1`:`r2`, high word first. The divisor is `r3`.
Description: Divides a 32-bit dividend by a 16-bit divisor. Successful division writes the remainder to `r1`, the quotient to `r2`, and zero to `r3`.
Operation:

```
if divisor is zero:
    r3 = E | DZ
else if quotient does not fit in 16 bits:
    r3 = E | OV
else:
    r1 = remainder
    r2 = quotient
    r3 = 0
```

Write-back: Successful division writes `r1`, `r2`, and `r3`. Divide-by-zero and quotient-overflow cases leave `r1`:`r2` unchanged and write the maths status word to `r3`.
Flags: CPU status flags are preserved. Maths errors are reported through `r3`, not through `sr`.
Exceptions: Missing maths hardware and unsupported divide operations raise invalid-opcode faults. Divide by zero and quotient overflow do not raise arithmetic exceptions. These operations perform no data-memory bus access.
Condition codes: If the condition is false, no maths command is dispatched and registers and status flags are preserved.
Timing: The `COPI` command dispatch uses the normal 6 execution phases. Hardware maths operations may take additional implementation-defined cycles before instruction fetch resumes. Software emulation timing is defined by the invalid-opcode handler.
Privilege: User-callable in protected mode and supervisor mode.
Signed division: `DIVS` treats `r1`:`r2` as a 32-bit two's-complement dividend and `r3` as a 16-bit two's-complement divisor. The quotient is truncated toward zero and the remainder has the sign of the dividend. The case `-2147483648 / -1` is quotient overflow.

Chapter 17

Meta-Instructions

17.1 Overview

Meta-instructions are assembly mnemonics that lower to ordinary SIRCIS instruction encodings. They exist to make common operations easier to read and write; they do not introduce additional CPU opcodes.

Meta-Instruction	Primary Documentation	Lowers To
`SHFT`	Chapter 13	`ORRI[S]`
`BRAN`	Chapter 15	`LDEA p, ...`
`BRSR`	Chapter 15	`LDEL p, ...`
`LJMP`	Chapter 15	`LDEA p, ...`
`LJSR`	Chapter 15	`LDEL p, ...`
`RETS`	Chapter 15	`LDEA p, (#0, l)`
`EXCP`	Chapter 16	`COPI`
`WAIT`	Chapter 16	`COPI #0x1900`
`RETE`	Chapter 16	`COPI #0x1A00`
`RSET`	Chapter 16	`COPI #0x1B00`
`ETFR`	Chapter 16	`COPI #0x1Cxy`
`ETTR`	Chapter 16	`COPI #0x1Dxy`
`DMAR`	Chapter 16	`COPI #0x28xx`
`DMAW`	Chapter 16	`COPI #0x29xx`
`DMAT`	Chapter 16	`COPI #0x2Ann`
`MULU`	Chapter 16	`COPI #0x3000`
`MULS`	Chapter 16	`COPI #0x3100`
`DIVU`	Chapter 16	`COPI #0x3200`
`DIVS`	Chapter 16	`COPI #0x3300`

Table 17.1: Meta-Instruction Cross-Reference

17.2 NOOP – No Operation

Syntax: `NOOP`

Assembles to: `ADDI[N] r1, #0`

Operands: None.

Operation: Executes an add-immediate-zero instruction that writes the original value of `r1` back to `r1` and preserves status flags.

Description: Does nothing for 6 cycles. Useful for timing delays, instruction padding, or placeholders during development.

Write-back: No architectural state changes.

Flags: Preserved.

Exceptions: Inherits `ADDI[N]` exception behavior. It does not access data memory and raises no instruction-specific faults; instruction-fetch faults can still occur before it executes, and trace faults can still occur after it commits.

Condition codes: If the condition is false, no state changes occur. Conditional `NOOP` is allowed but normally not useful.

Timing: Same as `ADDI`: 6 cycles, plus instruction-fetch wait states.

Privilege: Available in protected mode and supervisor mode.

Example:

```
1 NOOP                          ; Wait 6 cycles
2 NOOP
3 NOOP                          ; Total 18 cycles delay
```

Appendix A

Opcode Map

A.1 Complete Opcode Reference

This appendix provides a complete map of all 64 possible opcodes in the SIRCIS instruction set.

Hex	Binary	Mnemonic	Description
ALU Immediate (0x00–0x0F)			
00	000000	ADDI	Add Immediate
01	000001	ADCI	Add with Carry Immediate
02	000010	SUBI	Subtract Immediate
03	000011	SBCI	Subtract with Carry Immediate
04	000100	ANDI	AND Immediate
05	000101	ORRI	OR Immediate
06	000110	XORI	XOR Immediate
07	000111	LOAD	Load Immediate
08	001000	–	Undocumented
09	001001	–	Undocumented
0A	001010	CMPI	Compare Immediate
0B	001011	–	Undocumented
0C	001100	TSAI	Test AND Immediate
0D	001101	–	Undocumented
0E	001110	TSXI	Test XOR Immediate
0F	001111	COPI	Coprocessor Immediate
Memory and Control (0x10–0x1F)			
10	010000	STOR	Store Indirect Immediate
11	010001	STOR	Store Indirect Register
12	010010	STOR	Store Pre-Decrement Immediate
13	010011	STOR	Store Pre-Decrement Register
14	010100	LOAD	Load Indirect Immediate
15	010101	LOAD	Load Indirect Register
16	010110	LOAD	Load Post-Increment Immediate
17	010111	LOAD	Load Post-Increment Register
18	011000	LDEA	Load Effective Address Immediate
19	011001	LDEA	Load Effective Address Register
1A	011010	LDEA	Load Effective Address Pre-Decrement Immediate
1B	011011	LDEA	Load Effective Address Pre-Decrement Register
1C	011100	LDEL	Load Effective Address and Link Immediate
1D	011101	LDEL	Load Effective Address and Link Register
1E	011110	LDEL	Load Effective Address and Link Post-Increment Immediate
1F	011111	LDEL	Load Effective Address and Link Post-Increment Register
ALU Short Immediate (0x20–0x2F)			
20	100000	ADDI	Add Short Immediate
21	100001	ADCI	Add with Carry Short Immediate
22	100010	SUBI	Subtract Short Immediate
23	100011	SBCI	Subtract with Carry Short Immediate
24	100100	ANDI	AND Short Immediate
25	100101	ORRI	OR Short Immediate
26	100110	XORI	XOR Short Immediate
27	100111	LOAD	Undocumented
28	101000	–	Undocumented
29	101001	–	Undocumented
2A	101010	CMPI	Compare Short Immediate
2B	101011	–	Undocumented
2C	101100	TSAI	Test AND Short Immediate
2D	101101	–	Undocumented

A.2 Opcode Patterns

A.2.1 Format Identification

- Bits 5–4 = 00: Immediate format (0x00–0x0F)
- Bits 5–4 = 01: Memory/Control operations (0x10–0x1F)
- Bits 5–4 = 10: Short Immediate with Shift (0x20–0x2F)
- Bits 5–4 = 11: Register format (0x30–0x3F)

A.2.2 Save vs. Test

- Bit 3 = 0: Save result (0x00–0x07, 0x20–0x27, 0x30–0x37)
- Bit 3 = 1: Test only, discard result (0x08–0x0F, 0x28–0x2F, 0x38–0x3F)

A.2.3 Memory Operations Decoding

For opcodes 0x10–0x1F, bits encode memory operation type:

- Bit 4 = 1: Always set for memory/control section
- Bits 3–2: Operation type
  - 00 = Store
  - 01 = Load
  - 10 = Load Effective Address
  - 11 = Load EA with Link (jump to subroutine)
- Bit 1: Determines if both registers in address pair are updated
- Bit 0: Immediate (0) or Register (1) offset

A.3 Undocumented Instructions

The following opcodes are undocumented and should not be used in production code:

- 0x08, 0x09, 0x0B, 0x0D (Immediate format, test variants)
- 0x28, 0x29, 0x2B, 0x2D (Short Immediate format, test variants)
- 0x38, 0x39, 0x3B, 0x3D (Register format, test variants)

These opcodes may execute in hardware but their behavior is implementation-defined and may change between CPU revisions. See Appendix C for details.

A.4 Quick Lookup Table

Operation	Immediate	Short+Shift	Register
ADD	0x00	0x20	0x30
ADC	0x01	0x21	0x31
SUB	0x02	0x22	0x32
SBC	0x03	0x23	0x33
AND	0x04	0x24	0x34
OR	0x05	0x25	0x35
XOR	0x06	0x26	0x36
LOAD	0x07	0x27*	0x37
CMP	0x0A	0x2A	0x3A
TSA	0x0C	0x2C	0x3C
TSX	0x0E	0x2E	0x3E
COP	0x0F	0x2F*	0x3F

Table A.2: Instruction Variant Quick Reference

* These opcodes have no public assembly syntax and are undocumented.

Appendix B

Instruction Timing

B.1 Overview

All SIRCIS instructions follow the six-stage execution phase model and complete in 6 clock cycles when no bus wait states are required. A bus operation may hold the current phase until it is acknowledged, adding wait cycles without adding execution phases. This fixed execution sequence greatly simplifies software development and hardware design. See Chapter 2 for the bus signal timing, wait-state, and response-priority rules.

B.2 Six-Stage Execution Phases

For a detailed description of each phase, see Chapter 2.

1. **Instruction Fetch (High Word)** – 1 cycle
2. **Instruction Fetch (Low Word)** – 1 cycle
3. **Decode and Register Fetch** – 1 cycle
4. **Execute and Address Calculation** – 1 cycle
5. **Memory Access** – 1 cycle
6. **Write Back** – 1 cycle

B.3 Timing Characteristics

Instruction Type	Cycles
ALU (Register)	6
ALU (Immediate)	6
Load from Memory	6
Store to Memory	6
Branch (taken)	6
Branch (not taken)	6
Conditional (executed)	6
Conditional (not executed)	6
NOOP	6

Table B.1: Instruction Timing (All instructions)

The cycle counts in this appendix assume that every bus operation is acknowledged without a wait state.

B.4 Conditional Execution

When a conditional instruction's condition is not met:

- The instruction still takes 6 cycles to complete when no bus wait states are required
- All execution phases run normally
- The write-back stage is disabled (no register or memory modification)
- Status flags are not updated

This means conditional instructions do not branch and avoid control-flow stalls, but failed conditions still consume execution time.

B.5 Program Timing Examples

B.5.1 Simple Loop

```
; Process 10 elements (assuming no bus wait states)
ADDI r7, #0, #10              ; 6 cycles
loop:
    LOAD r1, (#0, a)+         ; 6 cycles
    ADDI r1, #1               ; 6 cycles
    STOR (#-2, a), r1         ; 6 cycles
    SUBI r7, #1               ; 6 cycles
    BRAN|!= @loop             ; 6 cycles
```

```
; Total per iteration: 30 cycles
; Total for 10 iterations: 6 + (10 * 30) = 306 cycles
```

B.5.2 Function Call Overhead

```
; Call function
LJSR a                          ; 6 cycles

; Function body executes...

; Return
RETS                            ; 6 cycles
; Total overhead: 12 cycles
```

B.5.3 Conditional Block

```
CMPR r1, r2                     ; 6 cycles
ADDR|HI r3, r4, r5              ; 6 cycles (exec if r1 > r2)
ADDR|LO r3, r6, r7              ; 6 cycles (exec if r1 <= r2)
; Total: 18 cycles (one conditional executes, one doesn't)
```

B.6 Performance Considerations

B.6.1 No Pipeline Stalls

The SIRC-1 does not implement pipelining in the traditional sense where multiple instructions are in flight simultaneously. Instead:

- Each instruction completes fully before the next begins
- No data hazards or phase bubbles
- No branch prediction penalties
- Base execution time is predictable before bus wait states are considered

B.6.2 Memory Access

- Instruction fetch, data memory access, and exception vector fetch each occupy one execution phase per 16-bit word transferred.
- If BACK is not asserted and neither BERR nor BPER aborts the request, the CPU holds that phase until the bus operation is acknowledged.
- Each wait state adds one clock cycle to the instruction

B.6.3 Optimization Guidelines

1. **Use conditional execution for short sequences**

```
; Instead of:
CMPR r1, r2
BRAN|<= @skip
ADDI r3, #1
skip:

; Use:
CMPR r1, r2
ADDI|HI r3, #1              ; Saves branch overhead
```

2. **Combine operations with shifts**

```
; Instead of:
ADDR r1, r2, r2
ADDI r1, #0, LSL #1

; Use:
ADDR r1, r2, r2, LSL #1     ; Multiply by 3 in one instruction
```

3. **Use post-increment/pre-decrement for iteration**

```
; Instead of:
LOAD r1, (#0, a)
ADDI a, #2

; Use:
LOAD r1, (#0, a)+           ; Same result, fewer cycles
```

4. **Loop unrolling reduces overhead**

```
; Instead of processing one item per iteration,
; process multiple items to reduce branch overhead
loop:
    LOAD r1, (#0, a)+
    LOAD r2, (#0, a)+
    LOAD r3, (#0, a)+
    LOAD r4, (#0, a)+
    ; Process 4 items...
    SUBI r7, #4
    BRAN|HI @loop
```

B.7 Theoretical Performance

At a given clock frequency:

- **Instructions per second:** $f_{clock}/6$
- **At 10 MHz:** $\approx$ 1.67 million instructions/second

- **At 25 MHz:** ≈ 4.17 million instructions/second

These are theoretical maximums assuming perfect code with no inefficiencies.

B.8 Comparison to Other CPUs

CPU	Avg CPI	Notes
SIRC-1	6.0	Fixed, all instructions
MOS 6502	2-7	Variable by instruction
Motorola 68000	4-158	Highly variable
ARM6	1.0	Single-cycle with pipeline
MIPS R2000	1.0	Single-cycle with pipeline

Table B.2: Cycles Per Instruction Comparison

The SIRC-1's fixed 6-cycle execution makes it slower than pipelined RISC CPUs but more predictable. It's comparable to early microprocessors while being simpler to implement in hardware.

Appendix C

Undocumented Instructions

C.1 Overview

The SIRCIS instruction set includes 14 undocumented instruction opcodes. These opcodes are valid encodings and will execute in hardware, but their architectural behaviour is undefined. Software must not depend on their results, side effects, timing, flag behaviour, or fault behaviour. Undocumented opcodes are reserved for future CPU revisions and may be repurposed without preserving their current simulator or hardware behaviour.

Warning: Use of undocumented instructions is not recommended for production code.

C.2 Undefined Operand Combinations

Undefined operand combinations are different from undocumented opcodes: the opcode is documented, but a particular choice of operands creates multiple architectural writes to the same address-register pair during the same instruction.

C.2.1 Aliased Address-Register Writes

The auto-update effective-address forms can write both a destination address-register pair and an updated source address-register pair. **LDEL** also writes the link-register pair. If two of those write-back targets are the same address-register pair, the final value is architecturally undefined.

```
1 ; Undefined: a is both destination and auto-updated source
2 LDEA a, -(#0, a)
3
4 ; Undefined: l is both link register and auto-updated source
5 LDEL p, (#0, l)+
6
7 ; Undefined: l is both link register and destination
8 LDEL l, (#0, a)
```

Programs should avoid aliased register writes. The assembler rejects the known hazardous forms so that programs do not accidentally depend on undefined write-back ordering.

C.3 Undocumented Opcode List

Opcode	Format	Pattern	Notes
0x08	Immediate	ADD-like + 0x8	Test variant (no save)
0x09	Immediate	ADC-like + 0x8	Test variant (no save)
0x0B	Immediate	SUB-like + 0x8	Test variant (no save)
0x0D	Immediate	SBC-like + 0x8	Test variant (no save)
0x27	Short Imm	LOAD-like	No public assembly syntax
0x28	Short Imm	ADD-like + 0x8	Test variant (no save)
0x29	Short Imm	ADC-like + 0x8	Test variant (no save)
0x2B	Short Imm	SUB-like + 0x8	Test variant (no save)
0x2D	Short Imm	SBC-like + 0x8	Test variant (no save)
0x2F	Short Imm	COP-like	No public assembly syntax
0x38	Register	ADD-like + 0x8	Test variant (no save)
0x39	Register	ADC-like + 0x8	Test variant (no save)
0x3B	Register	SUB-like + 0x8	Test variant (no save)
0x3D	Register	SBC-like + 0x8	Test variant (no save)

Table C.1: Undocumented Instruction Opcodes

C.4 Likely Behavior

Based on the systematic opcode organization, the undocumented opcodes with bit 3 set in the ALU operation field likely behave as "test" variants of arithmetic instructions:

C.4.1 0x08, 0x28, 0x38 (ADD Test)

Probably performs addition but discards the result, only updating flags. Similar to `CMP` but using ADD semantics instead of SUB.

```
; Hypothetical behavior
result = operand1 + operand2  ; Calculate
; result is discarded (not written to register)
; Flags updated based on result
```

Utility: Limited. `CMP` already provides comparison functionality.

C.4.2 0x09, 0x29, 0x39 (ADC Test)

Probably performs add-with-carry but discards the result, updating flags.

```
; Hypothetical behavior
result = operand1 + operand2 + C
; result is discarded
; Flags updated
```

Utility: Possibly useful for testing carry propagation without modifying registers.

C.4.3 0x0B, 0x2B, 0x3B (SUB Test)

Probably identical to `CMP` (which is already SUB without save).

Utility: None. Use `CMP` instead.

C.4.4 0x0D, 0x2D, 0x3D (SBC Test)

Probably performs subtract-with-borrow but discards the result.

```
; Hypothetical behavior
result = operand1 - operand2 - (1 - C)
; result is discarded
; Flags updated
```

Utility: Possibly useful for testing borrow propagation in multi-precision arithmetic.

C.5 Implementation Notes

C.5.1 Current Simulator

In the current SIRC-VM simulator (as of version 1.0):

- Undocumented instructions are recognized and decoded
- The undocumented ALU test-variant opcodes execute following the "test variant" pattern (no write-back)
- Flag updates may or may not be implemented correctly
- Behavior may change in future versions

C.5.2 Hardware Implementation

In a hardware implementation:

- The decoder treats bit 3 = 1 as "don't write back"
- ALU operations still execute normally
- Flags are still updated
- The write-back stage is simply disabled

This means undocumented encodings may execute, but their exact behavior depends on the hardware implementation details and is not part of the architectural contract.

C.6 Why Undocumented?

These instructions were left undocumented for several reasons:

1. **Limited utility**: Most have no practical use beyond documented instructions
2. **Incomplete semantics**: The designer hasn't fully defined their behavior
3. **Opcode space reservation**: May be used for future features
4. **Implementation flexibility**: Allows hardware optimizations

C.7 Recommendations

C.7.1 For Application Developers

Do not use undocumented instructions in production code.

- Behavior may change between CPU revisions
- Assemblers may reject them or emit warnings
- Code may not be portable
- Debugging tools may not recognize them

C.7.2 For Compiler Writers

- Do not generate undocumented instructions
- Use only documented instructions for all operations
- If optimization requires new operations, request formal documentation

C.7.3 For Hardware Implementers

- You may implement these opcodes following the logical pattern
- Document any implementation-specific behavior
- Consider reserving them for future standardization

C.7.4 For Researchers/Hackers

If you must experiment with undocumented instructions:

- Test thoroughly on your specific hardware/simulator version
- Document your findings
- Isolate usage in test code, not production
- Be prepared for behavior changes

C.8 Future Standardization

Some undocumented instructions may be formally specified in future SIRCIS revisions. Candidates for standardization include:

- **ADC Test (0x09/0x29/0x39)**: For carry flag testing in multi-precision
- **SBC Test (0x0D/0x2D/0x3D)**: For borrow flag testing in multi-precision

Others (SUB Test, which duplicates CMP) will likely remain undocumented or be repurposed.

C.9 Opcode Space for Expansion

The SIRCIS instruction set has limited expansion space:

- 14 undocumented opcodes (could be formally specified)
- Reserved shift type (0x111) could enable additional shift modes
- Reserved address register encoding could enable more address modes
- Standard optional coprocessors and model-specific coprocessors can provide extended functionality through the documented coprocessor compatibility rules

Any future expansion must maintain backward compatibility with existing code.

Appendix D

Example Programs

This appendix includes some sample programs to demonstrate how to use different features of the CPU.

D.1 Byte Sieve

Listing D.1: Byte sieve example

```
; https://rosettacode.org/wiki/Sieve_of_Eratosthenes

.EQU $MAX_PRIME #1000

; Reserved space for 128x32 bit exception vectors
.ORG 0x0000
.DQ @main

.ORG 0x0200
:main

; File I/O segment
LOAD    ah, #0x00F0
; Array Start = 0x00F0_0000
LOAD    al, #0x0000

; Stack segment (other end of mapped file)
LOAD    sh, #0x00F0
; Array Start = 0x00F0_0000
LOAD    sl, #0xFFFF

LOAD    r7, #2
LOAD    r1, #2
LOAD    r2, #1

:1

STOR    (r1, a), r2
ADDI    r1, #2
CMPI    r1, $MAX_PRIME
BRAN|<= @1
```

```
LOAD    r1, #3
LOAD    r3, #1

:2

LOAD    r2, (r1, a)
CMPI    r2, #1
BRAN|== @4
; THIS NUMBER IS A PRIME!
; TODO: Log this somehow
STOR    -(s), r1
LOAD    r7, r1
LOAD    r2, r1

:3
STOR    (r2, a), r3
ADDR    r2, r1
CMPI    r2, $MAX_PRIME
BRAN|<= @3

:4
ADDI    r1, #2
CMPI    r1, $MAX_PRIME
BRAN|<= @2

:DONE

; Halt CPU
COPI    #0x14FF
```

D.2 Faults

Listing D.2: Fault handling example

```
;; Segments
.EQU $PROGRAM_SEGMENT                #0x0000
.EQU $SCRATCH_SEGMENT                #0x0001
.EQU $PROGRAM_SEGMENT_HIGH           #0x0002

;; Devices
; Debug
.EQU $DEBUG_DEVICE_SEGMENT              #0x000B
.EQU $DEBUG_DEVICE_BUS_ERROR            #0x0000
.EQU $DEBUG_DEVICE_EXCEPTION_L5         #0x0005
.EQU $DEBUG_DEVICE_BUS_PROTECTION_ERROR #0x0006


;; Cpu Phases
.EQU $CPU_PHASE_INSTRUCTION_FETCH  #0x0000
.EQU $CPU_PHASE_EFFECTIVE_ADDRESS  #0x0003


```

```
; Reserved space for 128x32 bit exception vectors
.ORG 0x0000
.DQ @init
.ORG 0x0002
.DQ @bus_fault_handler
.ORG 0x0004
.DQ @alignment_fault_handler
.ORG 0x0006
.DQ @segment_overflow_fault_handler
.ORG 0x0008
.DQ @invalid_opcode_fault_handler
.ORG 0x000A
.DQ @privilege_violation_fault_handler
.ORG 0x000C
.DQ @instruction_trace_fault_handler
.ORG 0x000E
.DQ @level_five_interrupt_conflict_handler
.ORG 0x0010
.DQ @double_fault_handler
.ORG 0x0012
.DQ @bus_protection_fault_handler

.ORG 0x0020
.DQ @level_five_interrupt_handler

.ORG 0x0200
:init
LOAD    r1, #1
LOAD    r2, #1
LOAD    r3, #1
LOAD    r4, #1
LOAD    r5, #1
LOAD    r6, #1
LOAD    r7, #1

; Get the debug device to raise a fault on the bus
LOAD    ah, $DEBUG_DEVICE_SEGMENT
LOAD    al, $DEBUG_DEVICE_BUS_ERROR
STOR    (a), r1

; Jumping to an odd address triggers an alignment fault
LOAD    pl, #0x0201

:after_odd_address

; Loading/storing from an odd address should not trigger an alignment
    fault
LOAD    ah, $SCRATCH_SEGMENT
LOAD    al, #0x0
LOAD    r7, #0xCAFE
STOR    (#1, a), r7
LOAD    r7, (#1, a)


```

```
72 ; Calculating an effective address that overflows triggers a segment
       overflow fault
73 ; (but only if the SR is set up the right way)
74 LOAD     ah, $PROGRAM_SEGMENT
75 LOAD     al, #0xFFFE
76
77 ; This should not trigger fault
78 LOAD     r7, (#2, a)
79 ; Switch on trap on overflow bit
80 ORRI     sr, #0b0100_0000_0000_0000
81 ; This _should_ trigger a fault
82 LOAD     r7, (#2, a)
83 ; Switch off trap on overflow bit
84 ANDI     sr, #0b1011_1111_1111_1111
85
86 ; Segment overflow again, but this time the PC overflow causes it
87
88 LOAD     ah, $PROGRAM_SEGMENT_HIGH
89 LOAD     al, #0xFFFE
90
91 ; This should not trigger fault
92 LJMP     a
93
94 :return_from_testing_pc_overflow_no_fault
95
96 ; Switch on trap on overflow bit
97 ORRI     sr, #0b0100_0000_0000_0000
98
99 LOAD     ah, $PROGRAM_SEGMENT_HIGH
100 LOAD    al, #0xFFFE
101
102 ; This _should_ trigger a fault
103 LJMP    a
104
105 :return_from_testing_pc_overflow_with_fault
106
107 ; Switch off trap on overflow bit
108 ANDI    sr, #0b1011_1111_1111_1111
109
110 ; Instruction trace fault test
111 ; Set T bit - the next instruction will be traced and the handler will
       fire
112 ORRI    sr, #0b1000_0000_0000_0000
113 ; This instruction is traced
114 NOOP
115 ; T bit was cleared by the trace handler, so no further instructions are
       traced
116
117 ; Bus protection fault test
118 ; Get the debug device to raise a protection error on the bus
119 LOAD    ah, $DEBUG_DEVICE_SEGMENT
120 LOAD    al, $DEBUG_DEVICE_BUS_PROTECTION_ERROR
121 LOAD    r7, #0x1
122 STOR    (a), r7
```

```
123
124 ; There is no coprocessor at ID 4, so should trigger an invalid opcode
      fault
125 COPI     #0x4000
126
127 ; Switch to non privileged mode
128 ORRI      sr, #0b0000_0001_0000_0000
129 ; Try to escape the current segment
130 LOAD      ph, #0xFEFE
131
132
133 ; Get the debug device to raise an interrupt (this one should be fine)
134 LOAD     r7, #0x1
135 LOAD     ah, $DEBUG_DEVICE_SEGMENT
136 LOAD     al, $DEBUG_DEVICE_EXCEPTION_L5
137 STOR     (a), r7
138
139 ; Halt CPU
140 COPI     #0b0001_0100_1111_1111
141
142 .ORG 0x0300
143
144 :bus_fault_handler
145 ADDI     r1, #1
146 RETE
147
148 :bus_protection_fault_handler
149 ADDI     r5, #1
150 RETE
151
152 :alignment_fault_handler
153 ADDI     r2, #1
154
155 ; Fix up the return address because the original PC causes an alignment
      fault
156 ; Returning without fixing up the return address would cause an endless
      loop
157
158 ; Transfer current ELR into 'a' register
159 ETFR a, #6
160 ; Correct the lower word
161 LOAD     al, @after_odd_address
162 ; Transfer corrected address back to ELR
163 ETTR #6, a
164
165 RETE
166
167 :segment_overflow_fault_handler
168 ; Check the phase
169
170 ; Transfer exception metadata to r7
171 ETFR     r7, #7
172 ; Mask off the phase section of the status register
173 ANDI     r7, #0x7
```

```
174
175 CMPI    r7, $CPU_PHASE_INSTRUCTION_FETCH
176 BRAN|== @segment_overflow_fault_handler_pc
177 ; Mask off the phase section of the status register
178 ANDI    r7, #0x7
179
180 CMPI    r7, $CPU_PHASE_EFFECTIVE_ADDRESS
181 BRAN|== @segment_overflow_fault_handler_address
182
183 BRAN @segment_overflow_fault_handler_end
184
185 :segment_overflow_fault_handler_pc
186
187 ADDI    r3, #0x0F00
188
189 ; Fix up the return address because it isn't valid
190 ; Load address after fault
191 LOAD    ah, $PROGRAM_SEGMENT
192 LOAD    al, @return_from_testing_pc_overflow_with_fault
193 ; Transfer corrected address back to ELR
194 ETTR    #6, a
195
196 BRAN @segment_overflow_fault_handler_end
197
198 :segment_overflow_fault_handler_address
199
200 ADDI    r3, #0x000F
201
202 :segment_overflow_fault_handler_end
203
204 RETE
205
206 :invalid_opcode_fault_handler
207 ; A double fault will blat the EU link registers so we need to save them
      before we trigger it
208 ETFR #6
209
210 ADDI    r4, #1
211 ; Trigger another invalid opcode fault to test double fault handling
212 ; This will cause a fault while already handling a fault
213 COPI    #0xF000
214
215 ; Restore link register
216 ETTR #6
217
218 RETE
219
220 :privilege_violation_fault_handler
221 ADDI    r5, #1
222
223 ; Make sure that the CPU is back in privileged mode again after handling
      this fault
224
225 ; Transfer current ELR into 'r7' register
```

```
226 ETFR    r7, #6
227 ; Correct the lower word
228 ANDI    r7, #0xFEFF
229 ; Transfer corrected SR back to ELR
230 ETTR    #6, r7
231
232 RETE
233
234 :double_fault_handler
235 ; A double fault occurred - this is typically a last-chance handler
236 ; In this example, we'll increment r4 to show we were here
237 ; and then return (though in production you'd probably reset)
238 ADDI    r4, #1
239 RETE
240
241 :level_five_interrupt_conflict_handler
242 ADDI    r6, #0x1
243 RETE
244
245 :level_five_interrupt_handler
246 ADDI    r6, #0x1000
247
248 ; Trigger another L5 interupt while it is already being serviced
249 ; which will trigger a fault
250 LOAD    r7, #0x1
251 LOAD    ah, $DEBUG_DEVICE_SEGMENT
252 LOAD    al, $DEBUG_DEVICE_EXCEPTION_L5
253 STOR    (a), r7
254
255 RETE
256
257 :instruction_trace_fault_handler
258 ADDI    r6, #0x100
259 ; Clear T bit from the saved SR so RETE does not re-enable tracing
260 ETFR    r7, #6
261 ANDI    r7, #0x7FFF
262 ETTR    #6, r7
263 RETE
264
265 ; Trampoline for faults-high to come back
266 .ORG 0xFFF0
267 LOAD    pl, @return_from_testing_pc_overflow_no_fault
```

Listing D.3: High-address fault handling example

```
1 ;; Segments
2 .EQU $PROGRAM_SEGMENT                #0x0000
3 .EQU $SCRATCH_SEGMENT                #0x0001
4 .EQU $PROGRAM_SEGMENT_HIGH           #0x0002
5
6 ; Section to test program counter overflow
7 ; without trying to execute the vector table
8 ; that lives at 0x0000 in the main segment
9
```

```
10 ; Note: This is not linked together with the main program
11 ; This is compiled as a separate program and mapped as a separate
       segment
12
13 .ORG 0x0000
14
15 LOAD     ah, $PROGRAM_SEGMENT
16 LOAD     al, #0xFFF0
17 LJMP     a
18
19
20 ; TODO: Work out how to avoid padding this out
21 ; I had to pad it out because even though I said this segment has a
       length of 0xFFFF
22 ; I think the way files are mapped means that length is ignored
23 .ORG     0xFFFE
24 LOAD     r1, r1
```

D.3 Hardware Exception

Listing D.4: Hardware exception example

```
1  ; Reserved space for 128x32 bit exception vectors
2
3
4  .EQU $FALSE                          #0x0
5  .EQU $TRUE                           #0x1
6
7  ; When this character is typed, the program should exit
8  .EQU $EXPECTED_CHAR                  #0x61
9
10 ;; Segments
11 .EQU $PROGRAM_SEGMENT                #0x0000
12 .EQU $SCRATCH_SEGMENT                #0x0001
13
14 ;; Devices
15 ; Serial
16 .EQU $SERIAL_DEVICE_SEGMENT          #0x000A
17 .EQU $SERIAL_DEVICE_BAUD             #0x0000
18 .EQU $SERIAL_DEVICE_RECV_ENABLED     #0x0001
19 .EQU $SERIAL_DEVICE_RECV_PENDING     #0x0002
20 .EQU $SERIAL_DEVICE_RECV_DATA        #0x0003
21 .EQU $SERIAL_DEVICE_SEND_ENABLED     #0x0004
22 .EQU $SERIAL_DEVICE_SEND_PENDING     #0x0005
23 .EQU $SERIAL_DEVICE_SEND_DATA        #0x0006
24
25 ;; Scratch Variables
26 .EQU $MESSAGE_SEND_BASE              #0x0000
27 .EQU $MESSAGE_SEND_POINTER           #0x0001
28
29
30 ;; Exception Vectors
```

```
31 ; First vector is the reset vector
32 .ORG 0x0000
33 .DQ @start
34
35 .ORG 0x0080
36 .DQ @exception_handler_p3
37
38 ;;;;;;; MAIN CODE SECTION
39
40 .ORG 0x0200
41 :start
42
43 ; Enable all hardware interrupts (set bits 9-13 of SR)
44 ORRI sr, #0b0001_1110_0000_0000
45
46 ; Setup routines
47 BRSR @setup_serial
48 BRSR @print_help
49 ; Make sure this is after the initial help message print (see the
       subroutine for more info)
50 BRSR @enable_serial_recv
51
52 :wait_for_char
53 ; Wait for exception (will spin until interrupted)
54 WAIT
55
56 ; Pending byte should be in r7 after exeception handler runs
57 CMPI    r7, $EXPECTED_CHAR
58 BRAN|== @finish
59
60 BRAN @wait_for_char
61
62 :setup_serial
63
64 LOAD    r1, #9600
65 LOAD    ah, $SERIAL_DEVICE_SEGMENT
66 LOAD    al, $SERIAL_DEVICE_BAUD
67 STOR    (a), r1
68
69 RETS
70
71 :enable_serial_recv
72
73 ; When piping data to stdin, an EOF character gets sent which closes
       stdin
74 ; This means that we can't trigger any more interrupts manually after
       the data is piped
75 ; (stdin is currently the only way to externally trigger an interrupt)
76 ; If the data is coming in at the same type we are sending data out,
77 ; it ends up with the program gets stuck waiting for an interrupt and
       never finishes
78 ; TODO: Actually that shouldn't be a problem - better investigate this
       further (
```

```
79 ;   maybe the exception handler routines are conflicting with each other
       )
80
81 LOAD    r1, #0x1
82 LOAD    al, $SERIAL_DEVICE_RECV_ENABLED
83 STOR    (a), r1
84
85 RETS
86
87 :print_help
88
89 LOAD    r1, @help_message
90 BRAN    @print
91
92 :print_exit
93
94 LOAD    r1, @exit_message
95 BRAN    @print
96
97 :print
98
99 LOAD    ah, $SCRATCH_SEGMENT
100 LOAD   al, $MESSAGE_SEND_BASE
101 STOR   (a), r1
102
103 ; Align pointer
104 LOAD   r1, #1
105
106 LOAD   ah, $SCRATCH_SEGMENT
107 LOAD   al, $MESSAGE_SEND_POINTER
108 STOR   (a), r1
109
110 LOAD   r1, $TRUE
111 LOAD   ah, $SERIAL_DEVICE_SEGMENT
112 LOAD   al, $SERIAL_DEVICE_SEND_ENABLED
113 STOR   (a), r1
114
115 :wait_for_print_finish
116
117 LOAD   ah, $SERIAL_DEVICE_SEGMENT
118 LOAD   al, $SERIAL_DEVICE_SEND_ENABLED
119 LOAD   r1, (a)
120
121 CMPI   r1, $TRUE
122 BRAN|== @wait_for_print_finish
123
124 RETS
125
126 :finish
127
128 BRSR   @print_exit
129
130 ; Halt CPU
131 COPI   #0b0001_0100_1111_1111
```

```
132
133 ;;;;;;; EXCEPTION HANDLERS
134
135 .ORG 0x0400
136 :exception_handler_p3
137
138 ; Save the link register
139 LDEA s, (l)
140
141 BRSR @read_pending_byte
142 BRSR @write_pending_byte
143
144 ; Restore the link register
145 LDEA l, (s)
146
147 RETE
148
149 :read_pending_byte
150 ; Check if there is something we need to read
151 LOAD    ah, $SERIAL_DEVICE_SEGMENT
152 LOAD    al, $SERIAL_DEVICE_RECV_PENDING
153 LOAD    r1, (a)
154
155 ; Return early if not
156 CMPI    r1, $FALSE
157 RETS|==
158
159 ; Read pending byte
160 LOAD    al, $SERIAL_DEVICE_RECV_DATA
161 LOAD    r7, (a)
162
163 LOAD    al, $SERIAL_DEVICE_RECV_PENDING
164 LOAD    r1, $FALSE
165 STOR    (a), r1
166 RETS
167
168 :write_pending_byte
169 ; Check if the device is ready to write
170 LOAD    ah, $SERIAL_DEVICE_SEGMENT
171 LOAD    al, $SERIAL_DEVICE_SEND_PENDING
172 LOAD    r1, (a)
173
174 ; Return early if not (there is already a send pending)
175 CMPI    r1, $TRUE
176 RETS|==
177
178 LOAD    ah, $SCRATCH_SEGMENT
179 LOAD    al, $MESSAGE_SEND_POINTER
180 LOAD    r2, (a)
181 LOAD    al, $MESSAGE_SEND_BASE
182 LOAD    al, (a)
183
184 LOAD    ah, $PROGRAM_SEGMENT
185 LOAD    r3, (r2, a)
```

```
186
187 ; If the message buffer (zero terminated) has been sent. Stop sending
188 CMPI    r3, #0
189 BRAN|== @stop_send
190
191 LOAD    ah, $SERIAL_DEVICE_SEGMENT
192 LOAD    al, $SERIAL_DEVICE_SEND_DATA
193 STOR    (a), r3
194
195 ; Increment by two because of lack of packing (a word actually is padded
        to a DW)
196 ADDI    r2, #2
197 LOAD    ah, $SCRATCH_SEGMENT
198 LOAD    al, $MESSAGE_SEND_POINTER
199 STOR    (a), r2
200
201 LOAD    ah, $SERIAL_DEVICE_SEGMENT
202 LOAD    al, $SERIAL_DEVICE_SEND_PENDING
203 LOAD    r1, $TRUE
204 STOR    (a), r1
205
206 RETS
207
208 :stop_send
209 LOAD    ah, $SERIAL_DEVICE_SEGMENT
210 LOAD    al, $SERIAL_DEVICE_SEND_ENABLED
211 LOAD    r1, $FALSE
212 STOR    (a), r1
213
214 RETS
```

Listing D.5: Hardware exception data

```
1 ;;;;;;; DATA
2
3 ; TODO: A string data type that does this for us
4 :help_message
5 .DW #84
6 .DW #121
7 .DW #112
8 .DW #101
9 .DW #32
10 .DW #39
11 .DW #97
12 .DW #39
13 .DW #32
14 .DW #116
15 .DW #111
16 .DW #32
17 .DW #101
18 .DW #120
19 .DW #105
20 .DW #116
21 .DW #10
```

```
.DW #0

:exit_message
.DW #71
.DW #111
.DW #111
.DW #100
.DW #98
.DW #121
.DW #101
.DW #33
.DW #10
.DW #0
```

D.4 Software Exception

Listing D.6: Software exception example

```
; Reserved space for 128x32 bit exception vectors

; First vector is the reset vector
.ORG 0x0000
.DQ @start

; 64th vector (*2 words per vector)
.ORG 0x0080
.DQ @exception_handler
.DQ @exception_handler_that_should_be_ignored

.ORG 0x0200
:start

; Reset register
LOAD    r1, #1
; Trigger Software Exception
EXCP    #0x40

; r1 should be 0x0FF here after exception handler runs

; Halt CPU
COPI    #0x14FF

.ORG 0x0300
:exception_handler

; Load a value to detect if this code runs
; Final value of r1 should be this value
LOAD    r1, #0xFF
; Try and trigger nested software exception (should be ignored)
EXCP    #0x41
; Return from exception
RETE
```

```
35
36 .ORG 0x0500
37 :exception_handler_that_should_be_ignored
38
39 ; Load a value to detect if this code runs
40 ; Final value of r1 should _not_ be BB because this vector should never
       be jumped to
41 LOAD     r1, #0xBB
42 ; Return from exception
43 RETE
```

D.5 Store Load

Listing D.7: Store/load example

```
1 ; Reserved space for 128x32 bit exception vectors
2
3 .ORG 0x0000
4 .DQ @init
5
6 .ORG 0x0200
7
8 :init
9 LOAD     r1, #0xCAFE
10 LOAD     r3, #0xBEEF
11 LOAD     r5, #0x000A
12
13 LOAD     ah, #0x0008
14 LOAD     al, #0xAAF0
15
16 STOR     (#0x000F, a), r1
17 LOAD     r2, (#0x000F, a)
18 STOR     (r5, a), r3
19 LOAD     r4, (r5, a)
20
21
22 ; Lets test jumping!
23
24 ; TODO: Add some tests that:
25 ; LJMP to another segment
26 ; LJMP with offset
27 ; LJMP with shifted offset
28 ; Same three things but with LJSR
29 ; Make a maze of jumps so if the jump goes to the wrong place it will
       land in the forbidden zone where every instruction jumps to the fail
       label
30
31 ; r7 will store the test result
32 LOAD r7, #0
33 ; Use the stack address register pair to store fail address
34 LOAD sh, #0x0
35 LOAD sl, @fail
```

```
; Address register pair used as part of test
LOAD ah, #0x0
LOAD al, #0x0

BRAN #12              ; Step 1
LJMP s                ; FAIL!
LJMP s                ; FAIL!
BRAN #10              ; Step 3
LJMP s                ; FAIL!
LJMP s                ; FAIL!
BRAN #-6              ; Step 2
LJMP s                ; FAIL!
BRAN @ljmp_register_offset


:ljmp_register_offset
LOAD al, @ljmp_register_offset_anchor
LOAD r1, #-6
LOAD r2, #4
BRAN @ljmp_register_offset_anchor
LJMP s                ; FAIL!
LJMP s                ; FAIL!
LJMP s                ; FAIL!
LJMP a, r2
LJMP s                ; FAIL!
LJMP s                ; FAIL!
:ljmp_register_offset_anchor
LJMP a, r1
LJMP s                ; FAIL!
BRAN @ljmp_immediate_offset
LJMP s                ; FAIL!
LJMP s                ; FAIL!
LJMP s                ; FAIL!
LJMP s                ; FAIL!


:ljmp_immediate_offset
LOAD al, @ljmp_immediate_offset_anchor
BRAN @ljmp_immediate_offset_anchor
LJMP s                ; FAIL!
LJMP s                ; FAIL!
LJMP s                ; FAIL!
LJMP a, #4
LJMP s                ; FAIL!
LJMP s                ; FAIL!
:ljmp_immediate_offset_anchor
LJMP a, #-6
LJMP s                ; FAIL!
BRAN @ljsr_register_offset
LJMP s                ; FAIL!
LJMP s                ; FAIL!
LJMP s                ; FAIL!
LJMP s                ; FAIL!

```

```
90
91  :ljsr_register_offset
92  LOAD al, @ljsr_register_offset_anchor
93  LOAD r1, #-6
94  LOAD r2, #4
95  BRAN @ljsr_register_offset_anchor
96  LJSR s              ; FAIL!
97  LJSR s              ; FAIL!
98  LJSR s              ; FAIL!
99  LJSR a, r2
100 LJSR s              ; FAIL!
101 LJSR s              ; FAIL!
102 :ljsr_register_offset_anchor
103 LJSR a, r1
104 LJSR s              ; FAIL!
105 BRAN @ljsr_immediate_offset
106 LJSR s              ; FAIL!
107 LJSR s              ; FAIL!
108 LJSR s              ; FAIL!
109 LJSR s              ; FAIL!
110
111
112 :ljsr_immediate_offset
113 LOAD al, @ljsr_immediate_offset_anchor
114 BRAN @ljsr_immediate_offset_anchor
115 LJSR s              ; FAIL!
116 LJSR s              ; FAIL!
117 LJSR s              ; FAIL!
118 LJSR a, #4
119 LJSR s              ; FAIL!
120 LJSR s              ; FAIL!
121 :ljsr_immediate_offset_anchor
122 LJSR a, #-6
123 LJSR s              ; FAIL!
124 BRAN @finish
125 LJSR s              ; FAIL!
126 LJSR s              ; FAIL!
127 LJSR s              ; FAIL!
128 LJSR s              ; FAIL!
129
130
131 :finish
132 LOAD r7, #0x0FAB
133
134 LOAD sh, #0x0
135 LOAD sl, @pass
136 LJMP s
137
138 .ORG 0x0400
139
140 :fail
141 LOAD r7, #0xFA11
142
143 :pass
```

```
144
145 ; Halt CPU
146 COPI     #0x14FF
```